RESEARCH LEARNING HANDBOOK

视频生成知识手册

从表示与骨干，到训练、蒸馏与部署

覆盖范围

表示与架构	像素、潜空间与离散编码；卷积、Transformer、多模态与专家模型
生成原理	概率建模、扩散、流匹配、能量模型；连续动力学与数值采样
训练与蒸馏	数据管线、训练目标、训练加速、表示学习；一致性、轨迹与分布蒸馏
视频建模	时空注意力、条件控制、长视频、因果生成、流式与分辨率级联
后训练与评价	监督微调、偏好对齐、奖励模型、强化学习与视频能力评估
推理与部署	缓存、稀疏计算、低精度、算子优化、分布式并行与服务调度
研究延伸	生成 Agent、理解生成统一；语音、三维生成与世界模型

Jason

修订日期：2026 年 10 月 18 日

目录

阅读导引与符号约定 6

第一章 表示空间与视频压缩 8

1.1 像素、连续 latent 与音频 codec . . . 8
1.2 VAE 的条件后验、重参数化与 KL . . . 9
1.3 视频 VAE、patchify 与因果压缩 . . . 10
1.4 压缩率、token 预算与重建上限 . . . 12
1.5 重建、感知与时序损失 . . . 13
1.6 表征自编码器 RAE . . . 13
1.7 视频压缩器的具体实例与任务代价 . . . 14

第二章 网络骨干与条件接口 15

2.1 Attention、隐藏表示与任务输出头 . . . 15
2.2 U-Net、U-ViT、DiT 与多模态融合 . . . 15
2.3 MoE-DiT 与噪声阶段专家 . . . 19
2.4 时空注意力与三维位置编码 . . . 20
2.5 文本、参考图、结构与相机条件 . . . 20
2.6 产业模型的结构实例：单流、双分支与 MoE . . . 21

第三章 生成目标与训练估计 24

3.1 生成目标的统一视图 . . . 24
3.2 DDPM 的概率模型与噪声回归 . . . 24
3.3 Flow Matching：路径、速度与边缘分布 . . . 25
3.4 干净样本预测与损失参数化 . . . 27
3.5 噪声日程、SNR 与训练权重 . . . 28
3.6 Rectified Flow、Reflow 与 InstaFlow . . . 29
3.7 视觉自回归、离散码本与连续预测 . . . 30
3.8 对抗训练、偏好排序与判别器正则 . . . 30
3.9 能量模型、最大似然与 Langevin 采样 . . . 32

第四章 动力学与数值采样 34

4.1 SDE、ODE、score、drift 与 velocity . . . 34
4.2 预测转换、CFG 与采样成本 . . . 35

4.3 Euler 与 Heun 的数值积分 . . . 35
4.4 DDIM 与 DPM-Solver 系列 . . . 36
4.5 多步法、UniPC 与 DEIS . . . 37
4.6 采样误差、模型误差与近似计算的共同传播 . . . 38

第五章 视频时间建模与流式生成 39
5.1 自回归分解、因果可见性与训练历史 . . . 39
5.2 时间分块与分辨率级联 . . . 41
5.3 CausVid、Self Forcing 与 Causal Forcing . . . 43
5.4 SandAI MAGI-1：单调噪声与分块流水线 . . . 45
5.5 LongLive：长历史训练与局部监督 . . . 46
5.6 FramePack 与有限历史预算 . . . 46
5.7 分块接缝、归一化与参考 KV . . . 48
5.8 Vidu S1 与 Vivix：在线条件、长时稳定和交互 . . . 49
5.9 流式 refiner 与 SoL-Refiner . . . 50

第六章 训练配方与训练加速 53
6.1 视频数据管线与训练课程 . . . 53
6.2 基础训练循环与可变长度样本的梯度 . . . 54
6.3 各训练阶段的目标与监督接口 . . . 54
6.4 REPA 与特征辅助训练 . . . 56
6.5 LoRA、EMA 与目标分支的更新规则 . . . 56
6.6 训练显存：参数状态、激活与通信缓冲 . . . 56
6.7 训练吞吐、收敛速度与达到目标质量的成本 . . . 58
6.8 近期技术报告在体系中的位置 . . . 59
6.9 产业训练配方：数据、标注与系统协同 . . . 59

第七章 少步生成与一致性学习 61
7.1 Progressive Distillation、CM 与 LCM . . . 61
7.2 CTM 的两时刻一致性映射 . . . 61
7.3 PCM 的分段一致性 . . . 63
7.4 sCM 的连续时间一致性 . . . 64
7.5 rCM 的 score 规则化 . . . 65
7.6 Shortcut Models 的步长条件与自举 . . . 65
7.7 MeanFlow 的平均速度与训练恒等式 . . . 67
7.8 Flow Map 与轨迹映射 . . . 70

第八章 分布蒸馏与 on-policy 学习 73

8.1 On-policy 蒸馏、散度与教师接口 . . . 73
8.2 DMD / DMD2 的分布匹配与梯度 . . . 73
8.3 SiD 的 score identity 与 Fisher 目标 . . . 75
8.4 TDM 与采样步数相关的轨迹匹配 . . . 76
8.5 ADD、SDXL-Lightning 与 Hyper-SD . . . 77
8.6 OPSD-V 的特权历史与自蒸馏 . . . 78
8.7 AFD：成品比较到学生侧向量场更新 . . . 79
8.8 蒸馏配方在实际视频系统中的组合 . . . 80

第九章 后训练、奖励与强化学习 83
9.1 SFT 与直接偏好优化 . . . 83
9.2 ReFL、DRaFT 与可微奖励 . . . 84
9.3 VADER 与 VideoDPO . . . 84
9.4 VideoReward 与 Flow-DPO . . . 85
9.5 DDPO 的连续动作策略梯度 . . . 86
9.6 DanceGRPO 的组相对优化 . . . 86
9.7 Flow-GRPO：用随机转移建立连续策略密度 . . . 87
9.8 前向过程后训练：DiffusionNFT 与 DiffusionART . . . 88
9.9 Seedance 的多维奖励与后训练案例 . . . 90

第十章 推理效率与系统部署 91
10.1 特征缓存与刷新策略 . . . 91
10.2 Token 裁剪、合并与稀疏注意力 . . . 92
10.3 量化的对象、数制与误差预算 . . . 93
10.4 线性层量化：ViDiT-Q、SVDQuant 与时间敏感配置 . . . 95
10.5 SageAttention 系列：从 INT8 到 FP4 . . . 95
10.6 低精度训练：SageBwd 与 Attn-QAT . . . 97
10.7 Sparse-Linear Attention：稀疏与低秩的组合 . . . 98
10.8 相关稀疏路线与组合误差 . . . 100
10.9 算子融合、编译与 CUDA Graph 的执行模型 . . . 101
10.10 CUDA Graph：公开实机结果与可复现实验 . . . 104
10.11 Context Parallel 与 Sequence Parallel . . . 105
10.12 新一代 attention kernel 与容量路由 . . . 106
10.13 TurboDiffusion、TurboServe 与服务调度 . . . 106
10.14 分布式执行与服务案例：MagiAttention、MagiCompiler、Vivix . . . 108

第十一章 评价协议与研究方法 110

11.1 感知质量、偏好与视频评分 . . . 110
11.2 视频能力评价与实验面板 . . . 111
11.3 NFE、延迟、显存与收益量纲 . . . 111
11.4 实验设计、置信区间与总预算 . . . 112

第十二章 生成系统与跨模态前沿 114
12.1 生成 Agent 的规划、工具与反馈 . . . 114
12.2 理解生成统一与共享主干 . . . 116
12.3 音视频联合生成与推理驱动的生成接口 . . . 116
12.4 三维生成与动作条件世界模型 . . . 117
12.5 语音生成架构的对照 . . . 118
12.6 产业模型索引与公开资源 . . . 119

附录 A 术语与接口速查 121

一手资料与延伸阅读 122

阅读导引与符号约定

全书采用一条连续学习路径：表示空间 → 网络骨干 → 训练目标 → 动力学与采样 → 视频时间建模 → 训练工程 → 少步与分布蒸馏 → 后训练 → 推理部署与评价。

前四章建立变量、结构、目标和求解器的关系；第五至六章进入视频训练；第七至十章讨论少步学习、对齐和效率；最后两章组织评价与研究扩展。每章正文按概念和机制展开，章末练习提供推导、诊断与实验设计。

统一符号

符号	含义
X, x	像素图像/视频；部分沿用原论文的局部公式使用 x 表示一般连续状态
z_0, z_t	干净 latent 与噪声时间 t 下的 latent
c, ϵ, η	生成条件、采样噪声与辅助随机变量
t, s, r	去噪过程的时间；默认 0 为干净、1 为噪声，采样从 1 到 0
k, H_k	视频 chunk 序号与生成历史；与去噪时间分开
v_θ, s_θ	速度与 score；具体方法对 s 的其他用途在节内定义
u, Φ	配对路径速度或区间平均速度（按节定义）；两时刻流映射
$\theta, \bar{\theta}, \mathrm{sg}$	在线参数、目标/EMA 参数、停止梯度

公式以变量定义、计算图和边界条件一起阅读。教学数值例子用于推导；论文实验数字配有来源和比较口径。各机制图均为教学示意，实线表示前向数据，虚线表示条件或反馈，橙色点线表示梯度路径。

加速方法的定位

A：固定权重更换采样器；B：蒸馏已有教师；C：直接学习少步或跨时间映射；D：减少每次前向与系统开销；E：按时间或分辨率组织生成。这五类路线可组合，具体结果在质量–时间–费用三个轴上评价。

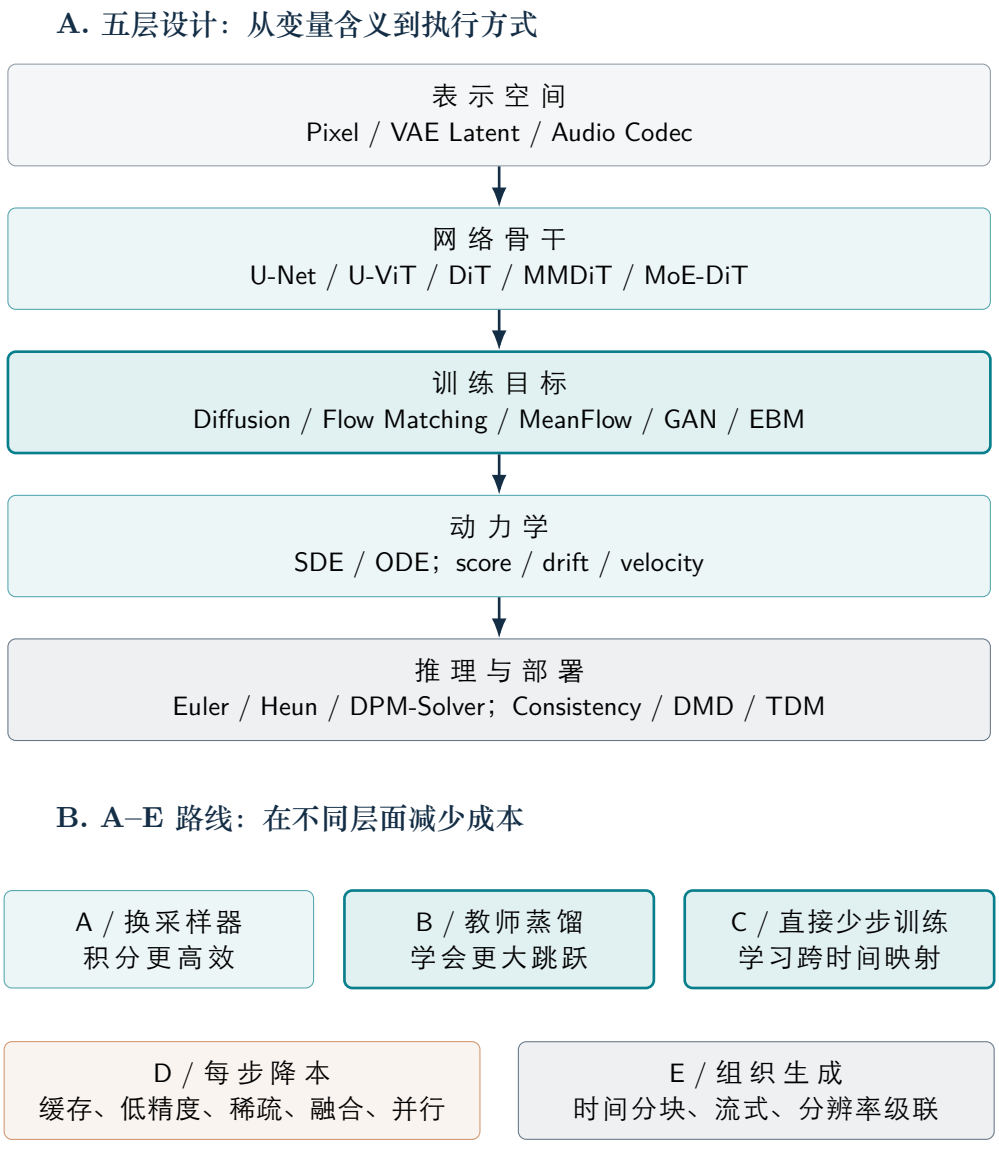


图 1　五层设计与 A–E 路线。表示、骨干、目标、动力学和执行策略共同构成一个生成系统。读图：先沿纵向确定系统配置，再查看各加速路线影响哪一层。[2, 5, 4, 80, 81]

第一章　表示空间与视频压缩

生成模型首先需要一个可操作的表示。本章从像素到 VAE latent，再到 patch 序列，明确分布、张量和压缩误差各自的含义。理解这些接口后，下一章讨论网络如何处理它们。

1.1　像素、连续 latent 与音频 codec

表示空间决定输出张量的含义、压缩率和解码方式。

Pixel：直接在像素上生成

把视频写成 $x \in \mathbb{R}^{T\times H\times W\times 3}$。即使文件用 8-bit 整数存储，模型训练时通常会把像素缩放为浮点数并加连续噪声。网络直接预测同形状的噪声或速度，最后把数值映射回颜色。优势是无需学习型解码器；成本集中在高维时空网格。若空间边长翻倍，像素数增至 4 倍；全局 attention 的配对数量可增至 16 倍。

VAE Latent：先压缩，再生成

$$z = E_\phi(x) \in \mathbb{R}^{T'\times H'\times W'\times C}, \qquad \hat{x} = D_\psi(z).$$

例如时间压缩 4 倍、空间各压缩 8 倍，位置数约缩小 $4\times 8\times 8 = 256$ 倍。通道数 C 通常增大，所以实际元素数、网络 FLOPs 和显存收益还取决于 C、patch 和骨干。连续 latent diffusion 在这个压缩空间学习分布。[2]

Audio Codec：压缩音频的表示接口

神经 codec 通常由编码器、量化器和解码器组成。以 residual vector quantization（RVQ）为例，逐层编码残差：

$$r_0 = E(x), \quad q_m = \arg\min_j \left\| r_{m-1} - e_j^{(m)} \right\|^2, \quad r_m = r_{m-1} - e_{q_m}^{(m)}.$$

解码器读取 $\sum_m e_{q_m}^{(m)}$。q_m 是离散码本索引；$e_{q_m}^{(m)}$ 是连续向量。生成器可以对索引做 AR 分类，也可以对连续声学表示做 flow / diffusion，再接波形解码器。EnCodec 是 RVQ 神经音频压缩的代表。[64]

表示	常见生成输出	主要代价
Pixel	连续颜色 / 噪声 / 速度	网格大，时空建模开销高
VAE Latent	连续 latent / 噪声 / 速度	编解码耗时与压缩失真
离散 Codec	每个码本的 logits / index	码率、量化失真、长序列
连续 Codec 特征	连续声学值或速度	后续解码与声学细节恢复

先问“模型生成的是索引还是向量”，再看输出头与 loss。attention 中的 token 既能承载码本 embedding，也能承载连续 latent patch。

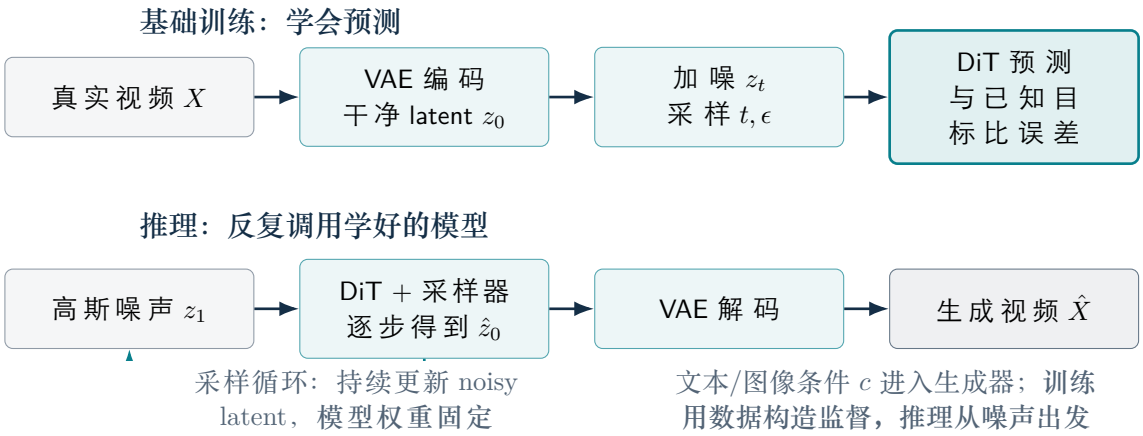


图 1.1　视频生成的训练与推理路径。压缩表示、条件特征、生成主干和采样器各自承担不同接口。基础训练以真实样本的随机加噪状态学习；推理从噪声反复更新。图为教学示意。[2, 5, 9]

1.2　VAE 的条件后验、重参数化与 KL

q 是编码器根据输入视频预测的条件概率分布。

设 X 是一段给定视频，Z 是压缩后的潜变量。VAE 编码器常常输出两个向量：

$$\mu_\phi(X), \qquad \log \sigma_\phi^2(X).$$

它们定义分布

$$\boxed{q_\phi(Z \mid X) = \mathcal{N}\big(\mu_\phi(X), \mathrm{diag}(\sigma_\phi^2(X))\big)\,.}$$

- q：近似后验分布的名称；字母 q 是常用的记号约定。
- 下标 ϕ：编码器的可训练参数。
- $Z \mid X$：已经看到视频 X 后，Z 可能取什么值。
- μ：分布中心；σ：标准差，控制采样波动。

一维条件分布示例

假设编码器看到一段视频后输出 $\mu = 2, \sigma = 0.5$。这代表 $Z \sim \mathcal{N}(2, 0.25)$，分布中心为 2。采到的值可能是 1.8、2.1 等；若使用 posterior mean，则确定性地取 2。大模型只是把这个例子推广到很多维。训练常用重参数化：

$$\eta \sim \mathcal{N}(0, I), \qquad Z = \mu_\phi(X) + \sigma_\phi(X) \odot \eta.$$

随机性放在与参数无关的 η 上，梯度仍可传入 μ 和 σ。

近似后验与 ELBO

生成模型定义先验 $p(Z)$ 和解码器 $p_\psi(X \mid Z)$。精确后验 $p_\psi(Z \mid X)$ 往往难求，因此用编码器 q_ϕ 近似它。训练时常最小化负 ELBO：

$$L_{\mathrm{VAE}} = \mathbb{E}_{Z \sim q_\phi}[-\log p_\psi(X \mid Z)] + D_{\mathrm{KL}}(q_\phi(Z \mid X) \| p(Z)).$$

前一项让压缩信息能重建 X，后一项约束 latent 的分布结构。实际视觉 VAE 常在这一概率目标上加入感知和对抗损失。[1, 2]

三个“q/Q”按语境分别理解：VAE 的 q_ϕ 是概率分布；attention 的 Q 是 query 特征矩阵；某篇蒸馏论文的 q 也可能表示学生分布。**阅读时先确定这个符号指向的对象。**

KL 的闭式与信息保留

对角高斯与标准高斯之间的 KL 有闭式：

$$\boxed{D_{\mathrm{KL}}(q_\phi(Z \mid X)\|\mathcal{N}(0,I)) = \frac{1}{2}\sum_j \left(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\right).}$$

逐项读：μ_j^2 不鼓励均值离零太远；$\sigma_j^2 - \log\sigma_j^2 - 1$ 在 $\sigma_j^2 = 1$ 时最小，既不喜欢方差无限大，也不喜欢完全塌缩为零。

KL 的数值示例

均值	方差	一维 KL	含义
0	1	0	正好是标准高斯
2	1	2	中心偏离较大
0	0.25	≈ 0.318	过于集中，也有惩罚

第二个例子：$\frac{1}{2}(4 + 1 - \log 1 - 1) = 2$。第三个例子：$\frac{1}{2}(0 + 0.25 - \log 0.25 - 1) \approx 0.318$。KL 同时取决于均值和方差。

信息保留与 posterior collapse

如果只最小化 KL，编码器确实可能对所有视频都输出几乎一样的分布，导致 posterior collapse。实际目标兼顾重建：

$$L = \lambda_{\mathrm{rec}} L_{\mathrm{rec}} + \lambda_{\mathrm{perc}} L_{\mathrm{perc}} + \beta L_{\mathrm{KL}} + \lambda_{\mathrm{adv}} L_{\mathrm{adv}}.$$

这里展示常见目标组合的结构。β 大小、warm-up、latent 维度和 decoder 能力都会影响平衡。

VAE 先验与扩散噪声边界

1. VAE KL 里的标准高斯：是潜空间规则化的先验。
2. diffusion 的起始高斯噪声：是生成过程的边界分布。

二者承担不同角色。视觉自编码器常使用较弱的 KL 约束，在保留内容的同时整理 latent 空间；扩散主干继续学习其中复杂的干净数据分布。[1, 2]

重建项保留信息，KL 整理分布形状。

1.3 视频 VAE、patchify 与因果压缩

常见 latent DiT 先 VAE 编码，再对 latent 做 patchify；解码时反过来。

$$X \xrightarrow{\text{VAE encoder}} z \xrightarrow{\text{patchify+projection}} h \xrightarrow{\text{DiT}} h' \xrightarrow{\text{output head+unpatchify}} \hat{z} \xrightarrow{\text{VAE decoder}} \hat{X}.$$

噪声通常加在 latent 上；图中省略了扩散时间和采样循环。

形状例子，所有尺寸仅为教学假设

假设输入 $X \in \mathbb{R}^{B\times 3\times 64\times 512\times 512}$，VAE 在时间上压缩 4 倍、空间上各压缩 8 倍，latent 通道数为 16，则大致变成 $[B, 16, 16, 64, 64]$。真实因果 VAE 的首帧处理会使时间长度公式有所不同。若 latent

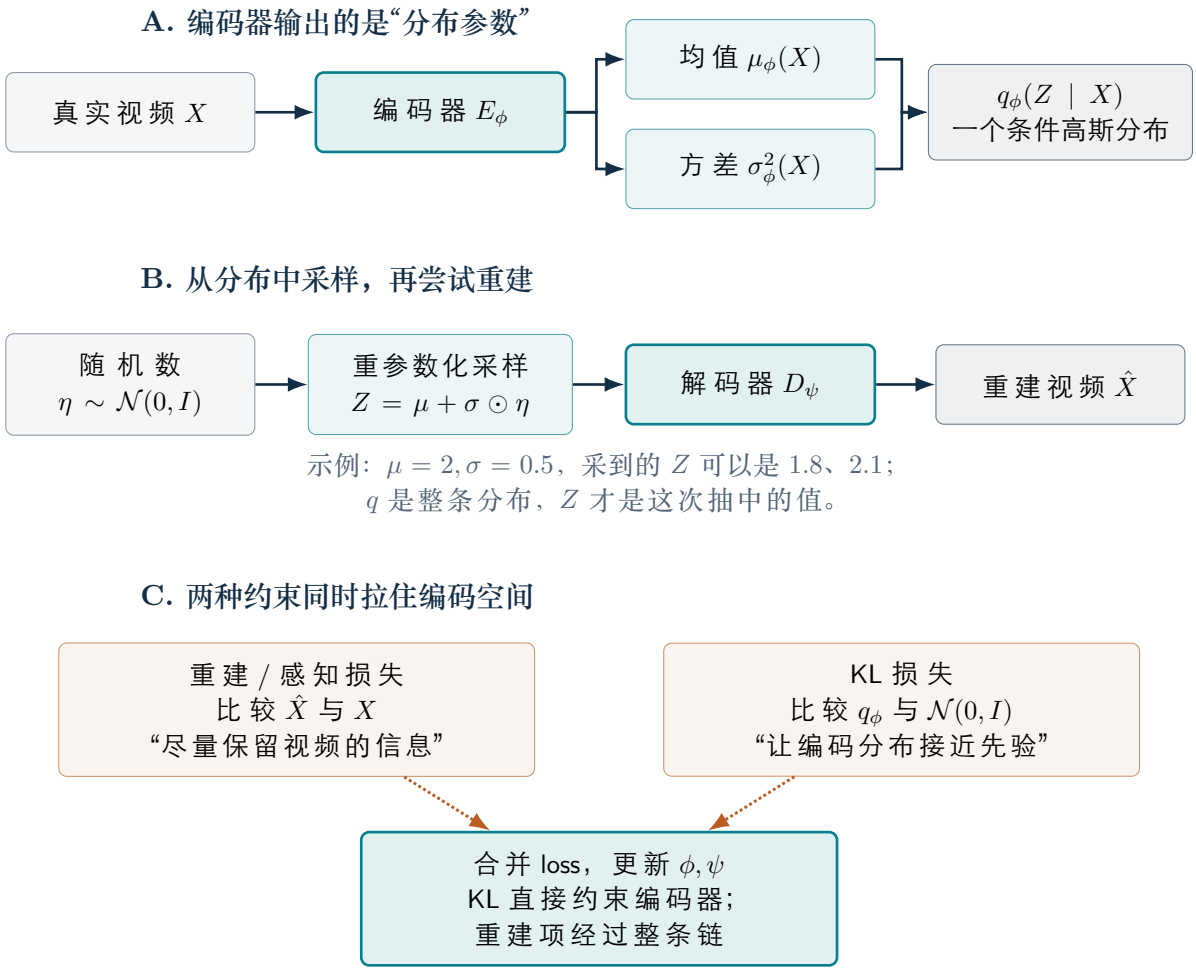


图 1.2 VAE：从一段视频到一个条件分布。先区分“分布参数”“抽中的 latent”“重建的视频”这三个对象。读图顺序：A 编码，B 采样与重建，C 看两种约束。编码器输出的参数定义 q_ϕ 分布，随后从中采样 latent。对角高斯 KL 可直接用闭式计算。[1, 2]

patch 大小为 $(p_t, p_h, p_w) = (1, 2, 2)$：

$$N = 16 \times 32 \times 32 = 16384, \qquad d_{\text{patch}} = 16 \times 1 \times 2 \times 2 = 64.$$

把每个 64 维小块线性投影到模型隐藏维度 d，得到 $[B, N, d]$。**reshape/切块本身无参数，patch embedding 的 Linear/Conv 有参数。**输出 head 把隐藏向量映射回一个 patch 所需的数值，unpatchify 再拼成 latent 张量。[5, 9]

VAE 的时间感受野与在线编码

视频 VAE 可用时空卷积、残差块、上下采样与 attention。非因果时间卷积可能使用未来帧；因果卷积只使用过去与当前，例如

$$y_k = \sum_{i=0}^{r} W_i x_{k-i}.$$

流式解码需要保存卷积边界状态，并保证 padding、时间下采样和归一化不会偷看未来。**整个系统的因果性需要 DiT、VAE、padding 与归一化共同满足。**

理解模型与生成模型的 patch 接口

二者都把局部视觉内容转成向量。理解型 VLM 往往只需语义特征，并不训练像素重建；生成模型的输出必须能还原成 latent/像素，所以需要对应的输出 head 和 decoder。二者都可使用连续 patch 特征，主要差别在**训练目标和输出链路**。

1.4 压缩率、token 预算与重建上限

用 $X \in \mathbb{R}^{T \times H \times W \times 3}$ 表示视频。设 VAE 的时间和空间压缩倍率分别为 s_t, s_h, s_w，latent 通道数为 C；忽略首帧特例时，其大小约为

$$T_z H_z W_z C \approx \frac{T}{s_t} \frac{H}{s_h} \frac{W}{s_w} C.$$

为避免记号混淆，实际实现直接记录 (T_z, H_z, W_z, C) 四个尺寸。再以 (p_t, p_h, p_w) 做 patchify，token 数为

$$N = \frac{T_z H_z W_z}{p_t p_h p_w}, \qquad d_{\text{patch}} = C p_t p_h p_w.$$

切分和 reshape 本身只重排元素；$d_{\text{patch}} \to d$ 的输入投影有可训练参数。输出投影恢复 d_{patch} 维，unpatchify 把它放回 latent 网格。

一个完整的预算例子

假设 $T_z = 20, H_z = 64, W_z = 112, C = 16$，patch 为 $(1, 2, 2)$，则 $N = 35840$、$d_{\text{patch}} = 64$。把空间 patch 改为 $(1, 4, 4)$，N 降为 8960：token 数缩小 4 倍，密集 attention 的配对数约缩小 16 倍。输入头每个 token 的维度同时增大到 256，细粒度信息压进更少的隐藏向量；质量变化仍需训练验证。

压缩器确定生成器能够恢复的细节

先测 $X \to \text{Enc}(X) \to \text{Dec}(Z)$ 的重建误差，再测生成质量。若细小文字、快速运动或手部结构在重建时已丢失，后续 DiT 只能依据统计规律补全这些细节。更强压缩可以节约主干计算，却可能增加解码难度、运动混叠和细节幻觉。空间压缩与时间压缩应分别消融。

1.5 重建、感知与时序损失

像素误差问“数字差多少”，感知误差问“特征看起来差多少”。

给定生成画面 $\hat{X}$ 和参考画面 X，设共有 N 个数：

$$L_1 = \frac{1}{N}\sum_i |\hat{X}_i - X_i|, \qquad L_2 = \frac{1}{N}\sum_i (\hat{X}_i - X_i)^2.$$

L2 更重罚大误差。若同一条件下存在多个合理答案，逐像素回归可能趋向条件均值，造成模糊；L2 的表现取决于监督目标和样本配对关系。

LPIPS 的核心公式

令 $\hat{f}_l(X)_{hw}$ 为冻结感知网络第 l 层经过通道归一化的特征：

$$L_{\text{LPIPS}}(\hat{X}, X) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot [\hat{f}_l(\hat{X})_{hw} - \hat{f}_l(X)_{hw}] \right\|_2^2.$$

先提取多层特征，再比较距离；w_l 是感知距离的通道权重。训练生成器时通常冻结 LPIPS 网络的参数，**但不切断它对输入图像的梯度**。[11]

视频中的逐帧与时序监督

最直接的是逐帧平均：$L = \frac{1}{T}\sum_k \text{LPIPS}(\hat{X}_k, X_k)$。它适合有对齐参考的重建、恢复或 refiner 训练。逐帧 LPIPS 衡量画面相似度，时序稳定性可另外用运动对齐损失衡量，例如：

$$L_{\text{temp}} = \sum_k \left\| M_k \odot \left[\hat{X}_{k+1} - \text{warp}(\hat{X}_k, F_k)\right] \right\|_1.$$

这里的 $\|\cdot\|_1$ 对像素与通道求和，也可按有效像素数归一化。**这是示意性的光流一致性约束**；M_k 要处理遮挡，光流也可能出错。通过运动补偿比较对应内容，可以保留正常运动，避免把静止画面当成最优解。

配对重建与自由生成分别选择监督

提示词“一只狗在跑”有无数正确视频，拿某一条真视频和自由生成结果逐像素/逐帧 LPIPS 对齐，可能惩罚合理的变化。可用配对监督的地方，与需要分布监督或偏好监督的地方，要分开。

“L2 loss”可能指预测误差 $\left\|\hat{X} - X\right\|^2$，也可能指参数正则 $\|\theta\|^2$。看到 L2 先问：**对谁取范数?**

1.6 表征自编码器 RAE

RAE：让预训练视觉表征成为可生成的 latent

Representation Autoencoder 使用预训练视觉编码器配合训练的解码器：[103]

$$z = E_{\text{frozen}}(x), \qquad \hat{x} = D_\psi(z), \qquad z_t = (1-t)z + t\epsilon.$$

第一阶段学会把表征重建成图像，第二阶段在这种表征中训练生成器。高维、强语义特征改变了噪声尺度和去噪网络设计；RAE 研究高维 latent 的训练与宽输出头。2026 年的文本到图像扩展进一步研究模型规模、数据和随维度调整的噪声日程。[104] RAE 改变生成变量的表示空间；第六章的 REPA 在保留既有 latent 接口时增加特征监督。

1.7 视频压缩器的具体实例与任务代价

MAGI-1 以因果视频压缩器连接流式像素与 latent。时间卷积状态能随块复用，空间切块则需要处理重叠与融合；即使主干因果，编码/解码的未来帧依赖也会影响响应延迟。[158]

MagiHuman 采用 Wan2.2 VAE 编码，并以重新训练的轻量 Turbo VAE decoder 加速解码。主干缩到少步后，解码占比增加，所以 decoder 可以成为独立的优化对象。其 latent 超分避免中间先解码再编码的往返。[161]

MiniMax H3 强调 H3-VAE 的高压缩和可学习性；官网所述“有效序列长度效率提升”是其系统比较口径，实际 N 应由编码器 stride 与 patch 配置计算。官方主干配置使用 24 通道视频 latent，patch 为 $1 \times 2 \times 2$；通道数只描述 latent 宽度，不直接给出压缩倍率。[164, 166]

比较 VAE 时联合检查重建细节、运动频率、音画同步、编码/解码延迟与 token 预算。重建很好但 latent 不利于主干学习的编码器，可能让生成训练更困难；压缩率越高，越需要观察小物体、文字与快速运动。

本章思考与进阶练习

1.1 压缩预算

题目 同一视频的空间压缩倍率从 8 改为 16，时间倍率与 patch 不变。token 数、密集 attention 配对数和 MLP 算量怎样变化?

解题思路 忽略尺寸取整，两个空间轴各减半，N 变为原来的 1/4；attention 配对项约为 1/16，MLP 约为 1/4。同时需重新测重建与运动质量，网络宽度或 VAE 通道变化会改变具体成本。

1.2 分布与信息

题目 若每个输入的 $q_\phi(z \mid X)$ 都被约束得完全等于同一个标准高斯，编码器还通过采样 Z 传递多少输入信息?

解题思路 此时 Z 与 X 独立，互信息为零。解码器仍能学习总体统计，但这个随机 Z 无法指认具体输入。重建与 KL 的权重正是在信息保留和规则化之间建立折中。

1.3 进阶：因果压缩

题目 DiT 采用严格分块因果 mask，而训练 VAE 编码器使用未来帧。实时训练会出现哪种隐性信息通路?

解题思路 当前 latent 已编码未来像素，因果 DiT 读到该 latent 时可间接获得未来信息。应从原始帧到编码、条件、attention 和解码逐层审计感受野，比较整段与真实在线管线。

第二章　网络骨干与条件接口

表示决定输入变量，骨干决定变量之间如何交换信息。本章建立 attention、输出头、单/双流、专家与时空条件的统一结构视图；相同骨干可以配合不同生成目标。

2.1　Attention、隐藏表示与任务输出头

attention 是信息交换模块；输出头才决定你预测词表概率还是连续视频数值。

对于 token 特征矩阵 $H \in \mathbb{R}^{N\times d}$:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad A = \mathrm{softmax}(QK^\top/\sqrt{d_k})V.$$

这里的 softmax 表示“每个位置关注其他位置多少”，归一化轴对应输入位置。

最后一层决定任务

模型	输出 head	输出含义
LLM	$\ell = hW_{\text{vocab}}$	词表中每个离散符号的 logit
视频 DiT	$\hat{v} = hW_{\text{out}}$，再 unpatchify	连续 latent 的速度/噪声/干净值
图像判别器	$r = hW_r$	一个实数质量或真假分数

LLM 的 logits 本身也是实数向量，但它们参数化离散类别分布；flow velocity 是对连续状态的变化率。前者的坐标对应词表类别，后者的坐标对应 latent 中的连续变量。

中间特征到速度预测的输出投影

通常流程是 attention、多层 MLP、残差、归一化等得到隐藏特征，最后通过任务输出头预测速度。attention 输出作为中间表示，再由任务输出头变成速度。[5, 7]

K/V 缓存的有效条件

历史位置已经计算过的 K/V 可以保存，后续查询只计算新的 Q/K/V。适合严格因果、历史状态不再变化的上下文。**双向 diffusion 中所有 noisy token 随去噪步更新，其 K/V 通常也改变**，跨步复用需要缓存近似与刷新策略。固定文本条件的 cross-attention K/V 与因果视频历史 K/V 是两类不同缓存。某些系统用近似特征缓存进一步加速，也要检查时间条件、位置编码和误差。

Q/K/V 属于 attention 模块，可处理离散 embedding 或连续 patch。纯卷积去噪器则可以完全通过卷积交换信息。

2.2　U-Net、U-ViT、DiT 与多模态融合

骨干的差异主要在尺度处理、跳连方式、条件融合和参数共享。

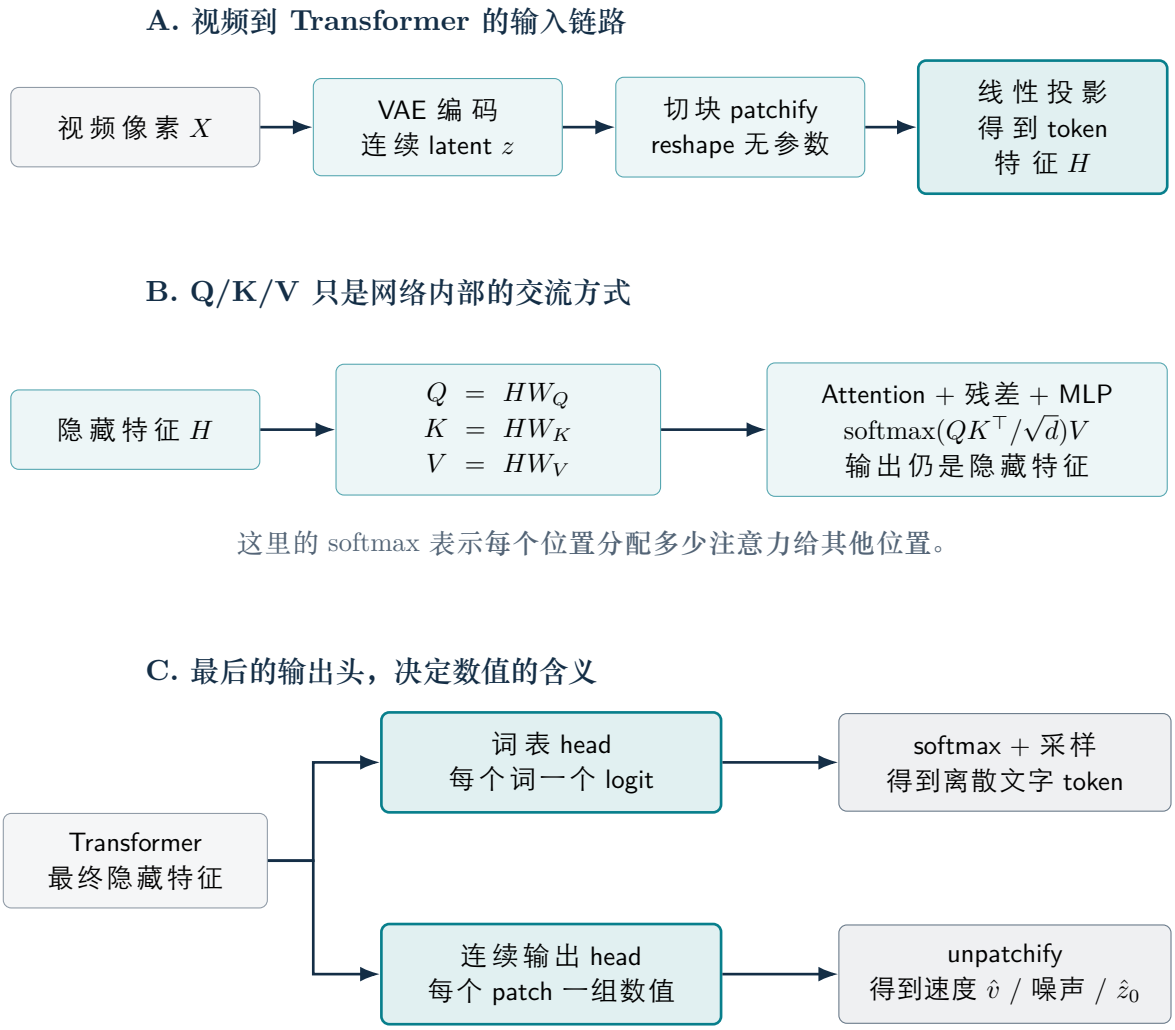


图 2.1 VAE、patch、Q/K/V 与输出头。内部都能使用 attention，但输出头决定是离散词表概率还是连续视频数值。**训练对象：**常见 latent diffusion 冻结已训练好的 VAE，训练 DiT 与输入/输出投影。patchify 的 reshape 本身没有权重；patch embedding 和输出 projection 通常有权重。[2, 5, 9]

U-Net：在多种尺度上处理画面

编码路径逐级降低空间分辨率、增加通道；瓶颈处理全局内容；解码路径上采样，并接收同尺度的 skip features。可写成

$$h_{l+1} = \mathrm{Down}_l(h_l), \qquad \tilde{h}_l = \mathrm{Block}_l([\mathrm{Up}_l(\tilde{h}_{l+1}), h_l]).$$

高分辨率跳连保留细节，低分辨率层处理大范围关系。ResBlock 中可注入时间 embedding，部分尺度加入 self / cross attention。视频版再引入时间卷积或时间 attention。

U-ViT：Transformer 中保留浅深层长跳连

把 noisy patches、时间和条件编码为 token，用 Transformer 建模；将浅层隐藏状态送到对应的深层块。这里的 U 主要体现在深度方向的 skip connection。原 U-ViT 以长跳连保留低层特征，让后部预测更容易恢复局部信号。[6]

DiT：规则 patch 网格上的 Transformer

典型块可抽象成

$$h' = h + g_1(t,c) \odot \mathrm{Attn}(\mathrm{AdaLN}_1(h;t,c)),$$
$$h'' = h' + g_2(t,c) \odot \mathrm{MLP}(\mathrm{AdaLN}_2(h';t,c)).$$

输入/输出投影负责 latent patch 与 hidden state 的转换。原 DiT 使用 adaLN-Zero 初始化，使残差分支从较温和的状态开始学习。规则堆叠便于扩大深度、宽度和 token 数。[5]

MMDiT：模态专属变换，联合注意力

文本和视觉各自做投影与调制，在联合 attention 中交换信息。简化写作

$$Q = [Q_T; Q_V], \quad K = [K_T; K_V], \quad V = [V_T; V_V], \quad H' = \mathrm{softmax}(QK^\top/\sqrt{d})V.$$

随后按模态拆回分支。这样能同时保留模态专属处理能力和跨模态对齐。HunyuanVideo 的混合设计还会把后部换成更共享的单流块。[7, 8]

选骨干时观察三件事：哪些层负责局部细节？全局信息在哪里汇合？条件通过 cross attention、联合 attention 还是归一化调制进入？

文本条件的特征传递

文本编码器先把完整 prompt 变成特征，视频模型读取这些特征。这些特征通过联合 attention 参与视频生成；生成新文字的模型还需文字预测 head 和相应训练目标。

AdaLN、LayerNorm、RMSNorm

对一个 token 的通道向量 h：

$$\mathrm{LN}(h) = \gamma \odot \frac{h - \mu(h)}{\sqrt{\mathrm{Var}(h) + \epsilon}} + \beta, \quad \mathrm{RMSNorm}(h) = \gamma \odot \frac{h}{\sqrt{\mathrm{mean}(h^2) + \epsilon}}.$$

AdaLN 用时间/条件生成缩放和偏移，例如 $(1 + a(t,c)) \odot \mathrm{LN}(h) + b(t,c)$。它负责把噪声等级等信息注入网络。**AdaLN 通常在 token 的通道维上归一化**；具体轴由实现定义。删掉调制不仅改变数值稳定性，也可能删掉关键条件能力。

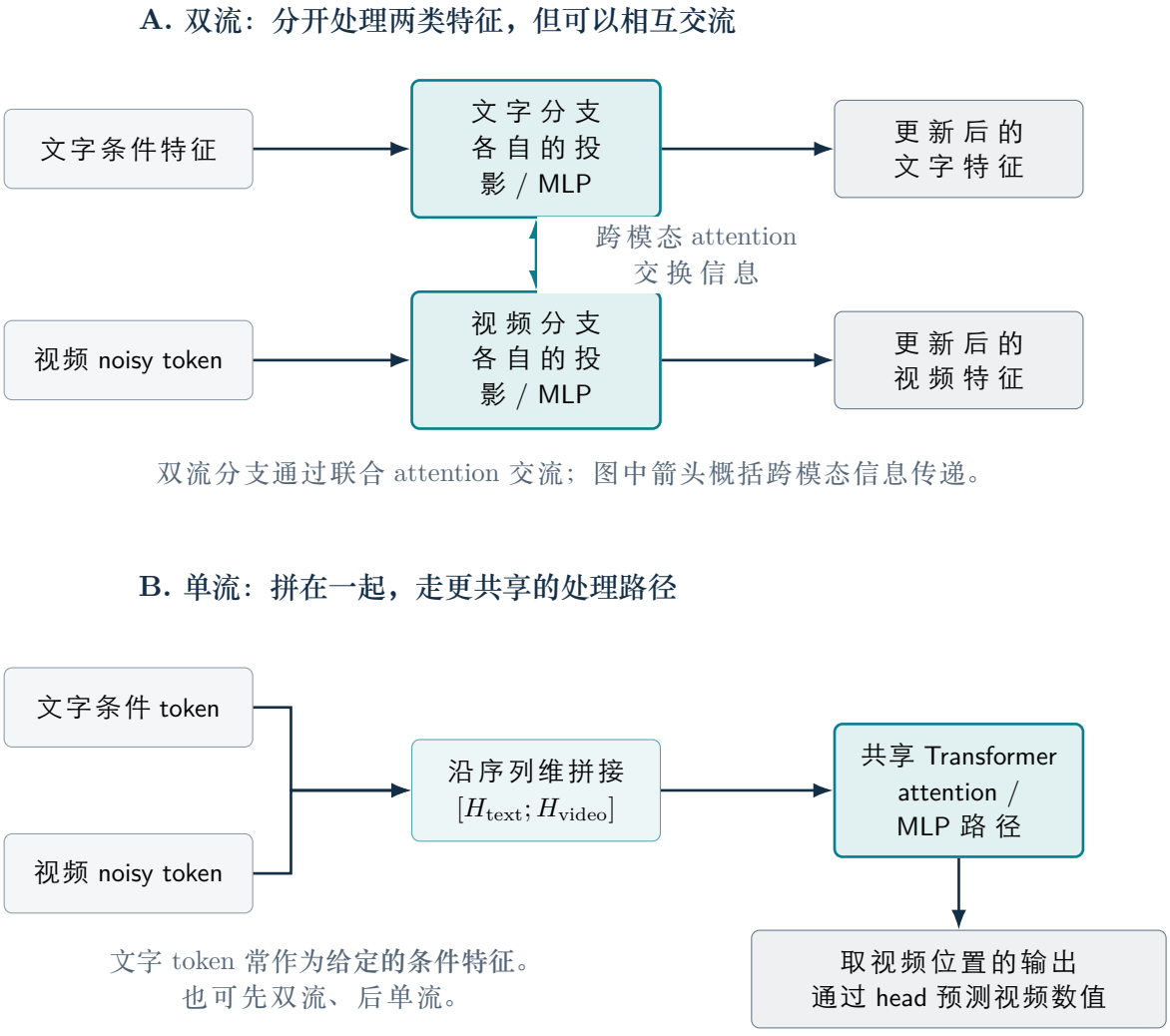


图 2.2　双流与单流：模态怎样交换信息?。双流强调模态各有处理分支；单流强调拼接后的更共享处理路径。读图：图展示模态交互方式。MMDiT 与 HunyuanVideo 的实际投影、调制和 attention 设计应分别查原实现。文字特征通常承担生成条件的角色。[7, 8]

2.3 MoE-DiT 与噪声阶段专家

路由器为每个 token 选择少量专家，专家池扩大模型容量。

在 dense DiT 中，每个 token 都通过同一组 MLP。MoE 常把这些 MLP 替换成多个专家 $E_1, \dots, E_M$，由路由器选择 top-k：

$$a_i(h) = \mathrm{softmax}(W_g h)_i, \quad S(h) = \mathrm{TopK}(a(h)),$$
$$h_{\text{out}} = E_{\text{shared}}(h) + \sum_{i \in S(h)} \pi_i(h) E_i(h).$$

π_i 是选中专家的路由权重；共享专家处理常见特征，稀疏专家提供分工。具体 MoE 设计可选是否配置共享专家。

总容量与激活计算的预算

若每个专家有 P 个参数，专家池占 $8P$；一个 token 实际使用约 $2P$ 的专家参数，另加 attention、router 和共享部分。所有专家权重仍需存储，训练还需梯度与优化器状态。

$$P_{\text{total}} = P_{\text{shared}} + MP_E, \qquad P_{\text{active/token}} \approx P_{\text{shared}} + kP_E.$$

因此报告模型规模时同时列**总参数、激活参数、FLOPs、设备数与延迟**，读者才能看清容量与成本。

路由负载与通信

- **负载均衡**：热门专家容易挤满，其他专家闲置。辅助平衡 loss 鼓励更均匀分配。
- **通信**：专家分布在不同 GPU 时，需要把 token 发往对应专家，再聚合输出。
- **时间变化**：同一个位置在高噪声和低噪声阶段可能需要不同专家。
- **细节质量**：路由容量、丢 token 策略和专家共享影响输出稳定性。

DiT-MoE 探索共享专家和稀疏专家路由，将扩散 Transformer 扩展到约 16.5B 参数。其扩展实验体现的是容量、数据与计算共同作用；落到视频部署时，需要进一步测量专家通信与实际激活成本。[65]

MoE 改变“哪个参数子集处理这个 token”。Sparse attention 改变“这个 token 读取哪些位置”。二者分别作用在专家选择和信息连接上，可以组合。

按噪声阶段路由整个模型

前面的 token-level MoE 让不同 token 选择 FFN 专家。视频模型还可以按噪声阶段选择整个去噪专家。Wan2.2 A14B 的官方设计包含高噪声与低噪声专家：高噪阶段侧重整体结构，低噪阶段侧重细节，按采样时间切换。[133]

$$v(z_t, t, c) = \begin{cases} v_H(z_t, t, c), & t \in \mathcal{T}_H, \\ v_L(z_t, t, c), & t \in \mathcal{T}_L. \end{cases}$$

这里的区间用本书噪声方向表示，部署应使用 checkpoint 配套日程和切换阈值。一次前向激活一个专家，总权重仍需存储或调入。与逐 token 路由相比，它把负载分配放在去噪时间轴上，通信和缓存模式也不同。

选择专家化方案时同时记录总参数、激活参数、峰值驻留参数和切换成本。相同激活参数下，更大的总参数可能增加存储与冷启动开销；不同架构的参数量相同，也会因 token 数、attention 结构和硬件利用率而具有不同延迟。

2.4 时空注意力与三维位置编码

视频模型需要同时编码帧内布局、帧间对应和时间尺度。

时空 token 序列

视频 VAE 输出 $T' \times H' \times W'$ 的 latent 网格，再按 patch 得到 $N = T'S$ 个 token，其中 S 为每帧的空间 token 数。注意力的时间索引 f 表示视频位置，去噪时间 t 表示当前噪声强度。

注意力组织	可直接交互的内容	主要配对计算量
全时空注意力	任意帧、任意空间位置	$O(T'^2S^2d)$
时空分解	先帧内，再同位置跨帧	$O(T'S^2d + ST'^2d)$
局部与稀疏	按窗口或选定连接交互	随窗口与连接数量变化

以上只计注意力配对，d 为特征宽度，未列 QKV 投影和 MLP。分解结构通过多层交替交换信息；完整时空连接提供更直接的跨位置运动联系。

从图像模型扩展到视频的路线

Video Diffusion Models 探索图像与视频联合训练；VideoLDM 将时间建模层加入图像 latent diffusion。Latte 系统研究空间与时间 Transformer 的组织。CogVideoX 使用三维全注意力，并以模态相关的 adaptive LayerNorm 处理文本和视频特征。其 expert modulation 描述模态参数分支；MoE-DiT 还涉及专家路由和稀疏激活。[106, 107, 109, 110]

3D RoPE：让 Q/K 感知相对位置

对一个二维通道对子，旋转矩阵为

$$R(\omega p) = \begin{pmatrix} \cos(\omega p) & -\sin(\omega p) \\ \sin(\omega p) & \cos(\omega p) \end{pmatrix}, \quad (R_p q)^\top (R_r k) = q^\top R_{r-p} k.$$

因此点积包含位置差。3D 扩展把通道组分配给 (f, h, w) 三个轴，或按实现组合三轴频率；HunyuanVideo 的报告给出了视频应用。[113, 111]

FPS 与时间压缩同样影响运动

原视频第 f 帧对应物理时间 f/FPS。改变抽帧步长会改变相邻帧的运动幅度；VAE 时间压缩又改变一个 latent 覆盖的帧范围。训练与推理需共同约定 FPS、抽帧步长、位置缩放和有效帧数。这样才能区分“动作更快”“帧更稀”和“生成更长”。

2.5 文本、参考图、结构与相机条件

不同条件通过相应接口进入生成网络，并由配套训练建立可控关系。

四类接口

接口	输入方式	典型用途
拼接与 mask	$[z_t, z_{\text{known}}, M]$	首帧、局部已知区域、补全
交叉注意力	视觉 Q 读取条件 K/V	文本、参考图、身份特征
残差控制分支	主干层加控制特征	深度、边缘、姿态、布局
几何与时间	相机射线、轨迹、帧时间	运镜、运动与时序控制

M 指示哪些位置已知；已知内容的噪声处理和拼接位置随模型配置。训练样本中的条件与目标需要对齐，推理沿用相同接口。

ControlNet 与 IP-Adapter

ControlNet 在冻结图像生成主干旁训练控制分支，通过零初始化卷积接入残差，使结构图影响逐渐学入主干。IP-Adapter 分离图像和文本的 cross-attention，可用下式理解组合：[114, 115]

$$h' = h + \text{Attn}(Q, K_{\text{text}}, V_{\text{text}}) + \lambda\, \text{Attn}(Q, K_{\text{image}}, V_{\text{image}}).$$

该式为接口示意。图像参考强调外观和语义；深度、姿态等对齐图提供空间结构。迁移到视频时需扩展帧间对齐、时间模块与训练数据。

AnimateDiff 与 CameraCtrl

AnimateDiff 在图像生成骨干中学习运动模块，研究把视频运动能力迁移到个性化图像模型。CameraCtrl 则使用相机轨迹监督专门的控制模块。[116, 117] 一条世界坐标射线由相机中心 o 和单位方向 d 定义，Plücker 表示可写为 (m, d)，其中 $m = o \times d$。逐帧、逐像素的射线特征把相机姿态变成与视频网格对应的条件；专用编码器将其送入视频网络。

对应到 **i2v / r2v**：首帧提供视频起始状态；参考图提供身份或外观约束；相机条件提供几何运动约束。KV 是传递条件的一种载体，具体任务能力取决于位置、mask、噪声约定和模型见过的训练关系。

2.6 产业模型的结构实例：单流、双分支与 MoE

Seedance：视频生成到音视频联合生成

Seedance 1.0 使用时空压缩表示和 diffusion Transformer，联合支持文本/图像条件及原生多镜头生成。1.5 pro 的公开报告描述音频、视频双分支 MMDiT 与跨模态交互；两个分支使用各自的表示，再交换语义与时间信息。它与“文本、视觉双流”共享多模态交互思想，但分支对应的模态不同。[167, 168]

2.0 报告扩展为统一多模态音视频生成，接收文本、图像、音频、视频参考。2.5 在官方发布说明中沿用这一统一路线，扩展到 30 秒单次生成、多轮延长与更强参考编辑。2.0/2.5 的公开资料主要解释能力、输入输出和评价；层数、路由算法及各损失权重应以具体公开实现为依据，不能从产品功能反推出同一套内部配方。[169, 170]

daVinci-MagiHuman：带模态特定端部的单流 Transformer

SandAI 与 SII-GAIR 的 MagiHuman 使用 15B、40 层主干，文本、视频、音频拼成统一序列，通过 self-attention 交互。前 4 层与后 4 层使用模态特定的投影/归一化，中间 32 层共享主干；采用 per-head gating，并省去显式 timestep embedding 路径。单流表示统一的 token 交互路径，不要求每一层所有参数完全相同。[161]

该模型说明：时间信息既可以显式输入，也可以在特定训练设计中让网络从带噪状态推断。省去显式 t 是经训练验证的架构选择；把已有 DiT 的 AdaLN 时间路径直接删掉，会改变原来学到的条件函数。

MAGI-2 Preview：多头潜表示中的细粒度专家

MAGI-2 采用来自 MagiHuman 的统一单流接口，公开预览没有直接沿用 MAGI-1 的 AR chunking。其重点转向模型、数据与系统扩展：约 114B 总参数、每 token 约 6B 激活参数，使用多头 LatentMoE。[162]

设每个 token 的表示被分为 M 个潜 head，各有独立 router 与专家池。普通 expert parallel 常需将完整 D 维 token 发往 top-k 专家，payload 近似 NkD；Head Parallel 先把 head 子空间分到对应设备，再在本地选专家，跨设备基础 payload 约为

$$NMd_h = ND, \qquad D = Md_h.$$

这里省略回传与其他常数，用于比较路由导致的通信结构。动态选择仍存在于每个 head 内，跨设备缓冲区却可预先确定。attention 成本仍取决于 token 数，6B active 也不能替代完整 FLOPs 或显存统计。

MiniMax：Hailuo 02 的 NCR 与 H3 的统一生成

Hailuo 02 官方公开了 Noise-aware Compute Redistribution（NCR）：按噪声阶段重新分配计算，在其同量级参数比较中报告约 2.5 倍训练/推理效率。公开说明给出设计方向与相对结果，未完整公开路由公式；它适合放在“噪声阶段与计算预算”的框架下，与 Wan2.2 噪声专家、TRACK 模型切换比较各自具体实现。[163]

H3 另选任务泛化导向的 Omni Transformer。官方开放仓库的 `transformer/config.json` 给出 50 层、hidden size 5376、56 个 attention heads、head dim 128、视频输入通道 24、音频输入通道 32、patch size $(1, 2, 2)$。注意 $56 \times 128 = 7168$ 与 hidden size 不同；投影内部维度可以不同于残差宽度，应按配置和 forward 核对 shape，不能机械套用 $D = Hd$。[165, 166]

H3 的工程流程包含 Context-IR、768p Base、Regenerate-2K；FL2VA 处理首尾帧等约束，Ref2VA 处理多模态参考。理解模块整理条件，主干生成音视频，高分辨率阶段重新利用原始条件和低清结果。表示与级联细节分别见第一、五章。

本章思考与进阶练习

2.1 token 与输出

题目 $N = 1000, d = 2048$，词表为 50000，latent patch 为 64 维。LLM head 与视频 head 的输出形状分别是什么?

解题思路 分别为 1000×50000 与 1000×64。前者每行参数化类别分布；后者的 64 个坐标写回连续 latent patch。两种网络内部都可使用 Q/K/V。

2.2 条件缓存

题目 固定 prompt 后，哪些文本 K/V 可以一次计算？联合文本–视觉 attention 的文本分支也满足同样条件吗?

解题思路 固定编码特征经过固定 cross-attention 投影得到的 K/V 可复用。若文本隐藏状态每层受到 noisy 视觉 token 或时间调制影响，投影输入随步数改变，就需要重新计算或采用经过验证的近似缓存。

2.3 进阶：两种专家化

题目 对比 token-level MoE 与高/低噪声专家，设计一个公平吞吐测试。

解题思路 统一输入 token、日程、精度和硬件，分别记录激活 FLOPs、驻留参数、路由或换权重耗时、通信和峰值显存。质量按相同任务测，参数总量与激活参数都单独报告。

第三章　生成目标与训练估计

本章从概率建模进入可计算的训练目标：噪声、干净样本和速度回归怎样对应，条件插值怎样得到边缘速度，离散 AR、GAN 与 EBM 又提供哪些监督。后续采样器只消费这些模型接口。

3.1　生成目标的统一视图

目标函数决定网络学的是局部去噪、运动方向、区间位移还是样本分布。

下面把条件 c 暂时省略；各式中的网络仍可使用同样的 U-Net 或 Transformer。

目标	网络输出	监督来源
Diffusion	噪声 $\hat{\epsilon}$、干净值 $\hat{z}_0$ 等	数据加噪时已知的噪声 / 数据
Flow Matching	瞬时速度 $v(z_t, t)$	选定条件路径的导数
MeanFlow	区间平均速度 $u(z_t, r, t)$	瞬时速度 + 平均速度恒等式
GAN	生成样本；判别器给分	真实 / 生成样本的可区分程度
EBM	标量能量 $E_\theta(z)$	提高数据概率，压低负样本概率

三种“回归一个向量”的区别

$$L_\epsilon = \mathbb{E}\,\|\epsilon_\theta(z_t, t) - \epsilon\|^2, \quad L_{\mathrm{FM}} = \mathbb{E}\,\|v_\theta(z_t, t) - \dot{z}_t\|^2.$$

噪声和速度都是向量标签。选定噪声日程后可做参数化转换，但时间权重与数值条件也会随之变化。MeanFlow 进一步输入两个时间，学习整个区间的平均变化率，适合大步更新。[3, 4, 80]

GAN：用分布级反馈训练生成器

令 $\hat{z} = G_\theta(\epsilon)$，判别器为 D_ψ。以 non-saturating logistic 目标为例：

$$L_D = \mathbb{E}_{z\sim p_{\mathrm{data}}}\,\mathrm{softplus}(-D_\psi(z)) + \mathbb{E}_{\hat{z}}\,\mathrm{softplus}(D_\psi(\hat{z})), \quad L_G = \mathbb{E}_{\hat{z}}\,\mathrm{softplus}(-D_\psi(\hat{z})).$$

生成器靠判别器的梯度改善样本。若 G 是一次前向网络，推理天然为一步；训练要处理判别器强弱、模式覆盖与稳定性。扩散蒸馏也能叠加 GAN 项，改善少步样本的细节。[12, 76]

多阶段目标与联合训练

视频 VAE 可以用重建 + KL + LPIPS + GAN 训练；主生成器用 FM 预训练；少步化时加 consistency 与分布匹配；后训练再引入偏好目标。每个阶段都有自己的更新对象和数据接口。

模型名字往往只突出一层设计。读训练代码时，依次找到 forward 的输出、target 的构造、loss 的权重和 optimizer 管理的参数。

3.2　DDPM 的概率模型与噪声回归

高斯反向转移的均值误差，可以换写成预测噪声的平方误差。

前向加噪过程

令 z_0 为干净图像或视频 latent，n 为离散加噪步；$a_n = 1 - \beta_n$，$\bar{a}_n = \prod_{j=1}^{n} a_j$。DDPM 使用 [3]

$$q(z_n \mid z_{n-1}) = \mathcal{N}(\sqrt{a_n} z_{n-1}, \beta_n I), \qquad z_n = \sqrt{\bar{a}_n} z_0 + \sqrt{1-\bar{a}_n}\epsilon.$$

$\epsilon \sim \mathcal{N}(0, I)$。$q$ 表示预先定义的加噪分布；训练时自己抽取噪声，因此监督目标 ϵ 已知。一次训练可直接抽 n 并构造 z_n，省去逐步模拟整条加噪链。

反向网络学习一个连续概率密度

设 $p_\theta(z_{n-1} \mid z_n) = \mathcal{N}(\mu_\theta(z_n, n), s_n^2 I)$。训练样本还提供 z_0，所以高斯后验 $q(z_{n-1} \mid z_n, z_0)$ 可解析计算。把均值参数化为

$$\mu_\theta = \frac{1}{\sqrt{a_n}} \left(z_n - \frac{\beta_n}{\sqrt{1-\bar{a}_n}} \epsilon_\theta(z_n, n) \right).$$

真实后验均值的同一位置放入实际噪声 ϵ。网络输出每个连续坐标的噪声估计，进而确定反向高斯的均值。

从 ELBO 到加权 MSE

负对数似然的变分上界包含各步后验与模型转移之间的 KL，以及起点重建项和终点先验项。对 $n \geq 2$、固定反向方差 s_n^2，单步 KL 为

$$D_{\mathrm{KL}}(q(z_{n-1} \mid z_n, z_0) \,\|\, p_\theta(z_{n-1} \mid z_n)) = \frac{\beta_n^2 \left\| \epsilon - \epsilon_\theta \right\|^2}{2 s_n^2 a_n (1-\bar{a}_n)} + C,$$

C 与均值网络参数无关。DDPM 的常用简化训练目标去掉该时间权重，得到 $\mathbb{E} \left\| \epsilon - \epsilon_\theta(z_n, n) \right\|^2$。因此“来自似然推导”与“实际使用简化权重”是两层设计。

连续模型的概率从哪里来？ 由高斯转移及其组合定义。Transformer 内部的 Q/K/V 负责计算特征，输出头给出均值相关的噪声预测；离散词表的 softmax 是另一种分布参数化。

3.3 Flow Matching：路径、速度与边缘分布

训练样本的噪声和时间是自己造出来的，因此连续监督目标完全可知。

噪声预测：自己加的噪声，自己知道答案

从真实视频得到干净 latent z_0，采样噪声 ϵ 与 t：

$$z_t = \alpha_t z_0 + \sigma_t \epsilon, \qquad L_\epsilon = \mathbb{E} \left\| \epsilon_\theta(z_t, t, c) - \epsilon \right\|_2^2.$$

这里无需把整条去噪轨迹跑完才能训练。若模型预测 z_0，则可用 $L_x = \mathbb{E} w(t) \left\| \hat{z}_{0,\theta} - z_0 \right\|^2$。两种参数化可转换，但损失权重不同，训练效果不必相同。[3]

线性 Flow Matching：学习连续变化的方向

本文约定干净端为 0、噪声端为 1：

$$z_t = (1-t) z_0 + t\epsilon, \qquad u_t = \frac{dz_t}{dt} = \epsilon - z_0,$$

$$\boxed{L_{\mathrm{FM}} = \mathbb{E}_{z_0, \epsilon, t, c} \left\| v_\theta(z_t, t, c) - (\epsilon - z_0) \right\|_2^2.}$$

推理从大 t 向小 t 积分：$z_s = z_t + (s-t) v_\theta(z_t, t, c)$，所以虽然监督速度是“数据到噪声”的方向，负时间步会带你往干净端走。[4]

一维数值例子

令 $z_0 = 2, \epsilon = -1, t = 0.8$，则 $z_t = -0.4$，监督速度为 -3。如果从 0.8 走到 0.7，更新是 $-0.4 + (-0.1)(-3) = -0.1$，确实更接近 2。这是一个已知配对样本的路径。实际网络在同一 z_t 处可能面对多种潜在来源，学习的是相应条件平均速度；**条件平均速度组成的采样轨迹仍可能弯曲。**

扩散 v-prediction 与 flow velocity 的日程

VP 扩散里常见 $v = \alpha_t \epsilon - \sigma_t z_0$。它与线性 FM 的 $\epsilon - z_0$ 分别由各自日程定义；解释 v 时同时检查日程与端点。

扩散模型的"答案"可以是噪声、干净 latent 或速度；它们都是与输入 latent 形状对应的连续张量，不需要语言词表。

条件回归与边缘场

训练样本给出的方向与模型在某个位置输出的方向，属于两个层次。设干净 latent 为 z_0，独立噪声为 ϵ，条件为 c：

$$z_t = (1-t)z_0 + t\epsilon, \qquad u = \epsilon - z_0.$$

固定一对 (z_0, ϵ)，路径是直线，u 沿这条直线保持不变。然而，同一个 z_t 附近会出现许多不同样本对，它们给出的 u 可以不同。网络输入只有 (z_t, t, c)，因此平方损失的最优解是条件均值：

$$v^*(z, t, c) = \mathbb{E}[u \mid z_t = z, t, c].$$

从平方损失推导最优预测

令 $m = \mathbb{E}[u \mid z_t, t, c]$，展开得到

$$\mathbb{E}[\|v - u\|^2 \mid z_t, t, c] = \|v - m\|^2 + \mathbb{E}[\|u - m\|^2 \mid z_t, t, c].$$

交叉项为零，第二项与待优化的 v 无关，所以回归到 m 最优。一个一维教学例子：同一观测状态下，有四分之三的配对速度为 $+2$，四分之一为 -2，最优预测为 1。网络在混合样本中学习共同的局部输运方向。

这个边缘速度场满足连续性方程

$$\partial_t p_t(z \mid c) = -\nabla_z \cdot [p_t(z \mid c) v^*(z, t, c)].$$

在正则性、精确学习与精确积分的条件下，它把各个时间的边缘分布连接起来。推理沿该场求解 ODE；边缘轨迹可以弯曲，即使训练配对路径都是直线。[4, 95]

训练一次前向与推理整条轨迹

基础 FM 训练通常随机采样一个 t，计算一次网络输出与 MSE；它通过大量样本覆盖整个时间区间。推理时才连续访问多个 t。因此，50 步采样的基模训练，并不天然要求每个训练样本都反传 50 次。蒸馏、可微奖励或 on-policy rollout 可能显式引入多步生成，它们的训练成本要另外计算。

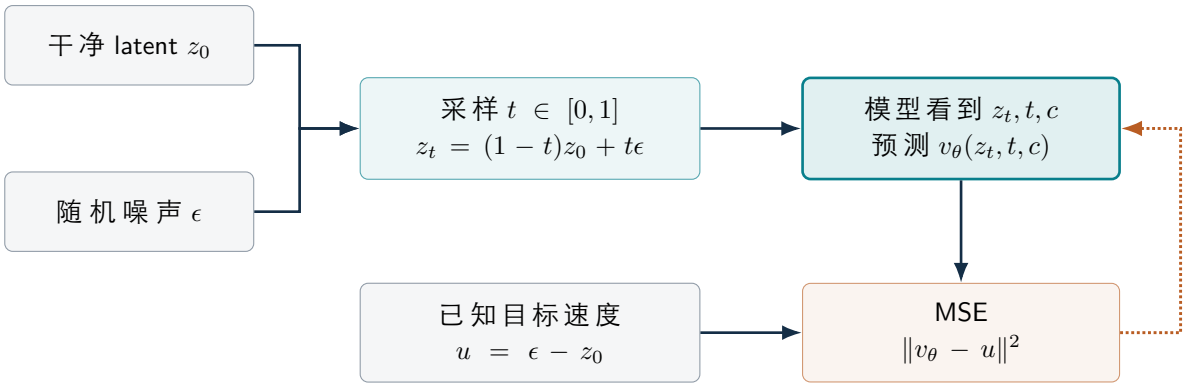
预测对象、损失空间与采样接口

可把网络输出组织成三层：a_θ 是 head 的直接输出，$T_t(a_\theta, z_t)$ 将它转换成 loss 所在空间，采样器读取转换后的速度或 score。例如直接预测 $\hat{z}_0$ 时，在线性路径内

$$\hat{v} = \frac{z_t - \hat{z}_0}{t}, \qquad \|\hat{v} - u\|^2 = \frac{\|\hat{z}_0 - z_0\|^2}{t^2}, \quad t > 0.$$

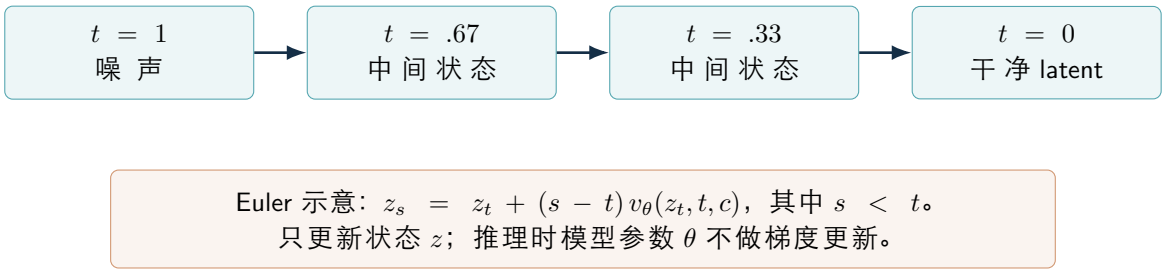
两种参数化可描述同一个理想场，但损失权重、输出尺度和数值稳定性随之变化。训练的 t 截断、预条件和权重决定模型把容量分配到哪些噪声区间。

A. Flow Matching 训练：随机抽一个噪声程度



模型不需要看完整去噪轨迹才有监督：我们自己构造 z_t，也知道目标 u。

B. 推理：从噪声开始，反向走回干净端



图中 3 步仅为教学示意。预测噪声、干净值或速度是不同参数化；连续速度由输出头预测，采样器用它更新 latent。

图 3.1 Flow Matching：训练与推理分开看。训练时我们知道怎样加噪，所以知道目标速度；推理时只反复调用学到的速度场。符号约定：$t = 0$ 为干净、$t = 1$ 为噪声。采样向低噪端走，因此 $s - t < 0$。这张图使用直线条件路径作为教学例子；不同噪声调度和参数化需要相应转换。[4, 5]

3.4 干净样本预测与损失参数化

预测参数化、损失空间与采样动力学可以分别选择。

网络输出、目标转换与损失

JiT（Back to Basics: Let Denoising Generative Models Denoise）研究像素空间 Transformer：直接预测干净图像，并考察大 patch 下的数据表示与去噪。其基本结构省去 VAE 和预训练表征编码器。[105] 按本文“数据为 0、噪声为 1”的时间约定，设

$$x_t = (1-t)x_0 + t\epsilon, \qquad \hat{x}_0 = f_\theta(x_t, t).$$

预测接口：最后的线性头还原 patch 中的像素值，拼回整幅 $\hat{x}_0$。

速度接口：对于 $t > 0$，从预测的干净数据计算

$$v_\theta(x_t, t) = \frac{x_t - \hat{x}_0}{t}, \qquad v^\star = \epsilon - x_0 = \frac{x_t - x_0}{t}.$$

损失接口：在速度空间做平方误差等价于带权的干净数据误差：

$$\|v_\theta - v^\star\|^2 = \frac{1}{t^2} \|\hat{x}_0 - x_0\|^2 .$$

例如 $t = 0.5$，像素平方误差会乘 4；$t = 0.1$ 时乘 100。实现会处理端点、时间抽样和数值稳定性。论文使用相反端点时，先换成同一约定再比较公式。

输出参数化的优化影响

大 patch 含有许多原始像素坐标，而语义图像位于更有结构的集合中。JiT 从这一角度研究直接预测干净数据与高维噪声预测的差别，并结合输入瓶颈等结构。参数化改变网络所需表示的函数，也改变相同 loss 写回输出空间后的权重。

数据预测与少步生成

一次前向得到 $\hat{x}_0$ 后，采样器仍可将其转换为速度并做多步积分；论文默认使用多步 Heun。一步生成另外需要考察端点预测能力与相应训练。与 RAE 比较时，先检查生成空间：JiT 的状态是像素，RAE 的状态是预训练视觉表征。两者都涉及 Transformer，但压缩、表示和解码接口各有配置。

阅读一篇新论文时按顺序标记：输入状态是什么；网络头直接输出什么；误差在哪里计算；采样器实际使用哪个量。四个答案组合起来，公式就容易对应。

3.5 噪声日程、SNR 与训练权重

加多少噪声、抽哪些时间点、给每个误差多大权重，共同决定训练重点。

信噪比的定义

对 $z_t = \alpha_t z_0 + \sigma_t \epsilon$，在数据与噪声已作尺度约定时，$\mathrm{SNR}(t) = \alpha_t^2 / \sigma_t^2$。大 SNR 更接近干净数据，小 SNR 更接近噪声。一般目标写成

$$\mathcal{L} = \mathbb{E}_{t \sim p(t), z_0, \epsilon} \big[w(t)\, \ell_\theta(z_t, t) \big].$$

日程 (α_t, σ_t) 定义扰动；$p(t)$ 分配采样次数；$w(t)$ 分配每次梯度权重。推理时间网格还可以单独选择。

EDM：把数值尺度整理好

EDM 使用 $z_\sigma = z_0 + \sigma\epsilon$，以数据标准差 σ_d 构造去噪器 [93]

$$D_\theta(z_\sigma, \sigma) = c_{\text{skip}} z_\sigma + c_{\text{out}} F_\theta(c_{\text{in}} z_\sigma, c_{\text{noise}}),$$

$$c_{\text{in}} = \frac{1}{\sqrt{\sigma^2 + \sigma_d^2}}, \quad c_{\text{skip}} = \frac{\sigma_d^2}{\sigma^2 + \sigma_d^2}, \quad c_{\text{out}} = \frac{\sigma \sigma_d}{\sqrt{\sigma^2 + \sigma_d^2}}.$$

$c_{\text{noise}} = \log \sigma / 4$。这些系数控制输入、跳连和网络残差的尺度；重建权重 $\lambda(\sigma) = 1/c_{\text{out}}^2$ 与预条件配套。EDM 还讨论噪声抽样、采样日程和求解器，构成一套可分开调整的设计空间。

Min-SNR：平衡不同噪声任务

Min-SNR 将时间点视作共享参数上的多个任务，限制高 SNR 对梯度的支配。对于 VP 日程 $\alpha_t^2+\sigma_t^2=1$，令 $S=\mathrm{SNR}(t)$，等价转换后的权重为 [94]

$$w_{z_0}=\min(S,\gamma),\qquad w_\epsilon=\frac{\min(S,\gamma)}{S},\qquad w_u=\frac{\min(S,\gamma)}{S+1}.$$

这里 $u=\alpha_t\epsilon-\sigma_t z_0$ 表示扩散文献的 v-prediction 目标；线性 flow 的速度另由路径导数定义。三个权重分别乘各自预测空间的 MSE。举例：$S=100,\gamma=5$ 时，z_0 误差权重由 100 限到 5。

收益读法：Min-SNR 报告的约 3.4 倍加速针对其图像实验的训练收敛；推理 NFE、单步耗时需要另行测量。

3.6 Rectified Flow、Reflow 与 InstaFlow

先学习运输速度，再通过重新配对噪声与样本改善轨迹几何。

线性插值提供容易获得的训练目标

按本文端点约定，$z_t=(1-t)z_0+t\epsilon$，目标速度为 $u=\epsilon-z_0$：

$$\mathcal{L}_{\mathrm{RF}}=\mathbb{E}\left\|v_\theta((1-t)z_0+t\epsilon,t,c)-(\epsilon-z_0)\right\|^2.$$

每一对端点之间的训练插值为直线。在某个位置、某个时间，不同样本对可能要求不同速度；回归网络学习这些条件速度的平均。因此实际 ODE 轨迹仍可能弯曲。[95]

Reflow：用旧模型建立更协调的端点配对

1. 抽噪声 ϵ，通过旧模型积分得到 $\tilde{z}_0=G_{\mathrm{old}}(\epsilon,c)$。
2. 固定这对端点 $(\epsilon,\tilde{z}_0)$，重新构造直线上的训练点。
3. 训练新速度场，使其沿这组配对学习运输：

$$\mathcal{L}_{\mathrm{reflow}}=\mathbb{E}\left\|v_\theta((1-t)\tilde{z}_0+t\epsilon,t,c)-(\epsilon-\tilde{z}_0)\right\|^2.$$

第一轮常用独立噪声与真实样本；Reflow 的两端通过同一个生成轨迹相连。配对关系改变，模型更容易用较平直的路径完成运输。生成端点数据需要旧模型采样成本。

InstaFlow：把轨迹整理与一步学生衔接起来

InstaFlow 将 Rectified Flow / Reflow 用于文本到图像，结合重新配对后的流与一步蒸馏。直观地，一步 Euler 学生为 $G_\theta(\epsilon,c)=\epsilon-v_\theta(\epsilon,1,c)$，可以对齐教师从同一噪声积分得到的终点。[96]

方法	优先处理的量
Rectified Flow	局部瞬时速度，以及由它定义的运输
Reflow	噪声与数据端点的联合配对及轨迹形状
CM / PCM	全程或分段的端点一致映射
MeanFlow	给定时间区间内的平均速度

收益来自更容易粗步积分的轨迹与进一步蒸馏；阅读结果时同时记录 Reflow 轮数、教师生成数据量、学生 NFE 和质量变化。

3.7 视觉自回归、离散码本与连续预测

AR 描述生成顺序；每一步的条件分布可以用分类头或连续生成头表达。

VQ-VAE：把视觉局部编码成码本索引

编码器给出连续向量 $h_i = E(x)_i$，码本为 $\{e_j\}_{j=1}^{K}$。量化选择

$$q_i = \arg\min_j \|h_i - e_j\|^2, \qquad \hat{x} = D(e_{q_1}, \ldots, e_{q_N}).$$

索引 q_i 是离散类别。训练视觉 AR 时，Transformer 输出 K 维 logits：

$$p(q \mid c) = \prod_i p_\theta(q_i \mid q_{<i}, c), \qquad \mathcal{L}_{\text{CE}} = -\sum_i \log p_\theta(q_i \mid q_{<i}, c).$$

量化器本身还需要重建、码本与 commitment 等目标。VQ-VAE 奠定了这条表示路线；MAGVIT-v2 进一步研究适合图像和视频的视觉量化与 tokenizer。[97, 99]

生成顺序与掩码预测

MaskGIT：训练预测被遮住的离散 token；推理多轮并行填充，依据置信度逐渐保留预测。**VAR**：把条件分解改为从粗到细的 next-scale prediction，同一尺度可并行预测。这些图像论文提供了视觉生成顺序的两个重要参照。[98, 100]

MAR：AR 骨干加连续 diffusion head

MAR 保留连续视觉 latent。骨干根据当前可见位置形成条件向量 h_i，小型扩散头学习待生成位置的条件分布。示意目标为 [101]

$$z_{i,\tau} = \alpha_\tau z_i + \sigma_\tau \epsilon, \qquad \mathcal{L}_i = \mathbb{E}\,\|\epsilon_\psi(z_{i,\tau}, \tau, h_i) - \epsilon\|^2.$$

这里 i 是视觉位置，τ 是该位置生成头的去噪时间。外层逐步增加可见内容，内层采样连续向量。MAR 还研究随机顺序与掩码式并行生成。

两条轴一起看：外层“下一词 / 下一尺度 / 下一帧块”决定条件分解；内层“softmax / diffusion / flow”决定输出分布。视频模型可以按 chunk 自回归，同时在每个 chunk 内做连续去噪。

3.8 对抗训练、偏好排序与判别器正则

它们规定分类间隔或标签概率，适合真假判断与偏好排序。

设判别器 $D(X)$ 输出**原始实数分数**：越大越像真实样本。真实画面为 X，生成画面为 $\hat{X} = G(\epsilon)$。判别器和生成器要优化不同损失。

Hinge：分对且超过间隔就不再催促

$$L_D^{\text{hinge}} = \mathbb{E}_X \max(0, 1 - D(X)) + \mathbb{E}_{\hat{X}} \max(0, 1 + D(\hat{X})), \qquad L_G^{\text{hinge}} = -\mathbb{E} D(\hat{X}).$$

判别器希望真样本分数至少 +1，假样本至多 -1。例：真样本得 0.3，惩罚 0.7；得 1.5，惩罚 0。假样本得 0.2，惩罚 1.2。**生成器使用 $-\mathbb{E}D(\hat{X})$，持续推高假样本分数。**

Logistic：平滑地奖励更可靠的分类

令 $\mathrm{softplus}(a) = \log(1 + e^a)$，常见 non-saturating GAN 写法：

$$L_D^{\log} = \mathbb{E}_X \,\mathrm{softplus}(-D(X)) + \mathbb{E}_{\hat{X}} \,\mathrm{softplus}(D(\hat{X})), \quad L_G^{\log} = \mathbb{E}_{\hat{X}} \,\mathrm{softplus}(-D(\hat{X})).$$

这是 logits 形式的二元交叉熵；$\sigma(D(X))$ 才是 sigmoid 后的概率。hinge 有明确的零损失区间；logistic 平滑趋近于零，而非达到某个间隔后立即截断。GAN 的稳定性还取决于判别器强弱、正则与交替更新设置。[12, 13]

偏好排序里也会见到 logistic

对于人更喜欢的 X^+ 和不喜欢的 X^-：

$$P(X^+ \succ X^- \mid c) = \sigma(r(c, X^+) - r(c, X^-)), \quad L_{\mathrm{BT}} = -\log \sigma(r^+ - r^-).$$

这是 Bradley–Terry 式成对偏好模型。它通过两个分数之差预测相对偏好。分差为零时，预测偏好概率为 0.5。

L1/L2：离答案多远？　hinge：有没有越过分类间隔？　logistic：当前标签/偏好有多可信？

R1 / R2：输入梯度正则

常见零中心梯度正则：

$$\boxed{R_1 = \frac{\gamma}{2}\mathbb{E}_{X\sim p_{\mathrm{data}}} \|\nabla_X D_\psi(X)\|_2^2, \qquad R_2 = \frac{\gamma}{2}\mathbb{E}_{\hat{X}\sim p_G} \left\|\nabla_{\hat{X}} D_\psi(\hat{X})\right\|_2^2.}$$

$\nabla_X D$ 表示：稍微改变输入像素，判别分数会改变多快？如果一丁点像素扰动就让打分剧烈跳变，梯度范数大，惩罚也大。[14]

输入梯度正则与参数正则

名字	被约束对象	直觉
L1 重建	$\hat{X} - X$	让生成值接近参考
L2 权重正则	θ	控制参数大小
R1	$\nabla_X D$，在真样本处	让真样本附近的判别面更平缓
R2	$\nabla_X D$，在假样本处	让假样本附近的判别面更平缓

R1/R2 里的“1/2”表示两种采样位置；上面两式都使用平方 L2 范数。

一维例子

若 $D(x) = ax + b$，则 $\frac{dD}{dx} = a$，R1/R2 都会惩罚 a^2。在这个极简线性例子里它看起来像权重衰减；深度网络的输入梯度依赖数据、激活和整条计算链，因此它约束的是数据附近的函数形状。

梯度路径与更新参数

常见判别器步骤：先把生成画面 detach，再计算真假分类 loss 和 R1/R2；更新判别器参数 ψ。虽然正则中对 X 求导，**真正要优化的是判别器参数**，因此需对“输入梯度的平方”继续对 ψ 求导，有二阶自动微分开销。

与 WGAN-GP 的比较

WGAN-GP 常在真/假之间的插值点上惩罚 $(\|\nabla_X D\|_2 - 1)^2$；R1/R2 则是零中心的 $\|\nabla_X D\|_2^2$。两类正则采用不同的目标梯度范数和采样位置。lazy regularization 则是隔若干步才计算，需相应调整权重和优化设置。

3.9 能量模型、最大似然与 Langevin 采样

能量低的位置概率高，归一化常数负责让整张概率分布积分为 1。

$$p_\theta(z) = \frac{\exp[-E_\theta(z)]}{Z_\theta}, \qquad Z_\theta = \int \exp[-E_\theta(z)]\, dz.$$

E_θ 输出一个标量。把它想成地形高度：低谷聚集更多概率质量。高维空间里直接计算 Z_θ 很难，但对状态求梯度时它会消失：

$$\boxed{\nabla_z \log p_\theta(z) = -\nabla_z E_\theta(z).}$$

所以显式能量和 score 可以通过梯度联系起来。

最大似然中的数据期望与模型期望

负对数似然 $L = \mathbb{E}_{p_{\text{data}}} E_\theta(z) + \log Z_\theta$ 的梯度为

$$\nabla_\theta L = \mathbb{E}_{p_{\text{data}}} \nabla_\theta E_\theta(z) - \mathbb{E}_{p_\theta} \nabla_\theta E_\theta(z).$$

梯度下降会降低真实数据处的能量，并提高模型负样本处的能量。第二项需要从当前模型采样，常借助 MCMC、重放缓存和有限步近似。[67]

Langevin：梯度下山，加一点随机探索

$$z^{(k+1)} = z^{(k)} - \eta \nabla_z E_\theta(z^{(k)}) + \sqrt{2\eta}\, \xi_k, \qquad \xi_k \sim \mathcal{N}(0, I).$$

第一项倾向走向低能量区域，噪声帮助探索。上式是单位温度的离散更新；有限步长和有限链长会带来采样误差。

能量、判别分数与 score 的关系

- GAN 判别器也给实数分数，但其监督是对抗分类；把分数直接叫能量，还需说明所定义的概率和训练方式。
- score diffusion 在每个噪声等级学习 $s(z,t) \approx \nabla_z \log p_t(z)$，通常直接输出向量。EBM 可先输出标量，再求梯度得到 score。
- EBM 可自然叠加能量来组合条件；生成成本受负样本采样和混合速度影响。视频的大维度使这些计算更重。

Diffusion / FM 常直接预测“往哪里走”；EBM 先描述“哪些地方概率高”，再通过能量梯度得到运动方向。

本章思考与进阶练习

3.1 预测参数化

题目 在线性路径 $t = 0.2$ 时，干净值预测误差为 0.1，对应速度误差多大?
解题思路 由 $\hat{v} = (z_t - \hat{z}_0)/t$，速度误差为 -0.5，平方误差为 0.25。这说明相同干净值误差在低噪声位置会被放大，需要结合权重和端点处理。

3.2 边缘场

题目 每一条训练插值都是直线，为什么减少采样步数仍会造成明显误差?
解题思路 模型回归多条配对路径的条件平均，得到的边缘速度随位置和时间变化，推理轨迹可弯曲。有限容量与数值积分又加入误差。Reflow 改配对，蒸馏改跨步映射，各自处理不同部分。

3.3 进阶：加权目标

题目 将时间采样从 $p(t)$ 改成 $q(t)$ 后，怎样保持原来的总体训练目标?
解题思路 在共同支撑且 $q(t) > 0$ 的位置，以 $p(t)/q(t)$ 乘原 loss 做重要性加权。若同时改了权重但未补该比例，优化目标本身也改变。高方差权重还会影响训练稳定性。

第四章　动力学与数值采样

训练目标给出一个场或预测量，推理需要把它转成状态更新。本章统一时间方向、score 与 velocity 的转换，比较一阶、高阶与多步方法，再分析模型误差和近似计算如何沿采样过程传播。

4.1　SDE、ODE、score、drift 与 velocity

SDE / ODE 描述演化规则；score、drift、velocity 是规则中的不同向量场。

采用 $t=0$ 数据、$t=1$ 噪声。前向加噪 SDE 为

$$dz = f(z,t)\,dt + g(t)\,dW_t.$$

f 是 drift，表示局部平均变化；$g\,dW$ 是随机扩散项。令 $s_t(z) = \nabla_z \log p_t(z)$，反向 SDE 在 $t:1\to 0$ 时写作

$$dz = [f(z,t) - g(t)^2 s_t(z)]\,dt + g(t)\,d\bar{W}_t.$$

$\bar{W}$ 是反向时间的 Brownian motion。与它具有相同单时刻边缘分布的 probability-flow ODE 为

$$\boxed{\frac{dz}{dt} = v(z,t) = f(z,t) - \tfrac{1}{2}g(t)^2 s_t(z).}$$

在理想精确场下，两条路径家族共享各时刻的分布；SDE 单条路径有随机扰动，ODE 给定起点后确定演化。[66]

由噪声预测恢复 score

对 $z_t = \alpha_t z_0 + \sigma_t \epsilon$，最优去噪预测给出

$$s_t(z_t) = \frac{\alpha_t \mathbb{E}[z_0 \mid z_t] - z_t}{\sigma_t^2} = -\frac{\mathbb{E}[\epsilon \mid z_t]}{\sigma_t}.$$

因此可用 $-\hat{\epsilon}_\theta/\sigma_t$ 估计 score。score 指向**概率密度增加的方向**，量纲是状态的倒数；velocity 是状态对时间的变化率。

同一高斯路径下，把 score 换成 velocity

在 $\alpha_t, \sigma_t > 0$ 的区间：

$$v(z,t) = \frac{\dot{\alpha}_t}{\alpha_t} z + \left(\frac{\dot{\alpha}_t}{\alpha_t}\sigma_t^2 - \sigma_t \dot{\sigma}_t\right) s_t(z).$$

它来自 $\dot{z}_t = \dot{\alpha}_t z_0 + \dot{\sigma}_t \epsilon$ 的条件平均。系数由噪声日程决定，所以 velocity、score 可以在明确日程下转换。

术语	直观解释
SDE / ODE	带随机扰动 / 确定性的演化方程
drift	SDE 的确定性局部变化部分
velocity	ODE 中状态每单位时间的变化率
score	对数概率密度对状态的梯度

4.2 预测转换、CFG 与采样成本

模型给方向，scheduler / solver 决定时间点和积分方式。

模型只输出 $\hat{\epsilon}$、$\hat{z}_0$ 或 $\hat{v}$，采样器把这些量转成新的状态。Euler 是一阶更新；Heun、多步求解器可用额外评估或历史估计减少数值误差。DDIM、DPM-Solver、UniPC 等必须与模型参数化、噪声日程匹配。

CFG：有条件与无条件预测的组合

以速度预测为例：

$$v_{\text{cfg}} = v_{\text{uncond}} + w(v_{\text{cond}} - v_{\text{uncond}}).$$

$w = 1$ 是原有条件预测；更大 w 往往增强条件作用，但可能引起过饱和、多样性下降或伪影。negative prompt 可替代空条件，通过指导方向影响结果。CFG 常需要两次分支评估，放在一个 batch 中执行时，仍要计算两个分支。部分蒸馏方法把指导效果烘焙进学生，从而避免推理时两分支；此时再外加很大 CFG 可能重复放大。[10, 18]

由预测结果转换干净 latent

噪声预测时：

$$\hat{z}_0 = \frac{z_t - \sigma_t \hat{\epsilon}}{\alpha_t}.$$

线性 FM 时：$\hat{z}_0 = z_t - t\hat{v}$。端点附近的数值稳定性、精度和权重要仔细处理，$\alpha_t = 0$ 的端点需使用对应的极限形式或稳定参数化。

端到端时间分解

$$T_{\text{total}} = T_{\text{text}} + T_{\text{encode}} + T_{\text{denoise}} + T_{\text{decode}} + T_{\text{IO}}.$$

纯 T2V 通常从文本和噪声出发，I2V 还包含参考图编码。若去噪占总时间的 90%，即使去噪无限快，整体加速上限也只有 $1/(1 - 0.9) = 10$ 倍。

NFE 统计网络评估次数；UI 中的 steps 通常统计采样器更新次数。比较收益必须统一 CFG 分支、solver 阶数、分辨率、帧数、batch、精度和是否包含 VAE。少步模型通过专门训练适应大跨度采样。

4.3 Euler 与 Heun 的数值积分

求解器用网络提供的局部方向，近似一段连续运动。

设 $s < t$，步长 $h = s - t < 0$，模型输出 $v_\theta(z_t, t)$。

Euler：用出发点的方向走完整步

$$k_1 = v_\theta(z_t, t), \qquad z_s = z_t + hk_1.$$

一次更新通常需要 1 次网络评估。若路径弯曲，大步长会偏离真实轨迹；减小步长可降低离散误差。

Heun：先试走，再平均首尾方向

$$k_1 = v_\theta(z_t, t), \quad \tilde{z}_s = z_t + hk_1, \quad k_2 = v_\theta(\tilde{z}_s, s),$$
$$\boxed{z_s = z_t + \tfrac{h}{2}(k_1 + k_2).}$$

通常每步 2 NFE。对光滑场，Euler 的全局误差为 $O(|h|)$，Heun 为 $O(|h|^2)$；网络估计误差仍然存在。

算一个数：$dz/dt = z$

从 $z(1) = 1$ 走到 $s = 0.5$，精确值是 $e^{-0.5} \approx 0.6065$。

- Euler：$1 - 0.5 \times 1 = 0.5$，1 NFE。
- Heun：试走到 0.5，再用两端速度的均值，$1 - 0.5(1 + 0.5)/2 = 0.625$，2 NFE。

这是数值积分例子。比较生成效果时，应固定总 NFE，再比较采样质量；10 步 Heun 和 10 步 Euler 的计算预算不同。

模型参数化与求解器接口

网络若输出 $\hat{\epsilon}$ 或 $\hat{z}_0$，scheduler 先按噪声日程转换成所需导数或更新量。时间网格、预测类型、终端噪声和 CFG 强度一起决定数值行为。

高阶求解器更精细地使用已学到的局部场；少步训练则让网络直接适应大跨度预测。二者优化的是积分误差和模型学习目标两个环节。

扩散 ODE 及专用高阶积分器背景见 [66, 69]。

4.4 DDIM 与 DPM-Solver 系列

固定预训练网络，用噪声日程的解析结构跨越更多时间。

统一写成 $z_t = \alpha_t z_0 + \sigma_t \epsilon$。这里 α_t 是信号系数；若某论文用 $\bar{\alpha}_t$，对应的信号系数常为 $\sqrt{\bar{\alpha}_t}$。

DDIM：保持同一噪声方向，重新组合信号与噪声

$$\hat{z}_0 = \frac{z_t - \sigma_t \hat{\epsilon}_t}{\alpha_t}, \qquad z_s = \alpha_s \hat{z}_0 + \sqrt{\sigma_s^2 - \nu_{t,s}^2}\, \hat{\epsilon}_t + \nu_{t,s}\xi.$$

$\xi \sim \mathcal{N}(0, I)$；$\nu$ 由采样随机性参数与日程决定。取 $\nu = 0$ 时，给定起始噪声后路径确定。它可复用 DDPM 去噪网络，在更稀疏时间点上更新。[68]

DPM-Solver：解析处理线性项，近似神经网络积分

定义半 log-SNR $\lambda_t = \log(\alpha_t/\sigma_t)$。相应 ODE 的精确积分形式为

$$z_s = \frac{\alpha_s}{\alpha_t} z_t - \alpha_s \int_{\lambda_t}^{\lambda_s} e^{-\lambda} \epsilon_\theta(z_\lambda, \lambda)\, d\lambda.$$

线性部分已被解出，只需近似积分里的网络函数。令 $h = \lambda_s - \lambda_t$，一阶公式为

$$z_s = \frac{\alpha_s}{\alpha_t} z_t - \sigma_s(e^h - 1)\hat{\epsilon}_t.$$

高阶版本利用额外评估或历史差分，更准确估计积分。原论文在 CIFAR-10 的设置中报告 10 NFE 的 FID 4.70、20 NFE 的 FID 2.87，体现计算预算与质量的权衡。[69]

DPM-Solver++：改用数据预测积分，适配 guided sampling

其一阶数据预测式为

$$z_s = \frac{\sigma_s}{\sigma_t} z_t + \alpha_s(1 - e^{-h})\hat{z}_0.$$

在相同日程与预测转换下，这与确定性 DDIM 的一阶更新相容。高阶 ++ 方法用数据预测、多步估计等设计改善大 guidance 下的稳定性。像素动态阈值可利用颜色范围；latent 取值范围则由编码器和缩放约定决定。[70]

相同一步公式可以对应不同训练方式。普通 DDIM 复用原噪声模型；PCM 等方法会继续训练网络，使它适应跨区间映射。

4.5 多步法、UniPC 与 DEIS

缓存过去几步的网络结果，用插值和校正提高每次评估的价值。

多步法的共同思想

假设已保存 $\hat{\epsilon}_{t_n}, \hat{\epsilon}_{t_{n-1}}, \ldots$。把网络随时间的变化拟合为低阶多项式，代入积分公式：

$$z_{n+1} = \Phi(t_{n+1}, t_n) z_n + \sum_{j=0}^{p-1} c_j \hat{\epsilon}_{t_{n-j}}.$$

Φ 负责线性演化，c_j 来自日程和插值积分。历史预测保存为小量额外状态，新一步通常只新增一次网络计算；起步阶段从低阶逐渐升阶。

DEIS：Diffusion Exponential Integrator Sampler

DEIS 利用扩散 ODE 的半线性结构，解析积分线性部分，对神经网络项做高阶近似；其多步版本利用过去模型输出构造积分。收益来自更有效地使用 NFE，训练权重保持不变。[72]

UniPC：统一 predictor 与 corrector

UniP 预测下一状态，UniC 在取得新模型输出后修正积分近似；校正环节复用已有评估，因此可在其设计的调用流程中提高阶数而维持 NFE 预算。原论文给出 CIFAR-10 无条件 10 NFE / FID 3.87 的结果。[71]

方法	利用什么结构?	选用时检查什么?
DDIM	当前去噪估计 + 日程	随机性参数、步长网格
DPM-Solver	噪声预测积分	单步 / 多步、阶数
DPM-Solver++	数据预测积分	CFG、阈值、模型输出类型
UniPC	预测 + 校正	corrector 调用、起步与末步处理
DEIS	指数积分 + 历史插值	插值变量、时间网格、精度

极少步条件下的高阶近似误差

大跨度内网络场可能明显弯曲；强 CFG 还会改变局部刚性。历史多步法在开始时只有一个评估，缺少可用历史。若目标是稳定的一步生成，通常会进入 B/C 类，让训练本身适配大跨度。

论文中的 FID 来自各自模型、数据和评估设置。工程选择可在同一 checkpoint、同一提示词/种子、相同 NFE 下绘制质量–延迟曲线。

4.6 采样误差、模型误差与近似计算的共同传播

设参考 ODE 的速度为 v，实际计算为 $\tilde{v} = v + \delta$。在间隔长度 $h = |\Delta t|$、v 的局部 Lipschitz 常数为 L 时，一阶更新的误差可估计为

$$e_{n+1} \le (1 + hL)e_n + h\|\delta_n\| + Ch^2.$$

三项依次表示旧误差放大、当前速度预测偏差、Euler 截断误差。对 p 阶方法，局部截断项换成 $O(h^{p+1})$，模型偏差项仍然存在。高阶求解器主要减少数值积分误差；当网络误差占主导，继续提高阶数的收益会减小。

少步采样下的近似预算

特征缓存、量化、稀疏 attention、大小模型切换都会改变 δ_n。步长越大，同一次速度偏差对终点位移的影响通常越明显。先对未近似模型选好采样预算，再逐项引入优化，并对联合方案重新测量。逐项质量稳定的优化，组合后仍可能因误差相关而破坏运动或身份。

CFG 对误差的影响

若条件与无条件分支误差为 δ_c, δ_u，CFG 组合误差为

$$\delta_{\text{cfg}} = w\delta_c + (1 - w)\delta_u.$$

大 w 可能放大分支间误差。对 guided sampling 做量化校准或蒸馏时，需要覆盖实际 guidance 范围；多步法还依赖历史预测的一致性。切换条件、模型或分辨率之后，求解器的历史项应按对应方法重新初始化或转换。

本章思考与进阶练习

4.1 NFE 计数

题目 20 个 Heun 区间，每区间两次网络预测，CFG 的两个分支分别执行。共有多少分支前向?
解题思路 按题设为 $20 \times 2 \times 2 = 80$ 次。批量合并 CFG 可以减少启动次数但仍计算两组样本；末步简化或复用预测的实现会改变计数，应单独注明。

4.2 反向时间

题目 $t = 0.8$，$z = 2$，速度预测为 3，走到 $s = 0.6$，Euler 输出是多少?
解题思路 $z_s = z + (s - t)v = 2 - 0.2 \times 3 = 1.4$。速度定义朝加噪方向，采样用负的时间差；换成去噪方向速度后应同时改变符号约定。

4.3 进阶：误差叠加

题目 缓存和低精度分别只带来小误差，组合后却失稳。提出两个机制和验证方法。
解题思路 其一，缓存保留了量化后的有偏状态；其二，大 CFG 放大两分支的相关偏差。固定同一参考轨迹测每步速度误差，再做 baseline、缓存、量化、组合四组终点实验，区分局部偏差和 rollout 放大。

第五章　视频时间建模与流式生成

视频除了去噪时间，还拥有事件时间与生成顺序。本章依次处理因果可见性、训练历史、长时漂移、缓存、分块边界和流式超分；少步分布目标的数学细节在第八章展开。

5.1　自回归分解、因果可见性与训练历史

AR 决定“先生成哪段”，diffusion 决定“这一段怎样生成”；两者可以同时存在。

视频按 chunk 分解为：

$$p(z_{1:K} \mid c) = \prod_{k=1}^{K} p(z_k \mid z_{<k}, c).$$

每个条件分布又可以由少步/多步 diffusion 实现。因此 AR 与 diffusion 可以在时间分解和块内生成两个层次协同工作。

因果和分块因果的区别

逐 token 因果 mask 只允许位置 i 看见 $j \le i$。分块因果允许同一 chunk 内双向交互，但禁止看未来 chunk：

$$M_{ij} = \begin{cases} 0, & \text{chunk}(j) \le \text{chunk}(i), \\ -\infty, & \text{其他。} \end{cases}$$

这样一个 chunk 可以同时去噪，chunk 之间仍按顺序生成。时间首帧、文本和条件 token 的 mask 可能另有规则。

Teacher、Diffusion 与 Self Forcing

策略	历史状态	主要问题/优点
Teacher Forcing	真数据提供的历史	训练快，推理却面对自己的误差
Diffusion Forcing	不同位置可有不同噪声等级	灵活覆盖干净/带噪上下文组合
Self Forcing	当前模型自行 rollout 的历史	更接近实际推理状态，训练更复杂

Diffusion Forcing 按位置分配噪声等级；attention mask 另行决定可见性。TF / DF / Self Forcing 与 KV 缓存也可按训练条件组合。[26, 28]

On-policy 状态分布

要看训练的状态是否来自当前策略。固定教师视频集提供离线监督；当前学生自行 rollout 的状态构成 on-policy 训练分布。视频里状态还包括历史 cache、噪声时间和当前 noisy latent，不止最终画面。

判断一个 AR diffusion 训练法：画出“历史来自谁、未来能否看见、哪一步有梯度、KV 何时写入”。这比仅记 TF/DF/AR 缩写更可靠。

TF 的输入、监督与一次迭代

沿用 $t=0$ 为干净、$t=1$ 为噪声的约定。真实第 k 个 chunk 为 $z_{k,0}$，采样 ϵ_k 与噪声时间 t_k，构造

$$z_{k,t_k} = (1-t_k)z_{k,0} + t_k\epsilon_k, \qquad u_k = \epsilon_k - z_{k,0}.$$

Teacher Forcing 的 FM 训练目标可写成

$$L_{\mathrm{TF}} = \mathbb{E}\sum_{k\in\mathcal{T}} w_k \left\| v_\theta(z_{k,t_k}, t_k, z_{<k,0}, c_{\le k}) - u_k \right\|^2 .$$

$\mathcal{T}$ 是本次被监督的目标块集合。历史由数据提供，当前块加噪；计算当前速度误差并更新主干。历史编码可参与反传，也可按实现 detach，二者要在计算图中注明。推理时，历史换成 $\hat{z}_{<k,0}$，其误差不再是训练时精确的真历史。

并行 TF 为什么可能需要两路 token

若一次同时训练第 2、3、4 块，第 2 块既要作为自身的 noisy target，又要作为第 3 块的 clean history。只放一份第 2 块张量无法同时满足这两个角色。可选实现包括逐目标训练、复制前缀，或构造 clean/noisy 两路 token：

- clean 路由真实块编码，按因果规则形成历史特征；
- noisy 目标 k 读取 clean 路的 $j<k$，并读取自己的 noisy 块；
- noisy 目标禁止读取对应 clean k 或未来块，避免答案泄漏；clean 路本身也禁止把未来信息带回历史。

这是通用 TF 计算图说明，具体模型的 token 排布和 mask 可以不同。该两路输入也与第二章的“文本/视觉双流主干”属于不同概念。

DF：每个位置有自己的噪声时间

Diffusion Forcing 用时间向量 $\mathbf{t}=(t_1,\dots,t_K)$，让不同块有不同不确定性：

$$L_{\mathrm{DF}} = \mathbb{E}_{z,\epsilon,\mathbf{t}} \sum_k w(t_k) \left\| v_\theta(z_{1,t_1},\dots,z_{K,t_K}, \mathbf{t}, c; M)_k - u_k \right\|^2 .$$

M 决定可见性。原始 DF 将逐 token 噪声与序列模型组合，使训练覆盖大量噪声组合；后续视频实现可采用 chunk-causal Transformer。时间向量决定“各位置有多清楚”，mask 决定“哪些位置能互相读取”。[26]

配置	4 个位置的 t 示例	当前可见信息
整段同步去噪	$(0.6, 0.6, 0.6, 0.6)$	通常整段双向
TF 训练第 4 块	$(0, 0, 0, 0.6)$	真历史 + noisy 当前块
一般 DF	$(0.2, 0.8, 0.1, 0.6)$	由因果或其他 mask 决定
单调噪声流水线	$(0, 0.2, 0.5, 0.8)$	更早块更干净，多个块可并行推进
Self Forcing	$(0, 0, 0, 0.6)$	前三个 0 对应学生生成值

以上为教学噪声表。TF 与 Self Forcing 即使使用相同的时间向量，历史值的分布仍不同。DF 的带噪真历史通常由前向噪声核构造；模型自身的身份漂移、运动失真和语义错误具有不同结构，因此 DF 与 on-policy rollout 各提供一类训练覆盖。

训练并行度、采样波前与 cache 写入

DF 训练可以一次对多个位置回归速度；推理时选择一个二维 schedule：横轴视频位置，纵轴去噪时间。可以先完成一个 chunk 再启动下一个，也可以让多个 chunk 处于不同噪声级，形成对角线波前。改变 schedule 后，应检查训练是否覆盖对应组合。

只有在后续步骤中保持相同输入状态和条件的历史特征，才适合精确复用。一个尚未去噪完成的块，其 K, V 会随 t 和 latent 改变；需重算或保存与去噪阶段匹配的 cache。跨步特征复用则带有近似误差。这个区别直接关联 MAGI-1 的流水线和 Vidu S1 的双缓存。



图 5.1 TF、DF 与 Self Forcing 的训练状态。时间标签可以相同，历史的来源仍然不同。**教学示意。**浅色表示低噪声；框内数值为本书噪声时间 t。mask、历史来源与噪声表分别核对。[26, 28, 158]

5.2 时间分块与分辨率级联

沿视频时间拆块，沿分辨率分层，分别改变生成顺序和任务难度。

Causal / Streaming Diffusion：时间方向上的条件分解

$$p(z_{1:K} \mid c) = \prod_{k=1}^{K} p(z_k \mid z_{<k}, c).$$

每个条件分布由 diffusion 或 flow 生成。当前块读取历史 KV，完成后写入缓存，再生成下一块。流式系统还要让 VAE、refiner、缓存和输出传输按同样节奏工作。

AR block generation：块内并行，块间顺序

若一块含 B 帧，块内 token 可以双向交换信息；块间按先后生成。大块提供更多同时可见的运动信息，降低块间调用频率；小块降低首包等待，但加强了历史依赖和接缝处理需求。

$$T_{\text{first}} \approx T_{\text{cond}} + T_{\text{first block}} + T_{\text{decode/lookahead}}.$$

稳态吞吐由流水线最慢阶段决定，首帧延迟则还包含预热与上下文等待。

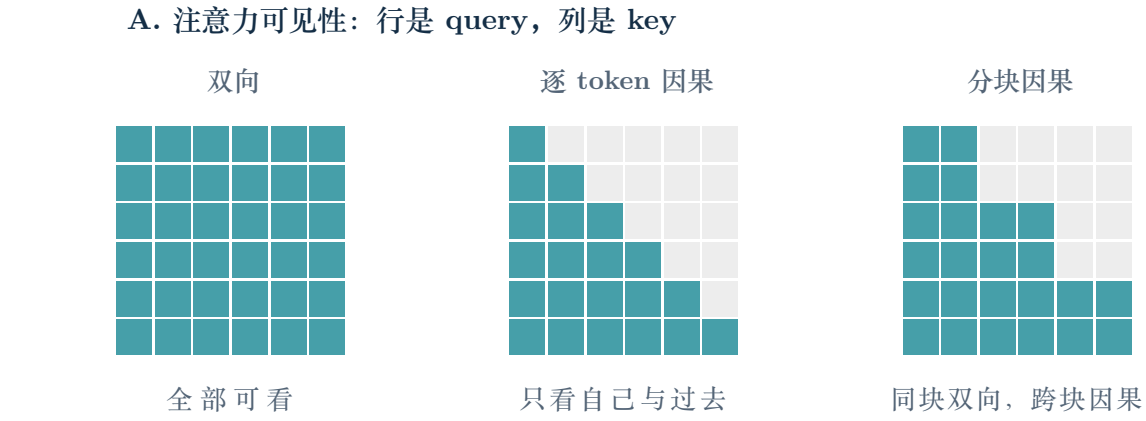


图 5.2　因果 mask、噪声等级与训练历史。“允许看哪些位置”“每个位置有多少噪声”“历史由谁生成”是三个独立维度。**判断流式模型：**除了 mask，还要查 VAE、位置编码、缓存写入时机和噪声时间条件。分块因果可以让当前块内部双向计算，但仍有至少一个块的等待延迟。[26, 28, 9]

Coarse-to-fine / Cascade：分辨率方向上的条件分解

令 $x^{(0)}$ 是低分辨率视频，$x^{(L)}$ 为最终高清结果：

$$p(x^{(0:L)} \mid c) = p(x^{(0)} \mid c) \prod_{\ell=1}^{L} p(x^{(\ell)} \mid x^{(\ell-1)}, c).$$

低清基模决定大尺度结构、主体运动；后续空间超分提升细节，时间超分增加帧率。Imagen Video 采用由基础模型、空间和时间超分组成的级联。[92]

级联的计算分配

把昂贵的全局内容建模放在低分辨率，高清阶段只处理受低清结果约束的恢复任务。训练时用低清条件退化或加噪等方式，让 refiner 适应基模误差；推理时每一层都贡献额外延迟。

生成组织与其他加速路线的组合

一个系统可以按 AR 块流式生成低清视频，每块用 MeanFlow / PCM 少步生成，主干执行 FP8 与稀疏 attention，再由一步 refiner 输出高清。评估时分别测量块边界、运动一致、细节、首包延迟与稳态吞吐。

时间分块解决“持续往后生成”；分辨率级联解决“把更多算力留给需要细化的地方”。

5.3 CausVid、Self Forcing 与 Causal Forcing

它们围绕“把高质量视频教师变成因果、少步、可流式学生”，但修补的训练缺口不同。

CausVid：双向教师到因果少步学生

用因果 AR 学生支持流式生成与 KV 缓存，用强双向 diffusion 教师提供分布监督，结合初始化与 DMD 风格蒸馏。其核心是**视频顺序上因果，每个 chunk 内连续去噪**。[27]

Self Forcing：用学生自身历史练习

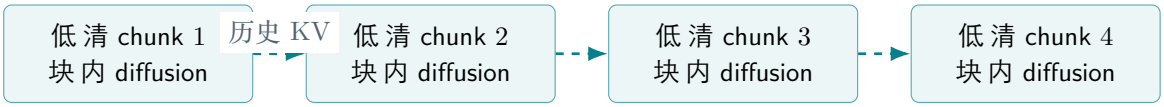
训练时用学生自己生成的 chunk 做下一段的上下文，和实际推理一样使用 KV。对生成序列进行分布匹配监督，并用截断梯度等机制控制开销。相比 TF/部分 DF 设置，重点是减少“训练历史过于干净、推理历史有自身误差”的差距。[28]

Causal Forcing：初始化的教师也要符合因果信息条件

若用双向教师的 ODE 轨迹直接初始化因果学生，教师可能借助学生不允许看到的未来信息。Causal Forcing 先把基础模型适配成 AR diffusion 教师，再使用因果相容的 ODE 初始化，之后继续分布蒸馏。[29]

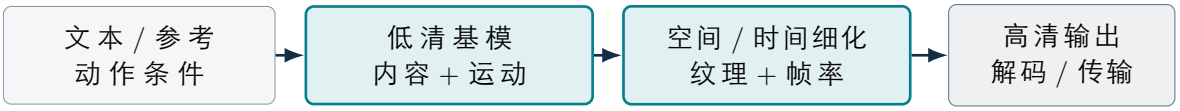
方法	强调的缺口	配套训练要点
CausVid	双向慢模型转为因果少步生成	进一步适配长期历史分布
Self Forcing	历史来自学生自身	配合长 rollout 与缓存管理
Causal Forcing	教师/学生可见信息与初始化相容	因果适配、ODE 初始化与分布蒸馏

A. 时间分块：块内并行，块间按顺序

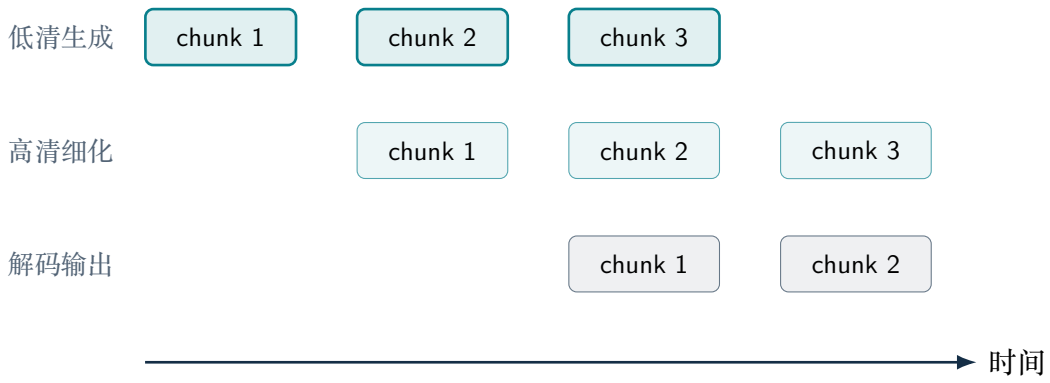


大块：同时看到更多运动上下文。小块：更早交付结果。

B. 分辨率级联：低清内容到高清细节



C. 组合成流式流水线



方块宽度用于表达依赖关系；流水线吞吐由最慢阶段与资源竞争决定。

图 5.3 时间分块与分辨率级联。低清流式基模与高清细化模块可以组成分阶段输出系统。**部署观察：**首包延迟取决于前置依赖，稳态吞吐取决于流水线瓶颈；同时检查块边界与时序质量。[28, 30, 92]

Self Forcing 的一轮训练计算图

初始化后，把学生视为共享权重的少步去噪网络，依次生成各 chunk。对第 k 块，状态为 $(z_{k,t}, t, H_{k-1}, c)$。原方法随机选择本轮停止的去噪步，以该步的干净预测作为输出；之前的去噪调用不保留梯度，保留最后选中步骤的计算图，并阻断梯度流入历史 KV。

$$H_{k-1} = \mathrm{sg}[\mathrm{KV}(\hat{z}_{<k})], \qquad \hat{z}_k = F_\theta(\mathrm{sg}[z_{k,t_j}], t_j, H_{k-1}, c).$$

F_θ 在这里表示从预测量转换出的干净 chunk。将多个 $\hat{z}_k$ 拼成视频后施加整体分布损失；不同 chunk 的当前可导分支都可以收到反馈。原论文验证 DMD、SiD 和对抗目标，随机停止使不同去噪步在训练过程中都有机会得到监督。[28]

这种截断保存了“学生历史”这一输入分布特征，同时省去完整多步和跨 chunk 反传。它引入了梯度近似，适合在已有能力上做后训练；长期信用分配和远超训练长度的漂移仍需进一步处理。

Causal Forcing 的三阶段接口

先以 Teacher Forcing 适配 AR diffusion 教师，再从该教师采集因果相容的 ODE 配对，最后做 Self-Forcing 风格分布蒸馏。若学生可见状态为 o、教师目标为 y_T，MSE 回归的最优值是 $\mathbb{E}[y_T \mid o]$。当双向教师依赖隐藏未来时，同一个 o 可对应多个目标；平均结果容易丢失动态细节。将教师可见信息与学生对齐，可以减少这一初始化歧义。[29]

可见信息改变后的初始化

蒸馏学生不仅要改步数，还可能改 attention 拓扑。一个原本依靠未来帧修正当前帧的双向模型，被切断未来后，原来的表示和速度估计会失配。良好初始化让后续分布优化不必同时修复全部结构变化。

5.4 SandAI MAGI-1：单调噪声与分块流水线

MAGI-1 从预训练阶段建立 AR diffusion 的时间组织。视频切成 chunk，块内 full attention，块间因果；各块独立噪声、噪声强度沿未来方向单调增加。原论文时间方向与本书相反；统一到本书约定后可写为

$$0 \le t_1 \le t_2 \le \cdots \le t_K \le 1, \qquad L = \mathbb{E} \sum_{k \in \mathcal{N}} w_k \| v_\theta(z_{\le k, \mathbf{t}}, \mathbf{t}, c)_k - u_k \|^2.$$

$\mathcal{N}$ 是 noisy chunks；clean chunks 参与 attention，但原方法只对 noisy chunks 施加去噪 loss。clean 部分依然可通过后续目标收到梯度，论文发现直接阻断其梯度会损害效果。[158]

一个 chunk 未完成，下一个就能开始

论文每块包含 24 个原始帧（24 FPS 下约 1 秒），最多同时推进 4 个 chunk。较早块达到一定去噪程度后即可启动下一块，形成“旧块较清楚、新块较嘈杂”的流水线。它能提高并行利用率；单个块的多步去噪与上下文依赖仍保留。

原论文提供 4.5B、24B 模型；从低分辨率到高分辨率逐阶段训练，并联合使用图像和视频。T2V、I2V 与视频续写通过提供不同长度的干净前缀统一。对 clean 前缀去掉文本依赖有助于复用视觉历史，加入小幅噪声增强对不完美前缀的容忍；这些细节与只修改 attention mask 的事后因果化不同。[158, 159]

从算法到系统

当 mask 包含 clean history、多个噪声块和多样文本区域时，训练负载与通信不均匀。MagiAttention 针对长上下文与异构 mask，提供可表达这些区域的 attention kernel、细粒度负载均衡和通信重叠。它属于分布式 attention 执行系统，与 MAGI-1 的生成概率分解分别理解。第十章讨论其通信路径。[160]

5.5 LongLive：长历史训练与局部监督

它把自生成历史、局部蒸馏监督和缓存管理组织成长视频训练流程。

训练短 clip 的模型即使能滚动推理，也可能没见过“连续生成 40 秒后”的受污染历史。LongLive 用 streaming long tuning 让训练也跨越长时间。

LongLive 的逐窗口训练

以论文使用的 5 秒监督窗口为例：生成 0–5 秒，算 DMD 更新；保留历史 KV 并 detach；继续生成 5–10 秒，只监督新片段；反复到预设时长后重置。

$$H_{k-1} = \mathrm{sg}[\mathrm{Cache}(\text{已生成历史})], \quad Y_k = G_\theta(\epsilon_k, H_{k-1}, c_k), \quad L_k = L_{\mathrm{DMD}}(Y_k, c_k).$$

教师只处理自己擅长的短片段；学生却接触长历史。5 秒对应监督 clip，其中包含多个 AR chunk。原实现 chunk 为 3 latent frames。[30]

短窗口 + frame sink

仅保留近期历史和开头少量帧作为锚点，限制 cache/attention 的增长。它提供局部连续与长期外观锚定，更久远的事件信息仍需额外记忆机制支持。

提示切换与 KV 重算

旧 KV 的隐藏表示可能已受旧文本影响。完全清空会断连，完全沿用可能不听新指令。于是保留已生成视觉上下文，在新 prompt 下重算缓存。**已交付的视频保持固定，更新的是条件相关特征。**训练也包含切换行为。[31]

训练资产与推理收益

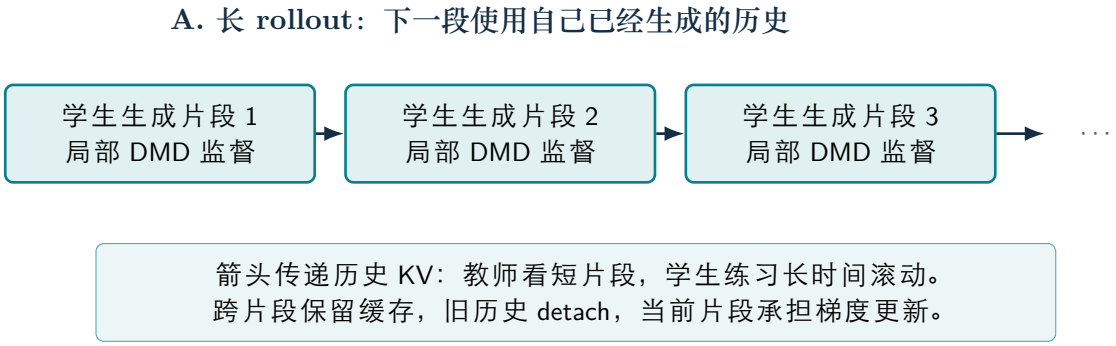
原版基于 Wan2.1-T2V-1.3B；先做 ODE/Self Forcing 初始化，长视频阶段用 Wan2.1-T2V-14B 教师和 LoRA 调优。长视频阶段使用生成样本进行蒸馏，基础模型则已从真实视频中学到生成能力。官方报告单 H100 的 20.7 FPS 与最长 240 秒演示。官方 FPS 统计去噪生成，VAE 解码时间需另计。[33]

LongLive-Plug 是另一个问题

后续 LongLive-Plug 研究把少步、CFG 和长视频等能力蒸馏成可复用的 adapter，减少对兼容下游模型逐个重蒸馏的需求。复用时需要兼容底座、adapter 接口和 attention 配置。[34]

5.6 FramePack 与有限历史预算

根据历史重要性分配表示分辨率，使长视频的上下文成本更可控。



一个 5 秒监督窗口包含多个 chunk。图中缓存长度与时间比例仅作示意。

图 5.4 LongLive：长期练习，局部监督。学生经历长历史，教师只评价适合自己的短片段；缓存长度保持可控。**训练与状态：**截断梯度把优化限制在当前训练窗口；frame sink 提供视觉锚点；re-cache 更新条件特征，保留已经交付的视频。[30, 31]

历史增长与计算预算

下一帧块预测通常读取历史 $H_k = (z_1, \ldots, z_{k-1})$。完整保留会使输入长度和注意力成本持续增长；固定滑窗能约束成本，但窗口外的细节会消失。FramePack 研究按重要性压缩输入帧，使模型在有限 token 预算内利用较长历史。[118]

一个预算示例

用 j 表示由重要到次重要的历史层级。教学示例取每层 $n_j = n_0 r^j$ 个 token，$0 < r < 1$，则

$$\sum_{j=0}^{J-1} n_j = n_0 \frac{1 - r^J}{1 - r} \leq \frac{n_0}{1 - r}.$$

例如 $n_0 = 256, r = 1/2$，前四级为 256、128、64、32，总计 480。实际使用离散 patch 尺寸、有限层级和选择规则。这里的几何预算只展示压缩逻辑；论文实现还包含具体 packing 配置和采样策略。

历史信息、遗忘与漂移

重要性：近期内容通常与下一段联系更强，也可结合特征相似度或其他选择规则。**遗忘**：压缩历史帮助保存更久远的上下文，但压缩程度决定能保留多少细节。**漂移**：每一段的预测误差会影响后续段，论文另外研究生成顺序、端点与 anti-drifting 策略。[118]

机制	保存什么	主要作用
滑动 KV cache	近期 token 的 K/V	复用特征、限制历史长度
历史压缩	不同精度的视觉历史	在预算内覆盖更长历史
Self-Forcing	学生自己生成的上下文	缩小训练与推理的历史分布差异
LongLive	流式长视频训练与更新机制	支持长时生成及条件变化

比较收益的合适方式

同时测峰值显存、每段延迟、总时长和跨段身份 / 场景一致性。固定历史输入预算解决的是增长成本；随生成时长变化的漂移曲线展示长期质量。与数据 batch 中的多分辨率 packing 比较时，分别标明“历史压缩”与“训练样本组织”的含义。

5.7 分块接缝、归一化与参考 KV

先定位不连续出现的层，再检查归一化、条件注入、位置编码与缓存。

闪烁的工程排查顺序

1. **VAE 边界**：整段解码与分块解码同一 latent 是否一致？卷积历史缓存、首帧 padding、时间上采样和重叠融合是否正确？
2. **因果与位置**：训练/推理的 mask、RoPE 位置偏移、首 chunk 规则是否一致？
3. **KV 状态**：cache 来自干净帧还是某个噪声时间？切 prompt 后是否混入旧条件？是否有过期或重复位置？
4. **噪声条件**：TF/DF 的逐帧 t 与 AdaLN 调制是否正确广播？把 $[B, T]$ 误当 $[B]$ 会改变整个 chunk 的条件。

5. **归一化轴**：是否跨 chunk/时间统计均值？滑动窗口改变统计量会产生亮度或色彩跳变。
6. **训练分布**：模型只见过干净真历史，却被迫在长期自生成历史上推理，可能出现累积漂移。

归一化替换的前提与诊断

只有当问题确实来自条件调制或统计实现，且重新训练/适配后，替换才可能有效。标准 token-wise LN 通常沿通道归一化，本身不跨时间。AdaLN 额外表达 t 与 c，直接移除可能破坏去噪能力。建议先做：固定同一 latent，比较全序列与分块输出；记录边界前后均值/方差、调制系数和 cache 内容。[5, 9]

I2V 和 R2V 的语义差别

I2V 常把给定图像当第一帧；R2V 的参考图可能只规定身份/外观，不要求作为视频的首帧。把参考图的 K/V 注入历史或专用条件区，可以传递视觉特征，但还需处理位置、噪声级、条件 mask、参考与动态历史的角色差异。

参考 K/V 提供视觉条件；多参考角色绑定和非首帧身份保持还需要相应能力。推理改造后，分别测试身份、构图泄漏、运动自由度与位置处理。

长历史与 prompt-conditioned cache 的具体例子见 LongLive 和 OPSD-V[30, 37]。上述诊断清单为基于机制的工程建议。

5.8 Vidu S1 与 Vivix：在线条件、长时稳定和交互

生数 Vidu S1：双向预训练、TF/DF 因果适配、少步蒸馏

S1 报告把音视频联合状态用于流式生成，训练包含三个阶段：双向生成先验；从双向模型初始化因果教师，并在每个样本上选择 TF 或 DF；最后用 DMD 与 PCM 感知一致性正则得到少步生成器。DMD 提供分布匹配方向，PCM 帮助抑制激进蒸馏中的退化，数学定义分别见第八、七章。[52]

TwinCache。对于生成历史，保存一个指定去噪阶段的 noisy cache 和最终的 clean cache。中间去噪步读取 noisy 历史，最后一步读取 clean 历史；持久参考区由用户首图与首个生成音视频状态形成。这个选择让连续运动传播和最终外观细化使用不同历史信息。

RoPE Repositioning。历史特征在施加 RoPE 前缓存。窗口推进时根据更新后的窗口坐标施加位置编码，保持位置落在训练范围内，避免重新跑全部历史主干。它处理位置状态；prompt 更新、音频条件变化等仍要检查缓存是否依赖旧条件。

输出口径。论文报告 3 步、540p、RTX5090 GPUs 配置下平均 42 FPS，并结合 TurboDiffusion 的 worker 优化和 TurboServe 的集群服务。报告的该处未明确给出单请求 GPU 张数，因此表述保留复数 GPUs；它不能直接作为单张 5090 的吞吐。[52]

Vivix-A1：把参考信息保持在流式生成环中

A1 技术报告强调原生多模态输入、局部连续先验与全局序列先验。视觉参考、声音与互动历史持续影响后续细粒度输出；参考可以规定身份、声音、环境和交互对象。其蒸馏描述包括少步化到 2 步、长期自分布一致性，以及 latent 与原始数据空间的联合优化。报告公开了机制层次与演示，具体 loss 权重、教师结构和优化器配方仍需对应后续代码/论文。[171]

Vivix-W1 与 IVI：生成模型和运行时的分工

W1 报告侧重多镜头音视频联合生成与实时输入，展示配置约 30B active parameters；Vivix-Turbo 同时处理少步、长时误差、分镜连续与参考一致性。IVI 则把离线可控内容、实时生成和 director 组合成运行时，决定何时切换、延长与调整情节。生成模型按连续状态生成，运行时处理交互节奏与制作控制，两层可以联合工作。[172, 173]

统一的系统状态表达

下面是比较这些系统的教学抽象，定义在线状态 $s_k = (H_k, r, u_{\le k}, m_k)$：历史缓存 H_k、参考 r、到当前的用户信号 u、叙事/记忆状态 m_k。系统循环为

$$a_k = \pi(s_k), \qquad \hat{z}_{k+1} = G_\theta(\epsilon, s_k, a_k), \qquad s_{k+1} = \mathrm{Update}(s_k, \hat{z}_{k+1}, u_{k+1}).$$

π 可以是规划器，也可以退化为直接编码当前控制；这一表达不指定上述系统的内部模块。测量要把输入到反应的 TTFR、稳定 FPS、长时漂移和指令响应分开。很高的离线 FPS 并不能消除一个长 chunk 已被锁定造成的响应等待。

5.9 流式 refiner 与 SoL-Refiner

低分辨率基模负责内容运动，高分辨率 refiner 负责细节；一步描述 refiner 自身的采样预算。

一般的两阶段架构

$$c \xrightarrow{\text{低分辨率生成器}} X_{\mathrm{LR}} \xrightarrow{\text{条件 refiner}} X_{\mathrm{HR}}.$$

refiner 可是恢复网络、条件 diffusion、flow 模型或其他生成式超分。它可能在上采样 latent 加噪后去噪，也可能以 LR 作为额外条件；不同实现使用不同条件接口。

流式与双向的上下文范围

- 双向 refiner 可以查看当前处理区间的未来帧，用更多上下文增强一致性，但需要等待。
- 因果/流式 refiner 随低清 chunk 到达就处理，依赖历史缓存、边界状态与可控 lookahead。
- 块内双向计算需要等待当前块到齐；整条链路的延迟还取决于 VAE 与其他模块的未来帧需求。

SoL-Refiner 的已核对定位

SoL-Refiner 是一步高分辨率视频精炼模型，训练分三阶段：高分辨率持续训练、以逐帧奖励为基础的 RL 后训练、最终一步蒸馏。它还提出共享输入比较的 Refiner-Bench，用不同基模的低清输出来评测 refiner。[49] 作者报告在其 2K 延迟设置下，**完整加速栈**相对指定的三步 LTX-2.3 Refiner 达到 8.91 倍精炼加速。该倍数统计精炼阶段的完整优化栈，包含一步化以外的系统优化；基模耗时另计。

训练 refiner 时常见的目标类别

有配对 HR/LR 数据时可以使用重建/感知监督；生成式 refiner 可用去噪/FM 监督；后训练可用 reward；蒸馏可匹配少步映射或分布。**具体组合由配对数据、训练阶段与质量目标决定。**

“4K 输出”要看实际像素尺寸；“实时”要同时检查首帧延迟、稳态吞吐、VAE 解码、lookahead 和基模速度。完整系统吞吐由最慢阶段和等待时间共同决定。

H3-Regenerate 与 MagiHuman latent 超分

H3-Regenerate-2K 把 Base 的 768p 输出和原始多模态上下文重新送回 H3，使高分辨率生成仍可读取参考、文字与声音。它复用基模生成能力，公开流程将其列为独立运行模块。[165]

MagiHuman 在 latent 中三线性上采样、加噪，再以独立超分 checkpoint 执行 5 个去噪步。超分主干同时读取音频，实际采样只更新视频 latent，音频由基模结果提供辅助条件；1080p 阶段还使用局部 attention 控制成本。两者都把条件信息带进高清阶段，但具体状态更新与 checkpoint 组织不同。[161]

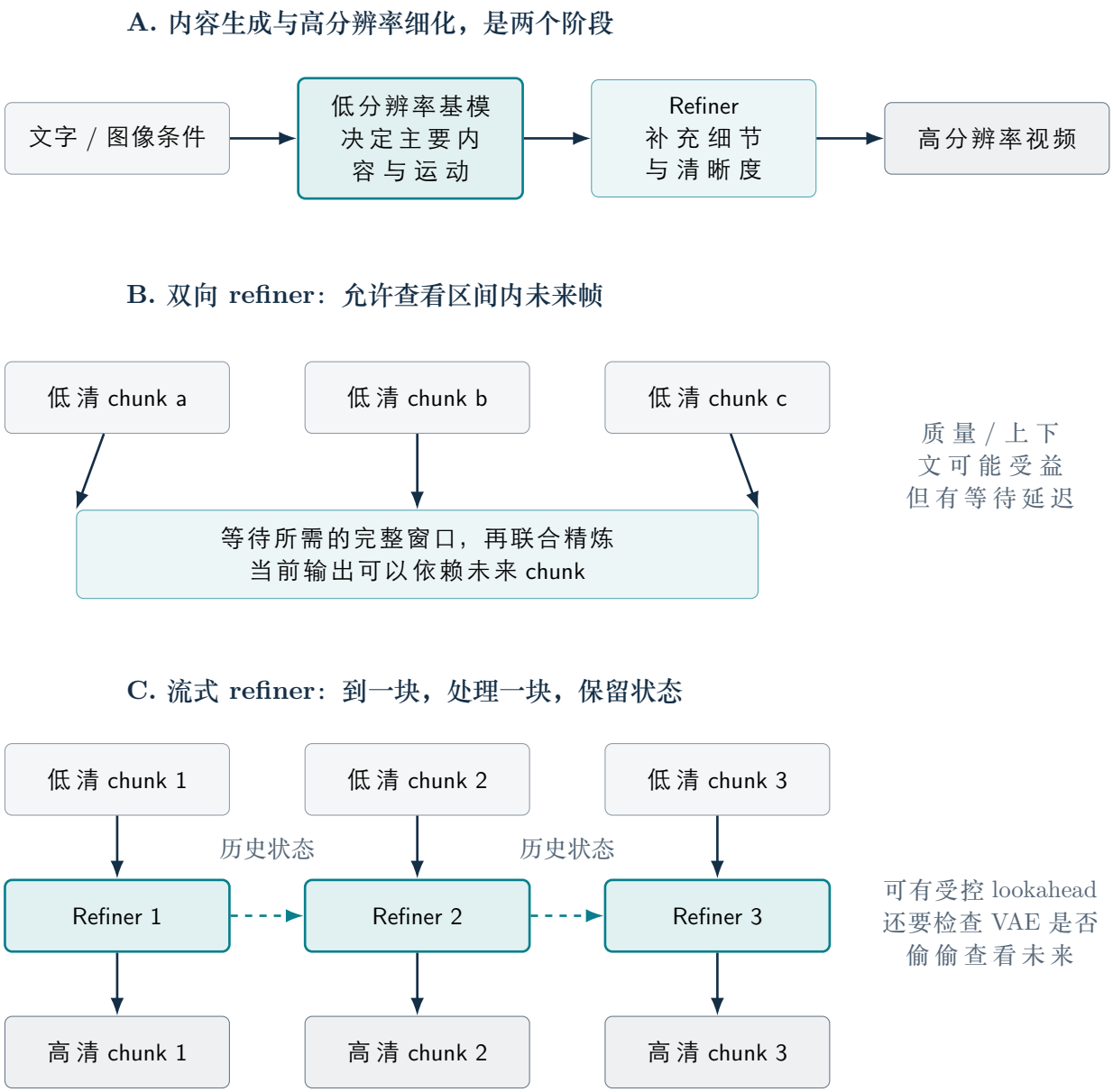


图 5.5 Refiner：高清细化怎样变成流式?。双向看完整窗口；流式按块推进，依靠历史状态与受控 lookahead。**计时范围：**完整链路包含基模生成、上下文等待、refiner、VAE 解码和传输。图示展示通用分块精炼流程；SoL 的具体组件见原论文。[49, 9]

本章思考与进阶练习

5.1 三个时间量

题目 区分视频帧时间、去噪时间和生成 chunk 序号，并说明位置编码与 AdaLN 各使用什么。
解题思路 帧时间描述事件在视频里的位置，chunk 序号描述生成组织，去噪时间描述噪声水平。时空位置编码主要描述前者；AdaLN 常读取去噪时间与条件。逐 chunk 采样会把后两者组合进状态。

5.2 梯度截断

题目 长期 rollout 中把历史 cache detach，模型是否仍能学习应对自身历史错误?
解题思路 能。后续输入仍来自学生生成分布，当前片段梯度可以训练对污染历史的响应；它省去了穿过所有历史的反传，因而当前损失无法直接修正那些旧历史生成步骤。

5.3 进阶：接缝定位

题目 如何区分 VAE 分块解码接缝与生成主干的内容跳变?
解题思路 先固定同一整段 latent，对比整段解码和带缓存分块解码；若接缝只在后者出现，定位解码边界。再比较 chunk 交界处 latent、位置和调制参数，使用统一解码器观察是否已有语义跳变。

训练历史进阶练习

5.4 并行 TF 的泄漏

题目 训练第 3 块时，mask 禁止读 clean 第 3 块，但允许读取一个双向编码过的 clean 第 2 块。是否仍可能泄漏答案?
解题思路 可能。第 2 块的表示已从第 3 块吸收信息，形成间接路径。需要限制 clean 路内部可见性，逐层检查从未来到当前的路径。

5.5 噪声与学生错误

题目 在真历史上加高斯噪声，是否充分覆盖学生生成的历史误差?
解题思路 高斯噪声覆盖局部连续扰动，学生错误还包含错误身份、动作和场景的结构性偏移。比较 DF 与 Self Forcing 时，应控制噪声范围与 rollout 长度，并测长期漂移。

5.6 流水线的 stale KV

题目 多个 chunk 处于不同噪声级，直接复用上一轮所有 KV 会发生什么?
解题思路 仍在去噪的历史块改变了 latent 与时间条件，其 KV 已过期。应重算相应状态、按阶段保存 cache，或显式采用近似缓存并测误差；已固定的历史则可按兼容条件复用。

第六章　训练配方与训练加速

完整训练配方由数据、目标、参数更新和分布式执行共同构成。本章给出可执行的训练循环，计算显存与有效 batch，并区分单步吞吐、收敛步数和达到目标质量的总成本。

6.1　视频数据管线与训练课程

数据的镜头边界、描述质量、运动分布和分辨率课程，直接影响模型能力。

从原始视频到训练样本

一个可执行的管线可分为六步：

1. **来源与切镜头**：记录授权和来源，按镜头切分，减少无关场景突变。
2. **解码与采样**：确定时长、FPS、帧间隔、裁剪范围，保留这些元信息。
3. **过滤与去重**：检查清晰度、压缩损伤、字幕覆盖、运动幅度、重复片段与内容类别。
4. **描述**：标注主体、动作、空间关系、镜头运动和事件顺序，区分主体运动与相机运动。
5. **分桶与编码**：按宽高比、分辨率、帧数组织 batch；需要时缓存冻结 VAE / 文本编码器的输出。
6. **课程与采样比例**：逐步提高分辨率和时长，安排图像、普通视频与精选视频的比例。

技术报告中的数据配方

SVD 讨论图像预训练、视频预训练、精选视频微调三个阶段，并研究数据过滤与描述策略。**CogVideoX** 给出多分辨率训练和 progressive training，并讨论高质量视频描述。**HunyuanVideo / Wan** 的技术报告把数据处理、视频 VAE、DiT 和分阶段训练串成完整系统。[108, 110, 111, 112]

数据分布与能力侧重

令 b 为数据桶或来源，π_b 为采样比例。总体目标可写成

$$\mathcal{L} = \sum_b \pi_b \, \mathbb{E}_{(z_0,c)\sim\mathcal{D}_b,\, t,\epsilon} \left[w_b(t) \left\| v_\theta(z_t, t, c) - (\epsilon - z_0) \right\|^2 \right].$$

改变 π_b 就改变主体、动作、时长与清晰度在梯度中的贡献。增加静态高质量图像可增强外观学习，同时需要视频数据继续提供运动信号。短片段课程降低早期成本，长片段阶段补长距离依赖。

数据消融的控制变量

记录有效视频小时数、去重方式、caption 版本、长宽比与帧数分布、图像视频比例、每阶段 token 数和 GPU-hours。训练 loss 下降与生成质量变化一起观察；固定评价 prompt 和采样配置，再归因于数据、网络或目标的改动。

本页六步管线与混合目标是依据技术报告整理的实施框架；各报告使用的阈值、数据来源和课程顺序需分别复现。

6.2 基础训练循环与可变长度样本的梯度

预训练先固定压缩器和文本条件接口，再反复优化生成主干。下面给出一个连续 latent FM 基线；扩散噪声回归只需替换路径与目标。

1. 从数据管线取 (X, c)，按分辨率、帧数、宽高比分桶；生成有效 token mask m。
2. 用冻结 VAE 得到 z_0，按模型约定做 latent scaling / shift；编码文本，按概率丢弃条件以训练 CFG 分支。
3. 采样 t 与 ϵ，构造 $z_t = (1-t)z_0 + t\epsilon$，目标 $u = \epsilon - z_0$。
4. 计算 $v_\theta(z_t, t, c)$，将 padding 位置排除，按时间权重求损失。
5. 反向传播、梯度归约、必要的裁剪、优化器更新；再更新 EMA。

$$L = \frac{\sum_{b,i} m_{bi}\, w(t_b) \|v_{bi} - u_{bi}\|^2}{\sum_{b,i} m_{bi}}.$$

这里表示按有效 token 平均的目标。若希望每段视频具有相同权重，应先按每段视频归一化，再对视频求平均。两种目标对长短视频的权重不同，需要在配置中明确。

有效 batch 与梯度累积

单次 microbatch 为 B_μ、数据并行卡数为 D、累积次数为 A，则等长样本条件下 $B_{\text{eff}} = B_\mu DA$。序列并行中的多张卡协同处理同一个样本，不计入独立样本数。可变长 microbatch 的损失应按整个累积窗口的总权重归一化，否则短 microbatch 可能获得过高权重。

数据缓存与在线增强

预计算 VAE latent 和文本特征可省去冻结模型前向；它同时固定了编码器版本、裁剪、caption 和部分增强。若训练需要随机时序裁剪、多尺度图像增强或不断修订 caption，应缓存中间表示或保留重算路径。计时同时记录数据读取等待、解码、条件编码和主干 forward/backward，避免 GPU 利用率高低被错误归因于模型。

6.3 各训练阶段的目标与监督接口

先定位训练阶段，再选择与监督接口相符的目标。

训练对象	常见主目标	需要的监督/注意点
VAE	重建 + 感知 + KL，可有 GAN	真视频；注意压缩率与时间边界
基础 DiT	噪声/x/velocity 回归	视频、条件、自己采样的噪声
配对 refiner	恢复/感知或条件 FM	对齐 LR/HR 或构造退化数据
少步学生	一致性/映射/分布蒸馏	依方法需要教师轨迹或 score
长视频学生	自 rollout + 局部监督	历史匹配、截断梯度、缓存策略
偏好模型	BT/logistic、回归或分布学习	人类/AI 偏好，明确标注协议
偏好生成器	DPO、可微 reward、DDPO/GRPO	偏好对或在线评分接口
GAN 判别器	hinge/logistic + 可选 R1/R2	真/假或教师/学生分布

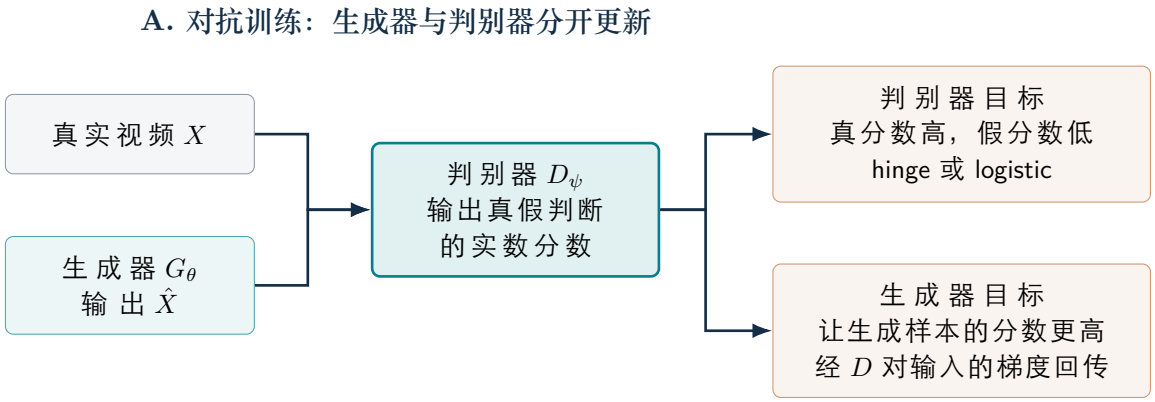


图 6.1　损失函数：谁打分，谁被更新？。hinge / logistic 管分类方向；R1 / R2 管判别器在输入附近有多陡。关键区别：冻结参数时，仍可保留对输入的梯度。R1/R2 的数字指两类取样位置，两式都可使用平方 L2 范数。选评分器时，同时核对它需要的参考图、提示词与时序输入。[12, 13, 14, 11, 46, 47]

6.4 REPA 与特征辅助训练

REPA 对齐中间特征；RAE 改变扩散模型所工作的表示空间。

REPA：借助视觉编码器提供中间监督

给干净样本 x_0 提取冻结视觉特征 $E(x_0)$，将带噪输入经过生成骨干得到的中间特征 h_i 投影后与之对齐。以余弦形式为例：[102]

$$\mathcal{L}_{\text{align}} = 1 - \frac{1}{N}\sum_i \frac{P(h_i)^\top E(x_0)_i}{\|P(h_i)\|\,\|E(x_0)_i\|}, \qquad \mathcal{L} = \mathcal{L}_{\text{gen}} + \lambda\mathcal{L}_{\text{align}}.$$

P 解决特征维度匹配；视觉教师冻结，生成骨干和投影层接收梯度。原有生成目标继续学习噪声或速度，特征对齐帮助它较早形成语义结构。位置网格存在差异时需先对齐分辨率。

训练步数与总训练成本

REPA 在 ImageNet 类条件生成、SiT-XL、无 CFG 的设置中报告：不到 40 万训练步达到原 SiT 700 万步的性能，约 17.5 倍的步数差。[102] 该比较对应**达到指定质量的训练步数**；每步多了教师特征计算，实际 GPU-hours 还需计入这项开销。迁移到视频时，还要评估特征教师的时序能力和显存成本。

6.5 LoRA、EMA 与目标分支的更新规则

对线性层 $W_0 \in \mathbb{R}^{d_o\times d_i}$，LoRA 写作

$$W = W_0 + \frac{\alpha}{r}BA, \quad B \in \mathbb{R}^{d_o\times r},\ A \in \mathbb{R}^{r\times d_i}.$$

W_0 冻结，优化器只更新 A, B；新增参数量为 $r(d_i + d_o)$。例如 $d_i = d_o = 4096, r = 16$，LoRA 有 131072 个参数，约为完整矩阵的 0.78%。实际还会选择注入哪些层、是否训练 bias / Norm 等。

EMA 在完成梯度更新后执行

$$\bar{\theta}_n = \mu\bar{\theta}_{n-1} + (1-\mu)\theta_n.$$

$\mu = 0.9999$ 时，典型平均窗口约为 $1/(1-\mu) = 10000$ 次更新，权重半衰期约为 $\log(0.5)/\log(\mu) \approx 6931$ 次。这个数量按优化器更新计数；改变更新频率或 batch 后，所覆盖的数据量也改变。

教师较稳定的来源

EMA 平滑学生参数的短期波动；在一致性训练中，目标分支通常停止梯度。教师分支的监督优势也可来自更多上下文、更细采样、强条件或预训练能力。单独做 EMA 只提供平滑，教师获得额外信息时才有对应的信息优势。

LoRA 因子平均与完整权重平均

若逐元素平均 A 与 B，得到的乘积 $\bar{B}\bar{A}$ 通常不同于平均更新矩阵 $\overline{BA}$。因此“EMA LoRA”应明确保存和平均哪些张量。算法复现时跟随原实现的参数化和更新顺序；把适配器合并进底座后再做 EMA，会得到另一条轨迹。

6.6 训练显存：参数状态、激活与通信缓冲

训练峰值可拆为

$$M_{\text{peak}} = M_{\text{state}} + M_{\text{activation}} + M_{\text{workspace}} + M_{\text{communication}} + M_{\text{other}}.$$

设可训练参数数目为 P，以一种常见混合精度 Adam 实现为例：BF16 权重 $2P$ 字节、BF16 梯度 $2P$、FP32 master weights $4P$、两个 FP32 moment $8P$，合计 $16P$ 字节。若保留 FP32 梯度，则该配置变为 $18P$。实际框架可能省去 master copy 或采用其他状态格式，按真实 dtype 核算。

ZeRO 与 FSDP 的状态切分

在上述 $16P$ 假设下，D 张数据并行卡的理想每卡状态预算为：[132]

$$\begin{aligned} &\text{复制全部状态：} && 16P, \\ &\text{ZeRO-1，切优化器状态：} && 4P + 12P/D, \\ &\text{ZeRO-2，再切梯度：} && 2P + 14P/D, \\ &\text{ZeRO-3，再切权重：} && 16P/D. \end{aligned}$$

式子只统计常驻模型状态。全分片训练在执行某层前通常需 all-gather 权重，之后做梯度 reduce-scatter；预取、临时完整权重和通信 bucket 会抬高峰值。FSDP 的实际开销取决于 wrap 粒度、重分片、预取和 offload 策略。

Activation checkpointing

设网络有 L 个块，每块需保存 a 字节激活。逐层保存约为 La。简化为每 s 块保存一个边界，并在反向时重算该段，激活规模可粗估为

$$M_{\text{act}} \approx a(L/s + s).$$

$s \approx \sqrt{L}$ 给出此简化模型下的折中。真实模型的块大小并不均匀，注意力和 MLP 的重算成本也不同。checkpointing 通过增加前向重算换显存；是否缩短总训练时间，要看它能否支持更好的 batch、减少 offload 或避免更慢的并行方案。

LoRA 与蒸馏辅助网络

LoRA 冻结大部分权重，可显著减少优化器状态和梯度；反向仍需穿过冻结层，因此激活保存成本继续存在。DMD 类训练还要容纳 teacher、student、fake-score，且不同阶段的可训练网络不同。把一个 14B 权重文件大小直接当训练显存，会遗漏优化器、激活和多个模型副本。

Attention 之外的激活峰值

设 token 数 N、隐藏维 d、MLP 扩展维 d_{ff}。全 attention 的理论算量通常随 $N^2 d$ 增长，MLP 随 $N d\, d_{\text{ff}}$ 增长。FlashAttention 避免显式存储完整 $N \times N$ 注意力矩阵，**精确全 attention 的配对计算仍随 N^2 增长。**

一个完全示意的显存估算

某张卡上的 MLP 中间激活为 $[N_{\text{local}}, d_{\text{ff}}]$，则：

$$M \approx N_{\text{local}}\, d_{\text{ff}}\, b \quad \text{bytes.}$$

若 $N_{\text{local}} = 200000, d_{\text{ff}} = 60000$，FP32 的 b=4，则单个张量约 48 GB（十进制）。即使 attention 不存大矩阵，这个激活仍能触发 OOM；还没算 gate、残差、通信 buffer、权重和优化器。

序列切分与本地激活

将 token 序列拆到不同卡，可减少本地 MLP/Norm 激活；Ulysses 等机制通过序列/头之间的重新分布计算 attention。检查 all-to-all 前后的实际 shape，才能算清每张卡的峰值。

流式的另一种内存压力：KV

粗略地，KV 大小随层数、保留 token 数、KV heads、head dim 和每元素字节数增长。短窗口、frame sink、KV 量化或淘汰策略能控制它，但牺牲的历史信息与引入的近似误差要单独评估。

实用排查清单

- 峰值发生在 prefill、MLP up-projection、attention、VAE 还是通信？
- 某些算子是否悄悄转成 FP32？当前 dtype 与 autocast 范围是什么？
- reserved 与 allocated 是不同统计；碎片和临时副本都要检查。
- 推理是否误保留计算图？训练 checkpoint 是否覆盖大激活？
- 分块/tiling 是否真的降低峰值，还是先构建整块再切？

上述数值由教学张量维度推算，实际峰值可通过 profiler 验证。低比特与 attention 优化的实例见 TurboDiffusion[50]。

6.7 训练吞吐、收敛速度与达到目标质量的成本

训练加速有两条独立路径：减少一次有效更新的时间，以及减少达到目标质量需要的更新次数。以目标质量 Q 定义

$$T(Q) = N_{\text{update}}(Q)\,\bar{T}_{\text{update}}, \qquad \text{GPU-hours} = D\,T(Q)/3600.$$

改大 batch 后，单步 token 数和更新次数同时变化，比较 tokens/s、有效样本数、优化器更新数与 $T(Q)$ 才能判断收益。

手段	节省位置	需要同时测量
长度/分辨率分桶	padding、编译重建、负载不均	数据混合比例与每桶质量
BF16 / FP8	GEMM、激活与通信字节	溢出、loss 曲线、最终质量
FlashAttention	attention 中间存储与访存	backward、dropout、mask 支持
checkpointing	保存的激活	重算耗时与最大可用 batch
ZeRO/FSDP	常驻参数状态	all-gather / reduce-scatter 开销
CP/SP/TP	单卡序列与层内计算	通信拓扑、重叠与局部 shape
REPA、课程与权重设计	达到目标质量的更新数	教师开销与总训练预算

计算与通信的下界估计

一次计算的粗略耗时可用 roofline 模型定位瓶颈：

$$T_{\text{kernel}} \gtrsim \max\left(\frac{F}{P_{\text{compute}}}, \frac{B}{B_{\text{memory}}}\right), \quad T_{\text{step}} \approx T_{\text{compute}} + T_{\text{exposed comm}} + T_{\text{input}}.$$

F 为浮点操作量，B 为访存字节，P_{compute} 与 B_{memory} 分别为实际可达算力和带宽；暴露通信指未被计算覆盖的部分。小矩阵、转置、mask、形状变化和启动开销会使实际值远离下界。

低精度训练与推理量化

推理量化只需验证固定权重下的生成误差；训练低精度还影响梯度、优化器统计和数千次更新后的累计偏差。缩放因子、累加精度和 master states 应分别记录。SageAttention3 的 8-bit 训练探索显示，微调与从头预训练的收敛表现有差别，因此 attention kernel 更快仍需配合质量匹配的训练时间比较。[128]

6.8 近期技术报告在体系中的位置

截至 2026 年 10 月 17 日，本节选择能补充架构、训练与部署机制的公开报告。下述数字均为论文或官方文档报告值。

HunyuanVideo 1.5：压缩、条件、稀疏与超分协同

报告采用 8.3B 的基础 DiT、54 个双流块，并配合视频超分阶段；其 VAE 空间压缩 16×、时间压缩 4×、latent 通道 32。I2V 同时接入 VAE 图像 latent 与 SigLip 语义特征，分别提供重建细节与语义条件。SSTA 将局部滑窗和动态块选择合并，在蒸馏阶段引入稀疏训练。该案例体现表示、条件接口和部署算子共同设计。[124]

Helios：在历史、token 与步数三个维度压缩

Helios 是 14B 自回归视频生成系统：已生成历史作为条件，当前生成窗口保持双向处理。它通过相对位置、首帧锚点和训练时的历史损坏模拟处理漂移；历史采用多尺度 patchification，当前 noisy context 使用金字塔式处理，再通过层级蒸馏降低采样步数。报告给出 50 到 3 步的配置与单 H100、384×640 分辨率下最高 19.5 FPS 的端到端结果（包含 VAE 与文本编码）。[126]

该系统说明，序列级 AR 分解可以由反复调用双向窗口模型实现。复现其速度时，应固定报告采用的分辨率、窗口、精度和端到端边界；其训练基础设施方案与普通全量 Adam 状态预算分别核算。

论文到代码的对照模板

接口	需要查到的具体对象
表示	VAE checkpoint、stride、latent scaling、首帧特殊规则
骨干	block 类型、位置编码、attention mask、条件投影
目标	输出参数化、时间采样、loss 权重、训练阶段
流式	历史压缩、cache 生命周期、位置偏移、解码边界
效率	NFE、CFG 分支、kernel、精度、编译与并行配置
发布资产	基模/蒸馏权重、LoRA 依赖、许可证、训练脚本

6.9 产业训练配方：数据、标注与系统协同

Seedance：预训练、持续训练与精细 SFT

Seedance 1.0 以多源数据清洗与精细 caption 支撑预训练，再用持续训练增强运动和语义；SFT 使用按风格、运动等维度组织的精选视频与核验 caption。后续偏好训练使用专门的数据反馈管线。1.5 pro 加入音视频一致性、声音描述与课程采样，把同步质量纳入数据选择。[167, 168]

Vidu S1：全 clip 与语音感知 chunk 两级标注

全片 caption 提供稳定语义，chunk caption 提供局部动作与交互条件。其标注分别从视频判断视觉属性、从音轨判断声音属性，减少跨模态误判，然后形成面向流式训练的时间对齐描述。TF/DF 因果适配需要这些按时间可获得的条件；若把未来语音描述提前送入当前 chunk，会改变任务信息边界。[52]

MiniMax H3：从描述目标到描述上下文关系

Contextual Omni Representation 同时描述目标内容、参考与目标的关系，以及多份参考之间的关系。训练系统分别处理理解与生成的异构工作量，再平衡样本间负载；官方博客报告其优化带来接近 30% 的端到端训练吞吐提升。该数字是其内部系统对比，迁移到其他模型需重新测序列分布、数据管线和通信。[164]

MagiMoE：长序列下的专家负载

MAGI-2 Preview 把 head 表示交换与专家状态切分映射到不同通信层级，配合小专家 kernel、路由稳定性和内存管理。衡量一个 MoE 批次时，同时记录每 rank token/专家负载、all-to-all 时间和峰值显存；只看平均 active parameters 会漏掉拖慢整个 step 的最忙 rank。[162]

本章思考与进阶练习

6.1 显存预算

题目 $P = 10^9$、8 卡、采用本章 $16P$ 字节 Adam 假设。ZeRO-2 的理想状态显存是多少?
解题思路 每卡 $2P + 14P/8 = 3.75 \times 10^9$ 字节，约 3.75 GB。激活、临时权重、通信 buffer 和框架开销尚需另加，GB 与 GiB 也应统一。

6.2 训练加速口径

题目 方法 A 每步慢 20%，达到目标质量的更新数减少一半。相同卡数下总训练加速多少?
解题思路 时间比例为 $1.2 \times 0.5 = 0.6$，加速为 $1/0.6 \approx 1.67$ 倍。若额外教师预计算或数据准备显著，应把这部分计入端到端训练预算。

6.3 进阶：EMA 与 LoRA

题目 为什么分别平均 LoRA 的 A、B 因子会改变有效更新?
解题思路 乘积是双线性的：$\mathbb{E}[B]\mathbb{E}[A]$ 与 $\mathbb{E}[BA]$ 相差因子之间的协方差项；因子的尺度重参数化也会影响平均轨迹。复现应明确平均的是因素参数还是完整更新矩阵。

第七章　少步生成与一致性学习

减少网络调用，可以学习更大跨度的映射、分段终点或平均速度。本章从已有教师的映射压缩进入一致性家族，再讨论支持直接训练的 Shortcut、MeanFlow 和 Flow Map。

7.1　Progressive Distillation、CM 与 LCM

一致性是约束；CM 是模型家族；LCM 把它用于 latent；progressive 是逐轮减半的蒸馏过程。

CM：同一条 ODE 轨迹，不同出发点，同一终点

对教师生成轨迹上的 z_t, z_s：

$$f_\theta(z_t, t, c) \approx f_\theta(z_s, s, c) \approx z_0, \qquad f_\theta(z_0, 0, c) = z_0.$$

“同一轨迹”很重要；独立加噪出的两个样本不一定满足教师 ODE 对应关系。**trajectory consistency 约束去噪轨迹；temporal consistency 衡量播放时的时序连续。**[15]

Consistency distillation 与 consistency training

CD 使用预训练 diffusion 教师构造邻近轨迹点；CT 则研究不依赖预训练教师的直接训练。因此 consistency 家族同时覆盖蒸馏与直接训练。一致性蒸馏常比较在线学生和 stop-gradient 目标分支：

$$L_{\mathrm{CD}} = \mathbb{E} d\big(f_\theta(z_t, t), \mathrm{sg}[f_{\bar{\theta}}(\mathrm{Solver}_T(z_t, t \to s), s)]\big).$$

这是概念形式，具体预条件、距离、时间采样和 EMA 规则随方法不同。

LCM：Latent Consistency Models

在压缩 latent 空间学习一致性映射，并针对文本条件与 guidance 进行适配。原 LCM 给出对应的训练目标、指导方式与采样设计。推理多步时常交替预测干净端点和重新加噪细化，因此多步行为与普通 ODE 小步积分不同。[16]

Progressive distillation：教师两步，学生一步

先让教师从 z_t 走两步到 z_r，训练学生一步抵达同一目标。再把学生升级为下一轮教师：

$$64 \to 32 \to 16 \to 8 \to 4 \to 2 \quad \text{（示意步数）}.$$

它蒸馏更大步长的采样映射，学生的网络层数可以保持原样。多轮训练和误差传递有成本，也通常与特定 sampler/时间网格耦合。[17]

CM：不同起点要答同一终点。　progressive：以一个学生步拟合两个教师步。

7.2　CTM 的两时刻一致性映射

CM 回答“这条轨迹的终点在哪”，CTM 进一步回答“沿轨迹走到指定时间会在哪”。

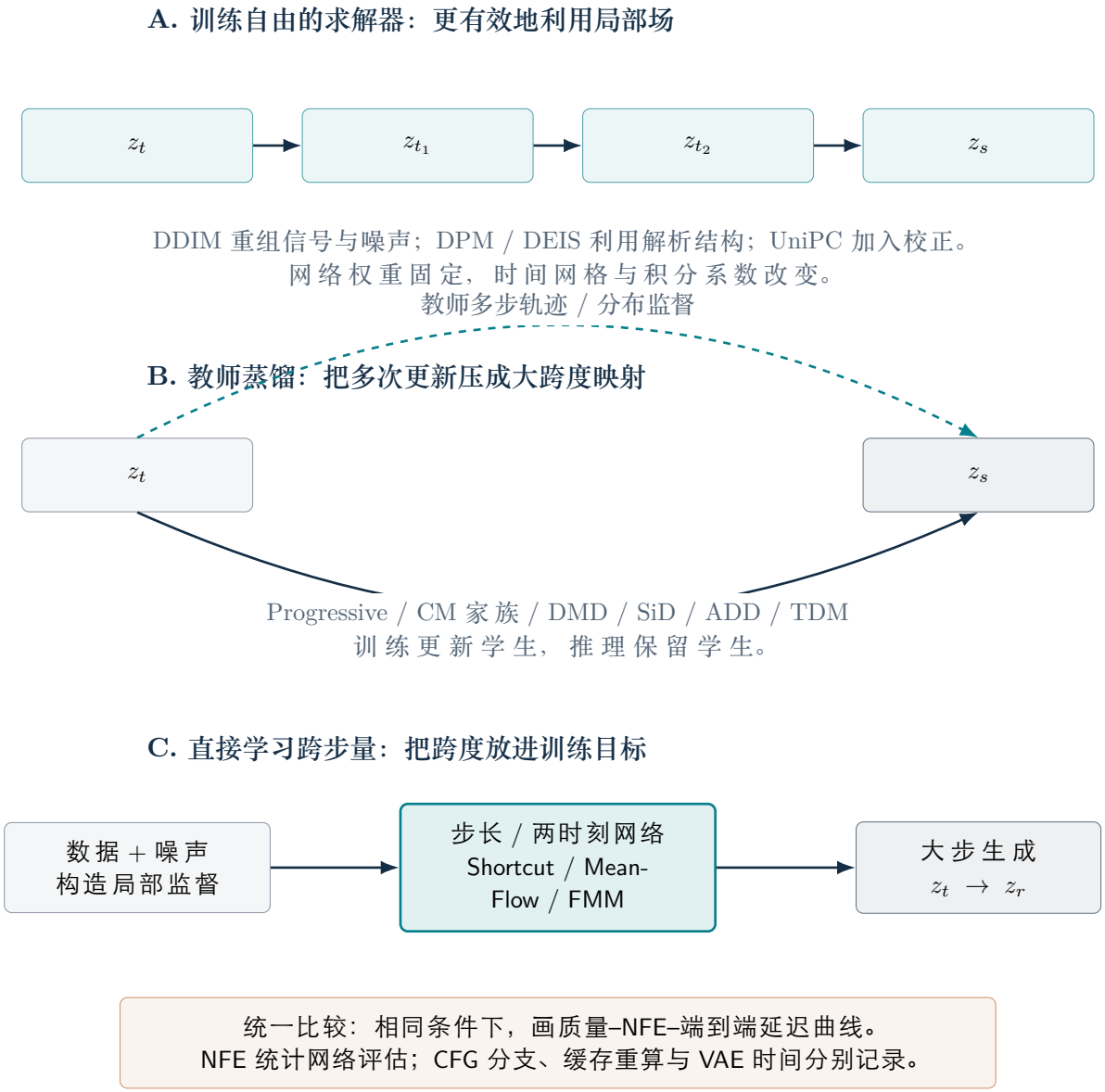


图 7.1 **三种少步思路。**求解器优化积分，蒸馏学习教师，直接训练把跨步关系纳入目标。**监督来源：**B 使用教师提供的轨迹、score 或样本；C 可由数据路径与自举关系学习。部分一致性与流映射方法同时支持两类训练。[15, 69, 79, 81]

令 $F_\theta(z_t, t, s)$ 直接预测时间 s 的状态，$0 \le s \le t \le 1$。当 $s = t$ 时满足边界条件

$$F_\theta(z_t, t, t) = z_t.$$

普通终点一致性映射可看作固定 $s = 0$ 的情形。输入目标时间 s 后，一个网络能表示许多不同跨度。

一致性目标的构造

取 $s < u < t$。教师先把 z_t 推进到 z_u^T。学生从 t 出发、目标分支从 u 出发，都预测到同一个 s：

$$z_u^T = \mathrm{Solver}_T(z_t, t \to u),$$

$$L_{\text{traj}} = \mathbb{E}\, d\big(F_\theta(z_t, t, s), \mathrm{sg}[F_{\bar\theta}(z_u^T, u, s)]\big)\,.$$

这是两时刻一致性的核心关系。CTM 的训练进一步把中间预测送到公共比较端点，并组合去噪与对抗监督，使不同跨度的映射兼顾轨迹和样本质量。[73]

任意目标时刻的映射能力

- 可直接从高噪声跳到较低噪声，继续使用少数大步。
- 小跨度下可提取局部运动信息；大跨度下能快速输运。
- 采样策略可以在计算预算、确定性和随机性之间选择。

原 CTM 在单步生成实验中报告 CIFAR-10 FID 1.73、ImageNet-64 FID 1.92。阅读收益时，连同教师、对抗项、评估分辨率和采样模式一起看。

与分段一致性和分布匹配的联系

方法	时间与监督的组织方式
CM	轨迹上的各点映到固定干净端点
PCM	各 phase 内映到该段低噪端点
CTM	同一网络表示可变的源时间与目标时间
TDM	在轨迹分布上定义匹配目标，适配不同步数

CTM 的新能力是“指定目的时间”；训练是否保留路径细节、是否加入 GAN，则由具体目标组合决定。

7.3 PCM 的分段一致性

沿去噪时间分段，每个 phase 对应一段噪声区间。

假设把噪声到干净划为四个 phase：$1 \to 0.75 \to 0.5 \to 0.25 \to 0$。对 $t, s \in [0.75, 1]$，只要求映射到 $z_{0.75}$，不必直接预测 z_0。令 r 为当前 phase 的低噪端点，$s < t$，教师生成邻近状态 $z_s^T = \mathrm{Solver}_T(z_t, t \to s)$。则：

$$L_{\text{PCM}} = \mathbb{E} w(t) \left\| f_\theta(z_t, t; r, c) - \mathrm{sg}[f_{\bar\theta}(z_s^T, s; r, c)] \right\|_2^2.$$

r 在式中用于强调目标端点，在实现中 phase 可由 t 所在区间确定。[18]

三个训练角色

冻结 diffusion 教师提供局部正确轨迹；在线学生学跨步；EMA 学生提供缓慢变化的目标。$\bar\theta \leftarrow \mu\bar\theta + (1-\mu)\theta$ 通过滑动平均更新目标分支，梯度优化在线学生。

Phase 边界与多步组合

phase 内的边界约束是 $f_\theta(z_r, r; r) = z_r$。相邻点的一致性把正确的终点信息传播到区间其他位置。不同 phase 的终点串联，就形成确定性的少步路径。通常由同一个网络共享各 phase 的参数。一种与 DDIM 形式相同的参数化为：

$$f_\theta(z_t, t; r) = \alpha_r \frac{z_t - \sigma_t \epsilon_\theta(z_t, t)}{\alpha_t} + \sigma_r \epsilon_\theta(z_t, t).$$

当 $r = t$ 时输出 z 本身。**更新形式与 DDIM 相容**，蒸馏网络学习适合跨区间映射的预测量。真实网络仍有逼近误差。

相对 LCM 的增量

PCM 研究确定性多步路径、CFG/负面提示的可控性，以及针对少步细节的对抗一致性监督。官方展示多种步数配置与视频扩展；对抗项的效果分别用细节、多样性和分布指标评估。步数、phase、scheduler 和 checkpoint 必须匹配。[19]

PCM 主要压缩去噪步数；长视频的历史漂移由 rollout 训练、缓存策略和时序条件继续处理。

7.4 sCM 的连续时间一致性

当两个轨迹点越来越近，一致性就变成沿轨迹的导数约束。

设终点预测为 $f_\theta(z_t, t)$。理想情况下它沿同一 ODE 轨迹保持不变：

$$\frac{d}{dt} f_\theta(z_t, t) = \partial_t f_\theta + J_z f_\theta \, v_T(z_t, t) = 0.$$

把 z 看成一维数，$J_z f$ 就是导数 $\partial f / \partial z$；视频里它对应巨大的 Jacobian。JVP 直接算“Jacobian 乘一个方向”，无需把整张矩阵存下来。

离散时间差与数值稳定性

若以 $[f(z_t, t) - f(z_s, s)]/(t - s)$ 估计导数，间隔很小时，两个相近浮点数相减会放大误差。网络切向量的大小、时间采样与预条件还会影响梯度尺度。sCM 围绕连续时间训练重新设计稳定化处理，包括参数化、切向量归一化与自适应权重等，使训练能扩展到更大的模型和高分辨率数据。停止梯度用于控制目标分支的优化路径。[74]

一个容易读懂的训练示意

1. 从数据加噪得到 (z_t, t)，教师提供局部轨迹方向。
2. 用自动微分求网络沿该方向的切向变化。
3. 对残差与切向量做相应归一化/加权，形成稳定的学习信号。
4. 更新在线参数，并按该配置更新目标分支。

这里的导数等式解释数学约束；论文训练算法还指定了输出预条件、权重、stop-gradient 和时间采样。

相对早期 CM 的收益

原 sCM 将连续时间一致性扩展到约 1.5B 参数，在 ImageNet 512^2 上报告 2 NFE、FID 1.88，并给出约 50 倍的扩散采样加速。它强调大模型训练稳定性以及少步质量。[74]

接到 rCM

本书后面的 rCM 在这类连续一致性目标上加入 score-based 分布匹配。这样，sCM 的局部轨迹约束和 DMD 的学生分布反馈可以共同作用；新增训练开销主要来自 fake-score 分支及相应生成样本。

7.5 rCM 的 score 规则化

本文 rCM 指 score-regularized continuous-time consistency model。

先理解 continuous-time consistency

若 f 沿同一 ODE 轨迹始终指向同一终点，则沿轨迹的全导数应为零：

$$\frac{d}{dt}f(z_t,t)=\underbrace{\partial_t f(z_t,t)}_{\text{时间自己变}}+\underbrace{J_z f(z_t,t)\,v_T(z_t,t)}_{\text{输入沿教师轨迹移动}}=0.$$

这条等式给出**连续一致性的理想条件**。sCM 通过特定参数化、预条件、归一化和停止梯度稳定训练。JVP 是 Jacobian-vector product，计算 $J_z f\,v$，不必显式构建巨大的 Jacobian。

rCM 要修补的短板

论文分析大规模 sCM 的细节质量与误差积累问题，使用 score-based 分布匹配做长跨度规则化：

$$\boxed{L_{\text{rCM}}=L_{\text{sCM}}+\lambda L_{\text{DMD}}.}$$

一致性分支负责保留轨迹约束和覆盖性；DMD 分支在学生生成分布上提供分布反馈。作者用 forward/reverse divergence 的视角解释两者互补，具体覆盖性和集中程度还受模型容量与优化过程影响。[20]

辅助 score 的职责

rCM 的 DMD 项需要比较当前学生分布与目标分布，因此仍会训练 fake-score 分支。这一分支使训练同时利用轨迹一致性和当前学生分布的反馈。

工程贡献也很关键

视频 latent 维度大、模型可达百亿参数。论文实现适配并行训练的 FlashAttention JVP，并处理时间导数的数值稳定性；时间 embedding 的精度会影响长期训练是否突然崩溃。[21]

收益的实验条件

作者在 Cosmos-Predict2、Wan2.1 等设置下报告 1–4 步和约 15–50 倍 diffusion sampling 加速。该数字统计**对应模型与采样条件下的去噪阶段**；端到端还包含 VAE 等开销。TurboDiffusion 的组合加速另包含量化、稀疏等组件。

CM：终点一致。　PCM：分段终点一致。　rCM：连续时间一致性 + 分布匹配。

分布匹配项的梯度与辅助 score 网络见第八章。

7.6 Shortcut Models 的步长条件与自举

短步用数据路径教，大步用两个半步的组合来教。

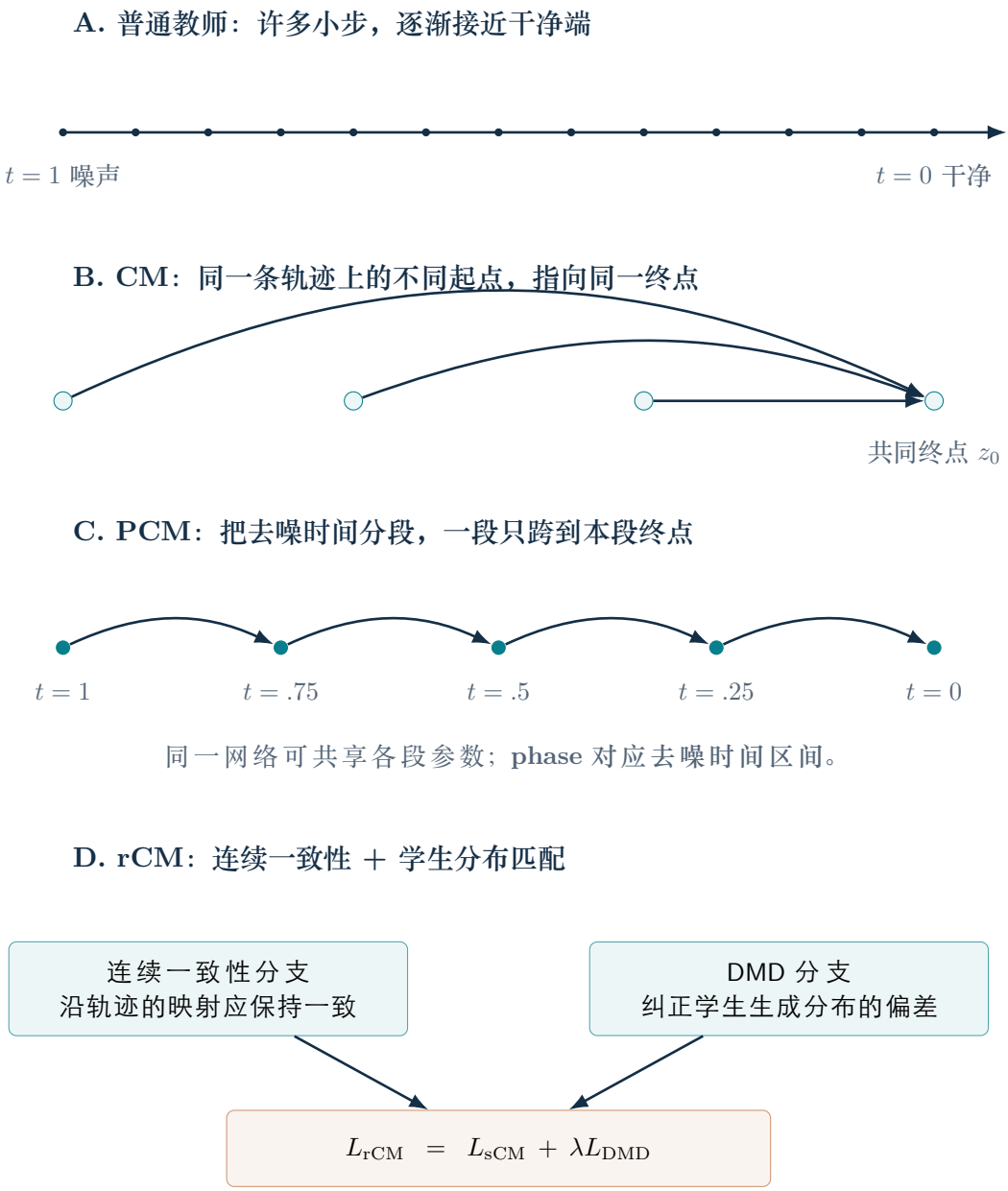


图 7.2 CM、PCM、rCM：压缩的是怎样的路径?。CM 学全程终点，PCM 学分段终点；rCM 把连续一致性与分布匹配结合。坐标：横轴为去噪时间，点位和步数用于说明映射关系。rCM 的两分支图是目标层面的分解，省略 JVP、预条件、归一化与停止梯度等实现细节。[15, 18, 20]

令 $d \geq 0$ 表示向低噪端走的时间跨度，学生输出区间方向 $s_\theta(z,t,d)$，状态更新为

$$S_\theta(z,t,d) = z - d\, s_\theta(z,t,d), \qquad d \leq t.$$

网络同时看到当前位置、当前时间和目标步长。训练随机覆盖不同 d，因此同一个模型可以采用不同步数。

小步分支：Flow Matching 提供锚点

取 $z_t = (1-t)z_0 + t\epsilon$，在 $d=0$ 的极限用

$$L_{\text{base}} = \mathbb{E}\,\|s_\theta(z_t,t,0) - (\epsilon - z_0)\|^2.$$

这让模型学到正确的瞬时方向，监督来自真实数据和自行采样的噪声。

大步分支：一步应等于两个半步

用目标网络先走两次 d：

$$a = s_{\bar\theta}(z,t,d), \quad z' = z - da, \quad b = s_{\bar\theta}(z',t-d,d).$$

两步总位移为 $d(a+b)$，所以跨度 $2d$ 的平均方向应为

$$\boxed{s_{\text{target}} = \tfrac{1}{2}(a+b), \qquad L_{\text{boot}} = \|s_\theta(z,t,2d) - \text{sg}[s_{\text{target}}]\|^2.}$$

这是一种自举：用模型已经能做的小步构造大步目标。FM 锚点与大步一致性共同训练，支持从头学习，无需先训练一个独立扩散教师。[79]

推理与收益

从 $t=1$ 开始，一步模式取 $d=1$；四步模式可依次取 $d=0.25$。训练覆盖的步长范围影响各档质量。收益是一个模型覆盖多种预算；额外训练成本来自生成自举目标和管理不同步长样本。

Progressive distillation 常把“当前学生升级成下一轮教师”，逐阶段压缩步数；Shortcut 把步长变成条件，在同一次训练中学习局部方向与跨步组合。

7.7 MeanFlow 的平均速度与训练恒等式

一步跨越整个区间，需要这个区间的平均速度。

设轨迹满足 $dz_t/dt = v(z_t,t)$。从 t 走到 r，$r<t$，定义

$$\boxed{u(z_t,r,t) = \frac{z_t - z_r}{t-r} = \frac{1}{t-r}\int_r^t v(z_\tau,\tau)\,d\tau.}$$

v 是某一时刻的切线方向，u 是整个区间的平均方向。知道 u 后可直接更新

$$z_r = z_t - (t-r)u(z_t,r,t).$$

平均速度的数值示例

取教学轨迹 $z(t)=t^2$，$v(t)=2t$。从 $t=1$ 到 $r=0$：

- 出发点的瞬时速度为 2。大步 Euler 给出 $1-1\times 2=-1$。
- 区间平均速度为 $(1-0)/(1-0)=1$。平均速度更新给出 $1-1\times 1=0$。

轨迹有弯曲时，出发点速度与区间平均速度就会不同。MeanFlow 直接学习后者。

平均速度恒等式的推导

对 $(t-r)u(z_t,r,t) = z_t - z_r$ 关于 t 求全导数，保持 r 固定：

$$u + (t-r)\frac{du}{dt} = v(z_t,t), \qquad \frac{du}{dt} = \partial_t u + J_z u\, v.$$

因此

$$\boxed{u = v - (t-r)(\partial_t u + J_z u\, v).}$$

右侧把可由 FM 学到的瞬时方向，与平均速度网络自身的方向导数联系起来。JVP 计算的是整条输入 (z,r,t) 沿 $(v,0,1)$ 的变化。

起点与目标时刻条件

同一个 z_t，走到 $r = 0.9$ 与走到 $r = 0$ 需要不同的平均方向。让网络看到 r、t，才能描述不同跨度。$r = t$ 时区间收缩，平均速度趋近瞬时速度。[80]

训练目标与 JVP

采样数据 z_0、噪声 ϵ 和 $0 \le r \le t \le 1$：

$$z_t = (1-t)z_0 + t\epsilon, \qquad v_{\text{pair}} = \epsilon - z_0.$$

网络预测 $u_\theta(z_t,r,t)$。计算方向导数

$$d_\theta = \text{JVP}_{(z,r,t)}\, u_\theta(z_t,r,t)\,[v_{\text{pair}},0,1].$$

构造目标并回归：

$$u_{\text{target}} = v_{\text{pair}} - (t-r)d_\theta,$$

$$\boxed{L_{\text{MF}} = \mathbb{E}\, w\, \|u_\theta(z_t,r,t) - \text{sg}[u_{\text{target}}]\|^2.}$$

这里 w 概括损失加权；实际 MeanFlow 配置还处理时间采样、归一化与 guidance。目标侧 stop-gradient 使优化沿在线预测分支进行。[80]

样本速度与边缘速度的条件期望

同一个 z_t 可由不同数据–噪声对得到。v_{pair} 是一对样本的标签；其条件均值 $\mathbb{E}[v_{\text{pair}} \mid z_t,t]$ 对应边缘速度。JVP 对方向线性，因而能用随机配对方向构造估计。网络最终需要跨大量样本学习共享的平均流结构。

平均速度模型的采样

一步时采样 $z_1 \sim \mathcal{N}(0,I)$，计算

$$z_0 = z_1 - u_\theta(z_1,0,1).$$

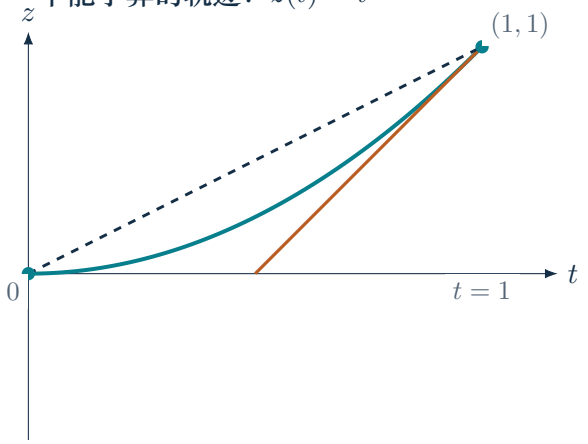
推理只需要网络前向，JVP 留在训练阶段。多步时可选择一组递减时间，逐区间调用平均速度网络。

相对 baseline 的收益

原 MeanFlow 在 ImageNet 256^2 从头训练实验中报告 1 NFE / FID 3.43。主要贡献是用平均流目标直接训练一步生成，而训练期需要额外的方向导数计算。评估视频应用时，应同样报告帧数、时序指标、VAE 与每次网络评估耗时。

FM 教“此刻怎么动”；MeanFlow 教“走完这一段，平均怎么动”。两者通过平均速度恒等式连接。

A. 一个能手算的轨迹：$z(t) = t^2$



青色曲线：真实轨迹
橙色切线：瞬时速度
虚线弦：区间平均速度

$v(1) = 2$
$u(1, 0, 1) = 1$

B. 从 $t = 1$ 一步走到 $r = 0$

用出发点瞬时速度
$z_0 \approx 1 - 1 \times 2 = -1$

用整个区间平均速度
$z_0 = 1 - 1 \times 1 = 0$

C. MeanFlow 恒等式连接两种速度

$$u = (z_t - z_r)/(t - r)$$
$$u = v - (t - r)(\partial_t u + J_z u\, v)$$

训练：数据路径速度 + JVP 构造目标
推理：$z_r = z_t - (t - r)u_\theta(z_t, r, t)$

图中曲线是教学解析例子；生成模型学习的是高维数据分布中的平均流。

图 7.3 MeanFlow 的瞬时与平均速度。弯曲轨迹上，一整段的平均速度可以直接给出终点位移。**数值例子：**曲线 $z = t^2$ 从 1 走到 0；橙线是 $t = 1$ 的切线，虚线连接区间端点。例子用于解释速度概念。[80]

7.8 Flow Map 与轨迹映射

直接学习两时刻之间的输运映射，可以把积分过程压缩成一次网络调用。

把时间 t 的状态送到时间 s，记作

$$\Phi_{t\to s}(z_t) = z_s, \qquad \Phi_{t\to t}(z) = z.$$

真实流映射满足组合关系：

$$\boxed{\Phi_{t\to r} = \Phi_{s\to r} \circ \Phi_{t\to s}.}$$

意思是：先到中间时间，再到目的时间，与直接到达相同。Shortcut 的两个半步目标、CTM 的跨时间一致性，都可以用这个语言理解。

它与局部速度场的数学联系

若 v 是对应 ODE 的速度场，则

$$\partial_s \Phi_{t\to s}(z) = v(\Phi_{t\to s}(z), s),$$

$$\partial_t \Phi_{t\to s}(z) + J_z \Phi_{t\to s}(z)\, v(z,t) = 0.$$

第一条说目的时间移动时，输出沿速度场变化；第二条说起点沿轨迹移动时，抵达同一目的时刻的结果保持一致。

流映射的监督来源

Flow Map Matching 框架既研究从预训练速度场蒸馏，也研究基于 stochastic interpolants / 前向扩散直接训练。因此“流映射”描述学习对象，“从头训练或蒸馏”描述监督来源。[81]

学习对象	网络回答的问题	常见约束
局部速度 v	此刻往哪里动?	FM / score 回归
平均速度 u	这一段平均怎么动?	MeanFlow 恒等式
流映射 Φ	指定时间的状态在哪?	两时刻导数 / 组合关系
轨迹模型	多个时刻的状态怎样对应?	轨迹一致性 / 分布匹配

收益与代价

推理可跨越较长时间区间，少数调用即可完成生成。训练则需保证不同跨度和组合路径的相容性。FMM 原论文在 CIFAR-10 / ImageNet-32 上报告约 10–20 倍生成时间改善；具体数字对应其模型与采样配置。

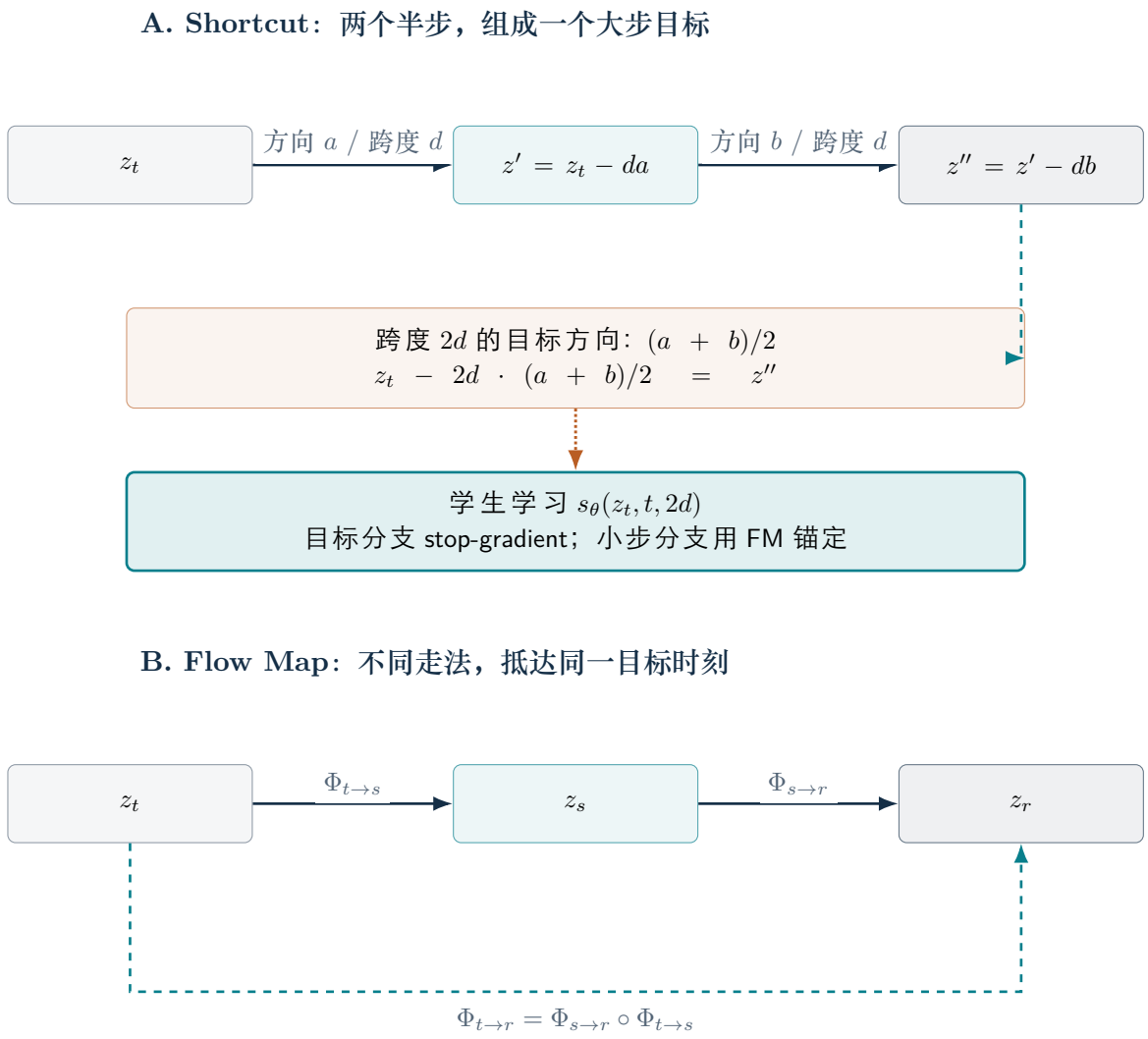


图 7.4 Shortcut 与流映射组合。两次半步的总位移，为一次大步提供自举目标。符号：d 为向低噪端推进的正步长，a, b 为两次方向预测；下图用 Φ 表示跨时间状态映射。[79, 81]

本章思考与进阶练习

7.1 平均速度

题目 对轨迹 $z = t^3$，从 $t = 1$ 到 $r = 0.5$，瞬时速度与平均速度分别多少?
解题思路 出发点瞬时速度为 3；平均速度为 $(1 - 0.125)/0.5 = 1.75$。平均速度一步得到 0.125；用起点速度做一次大步 Euler 得到 -0.5。

7.2 一致性边界

题目 只要求所有时刻的输出相同，会出现什么退化解?
解题思路 输出恒定向量也满足形式上的一致性。边界条件、数据/教师锚点和适当参数化负责把一致性绑定到正确轨迹终点。PCM 对各 phase 也要保留相应边界。

7.3 进阶：JVP

题目 MeanFlow 训练的 JVP 需要显式建立 $d \times d$ Jacobian 吗? 如果把它替换为有限差分，有哪些代价?
解题思路 方向导数可用自动微分直接求，避免显式矩阵。有限差分引入额外前向与步长选择，过小易消减误差、过大有截断偏差；时间 embedding 和低精度也会影响稳定性。

第八章　分布蒸馏与 on-policy 学习

轨迹回归保留配对关系，分布蒸馏约束学生成品的总体分布。本章先明确学生状态分布与教师接口，再追踪 score、判别器、特权历史和黑盒成品如何转为学生更新。

8.1　On-policy 蒸馏、散度与教师接口

on-policy 描述状态来自谁；logit KL 只是可能的监督方式之一。

LLM 中学生先采样自己的前缀 $y_{<i} \sim p_S$，教师在这些前缀上提供下一 token 分布。学生学的是它实际会走到的位置，而不只是数据集标准答案的前缀。

Forward KL、Reverse KL、JSD 与 skew KL

在相同样本空间上，设教师 P、学生 Q：

$$D_{\mathrm{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}, \qquad D_{\mathrm{KL}}(Q\|P) = \sum_x Q(x) \log \frac{Q(x)}{P(x)}.$$

前者重视教师概率质量的覆盖，后者更严惩学生把概率放在教师不认可的区域；有限模型容量下常表现出不同的覆盖/集中倾向。

$$M = \tfrac{1}{2}(P+Q), \qquad \mathrm{JSD}(P,Q) = \tfrac{1}{2}D_{\mathrm{KL}}(P\|M) + \tfrac{1}{2}D_{\mathrm{KL}}(Q\|M).$$

JSD 对称，使用自然对数时不超过 $\log 2$。skew KL 的一种约定是 $D_{\mathrm{KL}}(P\|(1-\alpha)P + \alpha Q)$，用混合分布平滑分母；**不同论文的系数方向可能相反，要看定义**。GKD 探索不同散度和学生生成数据的组合。[36]

MiniLLM 的监督接口与 tokenizer 对齐

MiniLLM 重点是 reverse KL 与相应优化。直接 token-level KL 需要可比较的词表事件和上下文切分。不同 tokenizer 的索引可能对应不同字符串，需要先建立共同事件空间。[35] 异构 tokenizer 可以通过字符串序列级蒸馏、重新分词后的监督、显式 token 对齐/投影等方式学习；这些方案分别建立序列级或 token 级监督。教师最终文本可用于 SFT，逐 token 蒸馏则进一步利用概率分布。

迁移到视频

视频可对齐连续速度、score、轨迹或最终样本分布；黑盒教师也可提供样本级比较。**OPD 可使用连续监督；具体目标由教师可提供的接口决定。**

8.2　DMD / DMD2 的分布匹配与梯度

教师与学生分布的 score 差，为生成器提供连续空间的更新方向。

设学生生成 $z = G_\theta(\epsilon, c)$，重新加噪 $z_t = \alpha_t z + \sigma_t \eta$。冻结教师估计目标分布 score s_T，可训练 fake-score 网络估计学生分布 score s_F。对噪声平滑后的分布做 KL 匹配，可得到形如：

$$\nabla_\theta L \approx \mathbb{E} w(t) \left[s_F(z_t, t, c) - s_T(z_t, t, c) \right] \frac{\partial z_t}{\partial \theta}.$$

score 是 $\nabla_z \log p_t(z)$，是连续空间中 log-density 的梯度，**描述概率密度的局部变化方向**。[22]

把 score 转换成干净 latent 预测

把两者转换为干净 latent 预测后，常见实现构造

$$g = \text{normalize}(\hat{z}_F - \hat{z}_T), \quad L_{\text{surrogate}} = \tfrac{1}{2}\,\|z - \text{sg}[z - g]\|_2^2 .$$

因为目标侧 stop-gradient，梯度对 z 等于 g，梯度下降沿 $-g$ 调整学生。这里 MSE 作为**代理损失**，把预定分布梯度 g 注入生成器。[32]

三个更新步骤

1. 学生 rollout 生成样本；用教师和 fake-score 计算更新方向；更新学生。
2. 将学生样本 detach，加噪；用普通去噪/flow 目标训练 fake-score，使它跟上学生分布。
3. 教师不更新；推理时只保留学生。

若 fake-score 跟不上，分布梯度可能失真；这解释了为什么 critic 更新频率、学习率和训练稳定性很重要。

DMD2 相比 DMD

DMD2 去掉原 DMD 中依赖配对数据的回归项，通过两时间尺度更新提高稳定性，并引入 GAN 监督和多步学生的训练设计。**score 分支估计分布梯度，GAN 分支提供真假判别监督**；实现可以共享部分结构。[23]

学生在某些指标或人类偏好上超过教师时，连同评价集、额外真实数据、GAN 项和采样设置一起解读，能看清改善来自哪些因素。

分布梯度的推导与实现

记 $p_{\theta,t}$ 为学生成品重新加噪后的密度，$p_{T,t}$ 为教师目标密度。对 $D_{\text{KL}}(p_{\theta,t}\|p_{T,t})$ 求参数梯度，利用可重参数化样本 z_t 和边界项消失等条件，可得到

$$\nabla_\theta L_t = \mathbb{E}\left[(\nabla_z \log p_{\theta,t} - \nabla_z \log p_{T,t})^\top \frac{\partial z_t}{\partial \theta}\right].$$

辅助 fake-score 网络近似第一项，教师提供第二项。这是分布目标的一种梯度实现；对每个学生样本直接回归一个固定教师成品，会形成另一种配对目标。

计算图与梯度符号

在高斯路径 $z_t = \alpha_t z + \sigma_t \eta$ 下，score 与干净预测的关系为

$$s(z_t, t) = \frac{\alpha_t \hat{z}_0 - z_t}{\sigma_t^2}, \quad s_F - s_T = \frac{\alpha_t}{\sigma_t^2}(\hat{z}_F - \hat{z}_T).$$

再乘 $\partial z_t/\partial\theta = \alpha_t \partial z/\partial\theta$，时间因子可合并进实现权重。代理损失 $\frac{1}{2}\|z - \text{sg}(z-g)\|^2$ 的梯度就是 g。将教师或 fake-score 当普通端到端损失继续求导，会额外引入 Jacobian 项，改变这条预定梯度。

三种梯度状态

学生更新时，样本到学生参数的路径保持可导，分布修正向量被视为固定目标；fake-score 更新时，学生成品 detach 后用加噪回归训练；教师全程冻结。共享 backbone、LoRA 或 offload 会改变内存实现，三个角色的数学职责仍应分开。[22, 23]

不同 latent 空间的接口问题

DMD 要求在同一学生噪声状态上得到目标 score，或者具备已推导的密度与坐标变换。仅仅把两个 VAE latent reshape 到同一尺寸，无法建立这种对应。教师只提供 RGB 成品时，可以选择成品 SFT、判别器或 AFD 一类样本接口；这时监督与计算图也随之改变。

8.3 SiD 的 score identity 与 Fisher 目标

学生生成的样本，加上自己采的噪声，就能建立可用于学习的 score 恒等式。

设学生一步生成 $x = G_\theta(\epsilon, c)$。为方便推导，这一节用加性噪声 $y = x + \sigma\eta$，$\eta \sim \mathcal{N}(0, I)$。学生平滑分布为

$$p_{\theta,\sigma}(y) = \int p_\theta(x)\,\mathcal{N}(y; x, \sigma^2 I)\,dx.$$

学生 score 记为 $s_S(y) = \nabla_y \log p_{\theta,\sigma}(y)$，教师 score 为 $s_T(y)$。

高斯去噪恒等式的推导

对高斯核求导，再除以边缘密度，可得

$$\boxed{s_S(y) = \frac{\mathbb{E}[x \mid y] - y}{\sigma^2}.}$$

给任意满足可积条件的向量函数 $h(y)$，再取期望：

$$\mathbb{E}_y[s_S(y)^\top h(y)] = \mathbb{E}_{x,y}\left[\frac{(x-y)^\top h(y)}{\sigma^2}\right].$$

右边使用可采样的 (x, y) 对。直观上，未知的边缘 score 可以通过条件去噪关系参与目标估计。

Score 匹配的分布目标

用 Fisher divergence 视角看，目标是让两种 score 接近：

$$\mathcal{F} = \mathbb{E}_{y\sim p_{\theta,\sigma}} \left\| s_S(y) - s_T(y) \right\|^2.$$

SiD 用 score identities 推导可训练的 surrogate，并训练辅助去噪器估计当前学生分布。更新时交替训练生成器和辅助网络，教师保持冻结；具体 surrogate、权重与停止梯度按论文配置执行。[75]

把未知 score 项替换为可采样量

令 $h(y) = s_S(y) - s_T(y)$，代入上一恒等式，可把理想 Fisher 目标写成

$$\mathcal{F} = \mathbb{E}_{x,y}\left[\left(\frac{x-y}{\sigma^2} - s_T(y)\right)^\top (s_S(y) - s_T(y))\right].$$

这是精确 score 下的等式；学生样本和噪声提供 $(x-y)/\sigma^2$，辅助去噪器近似剩余学生 score。SiD 进一步组合不同恒等式得到的 surrogate，控制估计误差与训练稳定性。由恒等式替换得到的目标值等价性，与把近似网络冻结后对生成器求导的性质，需要分别分析。[75]

与 DMD 的关系

方法	主要推导视角	共同的工程对象
DMD	KL 的分布梯度，出现 $s_S - s_T$	学生样本、教师去噪器、fake-score
SiD	score identity 与 Fisher 目标	学生样本、教师去噪器、辅助去噪器

蒸馏的数据访问与计算成本

蒸馏阶段可依靠教师和学生生成样本组织监督，减少对原始训练数据的依赖。推理保留一步学生，训练期则承担教师和辅助分支的成本。比较实验时分别列出数据访问、教师接口、训练算力、NFE 和质量。

SiD 的“identity”指概率关系里的恒等式。它把难以直接计算的分布量，转写为可从生成样本和加噪过程估计的训练量。

8.4 TDM 与采样步数相关的轨迹匹配

区分匹配“点、轨迹、分布、映射”，再看能否组合。

路线	想学什么	常见代价/限制
Progressive	两步映射压成一步	多轮蒸馏，依赖时间网格
CM / LCM	全程终点一致	一步难度大，多步需特定采样
PCM	phase 内终点一致	phase 与 sampler 要协调
rCM	连续一致性 + 分布规则化	JVP、fake-score 和精度开销
DMD / DMD2	学生样本分布接近目标	score 访问、critic 稳定性
TDM	轨迹上的分布匹配	按时间组织分布监督
GAN/对抗蒸馏	判别器难区分两种分布	稳定性与多样性权衡
Reflow / 流映射	更直或更大跨度的输运	路径、数据生成和训练成本

轨迹中间分布与步数条件

TDM 研究 *Learning Few-Step Diffusion Models by Trajectory Distribution Matching*。让 K 步学生生成整条离散轨迹，保存各中间状态；目标是在对应噪声时间匹配教师分布：

$$L_K = \sum_i D_{\mathrm{KL}}(p^{(K)}_{\theta,t_i} \| p_{T,t_i}).$$

为了利用教师 score，把学生中间状态再加噪到相应区间的 τ，并匹配这些平滑边缘分布。记其密度为 $p_{\theta,\tau|t_i,K}$，则目标概括为

$$L = \mathbb{E}_K \sum_i \mathbb{E}_{\tau \in I_i^K} w_\tau D_{\mathrm{KL}}(p_{\theta,\tau|t_i,K} \| p_{T,\tau}).$$

K 决定学生采样预算，I_i^K 是相应中间状态的加噪区间。学生与 fake-score 同时接收 K 条件，分别描述不同预算下的生成分布。[24]

若把两个预算 K_1, K_2 的样本混在一起训练无条件 fake-score，它估计的是混合分布。等比例混合时

$$s_{\mathrm{mix}}(z) = \frac{p_1(z)s_1(z) + p_2(z)s_2(z)}{p_1(z) + p_2(z)}.$$

这一般偏离任一预算所需的单独 score。步数条件由此具有明确的统计作用。TDM 的差异还包括中间轨迹分布监督和稳定化代理目标；它把逐点轨迹回归放宽到分布匹配，采样过程仍按指定时间网格执行。

论文在 CogVideoX-2B 的 4 NFE 设置中报告 VBench total 从教师 80.91 到 81.65，即提高 0.74 分，相对约 0.91%。这项质量比较与端到端速度分别记录。[24]

TDM 与 TMD 不同

TMD 是 *Transition Matching Distillation*，研究以较重 backbone 和较轻 flow head 构成转移过程。缩写仅交换一个字母，方法却不同。本书分别列出两篇来源。[25]

少步蒸馏与系统优化的组合

稀疏/局部 attention、低比特量化、特征缓存、减少分辨率、分阶段超分、多卡并行、编译融合，都能降成本，但它们不必改变生成分布目标。实践中常组合：**少做几步 × 每一步更便宜 × 服务更高效**。组合后的瓶颈会变化，最终收益通过统一计时确认。

先问“蒸馏对象是什么”，再问“是否 on-policy”，最后问“推理省在哪”。这三个标签相互独立。

8.5 ADD、SDXL-Lightning 与 Hyper-SD

轨迹监督给出学习方向，对抗监督补充成品的细节与真实感。

ADD：Adversarial Diffusion Distillation

学生用少数步骤生成 $\hat{x}$，冻结扩散教师提供蒸馏信号，同时训练判别器区分真实与生成样本。目标结构可以写成

$$L_G = \lambda_{\text{dist}} L_{\text{teacher}}(\hat{x}) + \lambda_{\text{adv}} L_{\text{GAN}}(D_\psi(\hat{x})).$$

教师项维持生成语义与合理结构；判别项直接关注样本外观。生成器更新时，梯度经过冻结的教师 / 特征网络与判别器传回学生；判别器更新时，生成样本通常 detach。ADD 使 SDXL-Turbo 等模型支持 1–4 步生成。[76]

SDXL-Lightning：逐级减少步数，配合对抗蒸馏

从预训练 SDXL 出发，用 progressive 与 adversarial distillation 学习更少步的学生，提供 1、2、4、8 步等 checkpoint / LoRA 配置，目标分辨率为 1024^2。阶段式蒸馏让学生逐渐适应更大跳跃，对抗项增强最终样本质量。[77]

配对回归与成品分布的差异

学生容量有限，一步逼近复杂映射时容易平均掉细节；低回归误差与人眼偏好的纹理质量之间存在差距。判别器从样本分布提供补充信号，同时带来训练稳定性与模式覆盖的权衡。

路线	学习信号	部署对象
Progressive	两次教师更新对应一次学生更新	特定少步学生
ADD	扩散教师信号 + 对抗信号	一步 / 少步生成器
SDXL-Lightning	分阶段蒸馏 + 对抗信号	步数对应的模型或 LoRA
DMD2	分布匹配 + 稳定化 + GAN 等	一步 / 少步学生

少步质量的比较协议

先找到原模型的采样步数、分辨率、CFG 和评价方式；再检查质量指标或人评。减少 NFE 是明确的计算变化；“接近”则由具体数据与指标定义。实际部署还需使用该 checkpoint 对应的 prediction type、时间表和 guidance 设置。

Hyper-SD：跨步数蒸馏与偏好优化

Hyper-SD 关注多种计算预算下的生成质量。其核心组件包括 trajectory-segmented consistency distillation、人类反馈优化，以及进一步压缩到极少步的 score distillation。[78]

轨迹分段的优化作用

把整个去噪过程拆成若干区间，每段先学习较短跨度，再逐步合并区间。设 r 为当前区间终点，训练关系可概括为

$$F_\theta(z_t, t \to r) \approx \mathrm{sg}[F_{\bar{\theta}}(z_s^T, s \to r)], \qquad r \le s < t.$$

通过阶段安排把短跳逐步变成长跳，降低一次学习全程映射的难度。它与 PCM 都使用分段思想，具体训练阶段、反馈信号和跨步数支持各有设计。

反馈与分布监督的分工

1. **轨迹学习**：保留教师的生成路径与条件能力。
2. **人类反馈**：偏向更符合人类喜好的样本。
3. **score distillation**：进一步校正极少步学生的生成分布。

论文还提供 unified LoRA，覆盖 1–8 步的使用范围；使用时仍按发布配置选择步数与 scheduler。

基线与收益口径

论文在其 SDXL 单步比较中报告，相对 SDXL-Lightning，CLIP Score 增加 0.68，Aesthetic Score 增加 0.51。这是对应评分量表的分数差。判断收益还要看图像分布、文本对齐、多样性、人评和实际耗时。

轨迹、分布与偏好目标的组合

关注点	代表路线
学习更大步长的确定性映射	progressive / PCM / CTM
连续时间训练稳定与扩展	sCM / rCM
用学生分布反馈纠正生成器	DMD / DMD2 / SiD
增加外观与偏好信号	ADD / Lightning / Hyper-SD

读组合型方法时，把总 loss 拆成“轨迹信号、分布信号、偏好信号”，再标出各自依赖的教师、数据与可训练分支。

8.6 OPSD-V 的特权历史与自蒸馏

教师优势来自更干净的长视频上下文，而不只是 EMA 参数更平滑。

OPSD-V 在已有少步 AR 模型上继续后训练。学生保留自己生成的历史；教师在同一个 noisy latent 和同一个去噪时间上预测，但使用含真实长视频的更干净 cache。

一个直观例子

生成第 7 个 chunk 时：

- 学生历史：真实首 chunk，加自己生成的第 2–6 chunk。
- 教师历史：真实首 chunk 和真实第 2–5 chunk，加学生刚生成的第 6 chunk。

保留最近的学生 chunk 是为了维持当前接续状态；替换更早历史是为了减少累计污染。真实视频作为教师的特权上下文，帮助它在学生状态上给出更可靠的速度预测。

核心监督

$$L = \mathbb{E} \sum_{k,j} \left\| v_\theta(z_{k,t_j}, t_j, c, H_k^S) - \mathrm{sg}[v_{\bar{\theta}}(z_{k,t_j}, t_j, c, H_k^T)] \right\|_2^2 .$$

这里两分支看的是同一学生访问的状态，区别在 cache。公式省略了 warm-up mask 等实现细节。论文沿用原四步采样器，监督各去噪步，不新增 DMD、reward 或对抗 loss。[37]

LoRA 学生与 EMA 教师

设冻结底座为 W_0，学生有效权重为 $W_0 + \Delta W(a_S)$，教师使用 $W_0 + \Delta W(a_T)$。在线 LoRA 参数 a_S 用梯度更新，目标参数用

$$a_T \leftarrow 0.9999 a_T + 0.0001 a_S .$$

梯度只更新学生 LoRA，教师 LoRA 由滑动平均更新，底座保持冻结。LoRA 与有效权重平均的区别见第六章。

数据与部署

论文使用约 3,800 条一分钟长视频，并在 Self Forcing 与 LongLive 上验证。它在已有四步模型上改善长程生成质量。推理不需要真实未来视频或教师分支。[37]

8.7 AFD：成品比较到学生侧向量场更新

AFD 的共享接口是 (c, X)：黑盒教师返回完整视频，AR 学生在自己生成的历史上 rollout。教师和学生可以使用不同 VAE、主干和步数。判别器通过同一提示词下的成品比较提供自适应反馈。[38]

$$L_D = -\mathbb{E} \log \sigma[D_\phi(X^T, c) - D_\phi(X^S, c)].$$

这里 D_ϕ 是实数评分，差值经过 sigmoid 后构成 Bradley–Terry 偏好概率。将学生得分减去 baseline、停止梯度，再归一化成 $w_i \in [0, 1]$，用于控制样本被强化或抑制的程度。

已知的前向路径提供速度标签

编码学生成品得到 z_i，固定该 rollout 并重新采样噪声：

$$z_{t,i} = \alpha_t z_i + \sigma_t \epsilon, \qquad u_i = \dot{\alpha}_t z_i + \dot{\sigma}_t \epsilon .$$

$t = 0$ 为干净端。u_i 由学生的前向日程直接计算，教师只通过 w_i 影响偏好方向。对加权成品分布 $p^+(z) \propto w(z)p_S(z)$，最优 FM 预测是

$$v^+(z_t, t) = \frac{\mathbb{E}[w(z)u \mid z_t, t]}{\mathbb{E}[w(z) \mid z_t, t]}.$$

这是加权最小二乘的条件均值解释，适用于非负权重且分母为正的状态。它说明成品权重如何影响整个噪声区间的目标场。

正负分支的 stop-gradient 结构

记当前预测 $v = v_\theta(z_t, t, c)$，AFD 采用 DiffusionNFT 形式：[134]

$$v_\theta^+ = (1-\beta)\,\mathrm{sg}(v) + \beta v, \qquad v_\theta^- = (1+\beta)\,\mathrm{sg}(v) - \beta v,$$

$$L_{\mathrm{NFT}} = \mathbb{E}\big[w\|v_\theta^+ - u\|^2 + (1-w)\|v_\theta^- - u\|^2\big].$$

前向求值时两个分支均等于 v；反向导数分别为 $+\beta I$ 与 $-\beta I$。因此当前评估点的预测梯度为

$$\frac{\partial L_{\mathrm{NFT}}}{\partial v} = 2\beta(2w-1)(v-u).$$

$w > 1/2$ 时靠近标签，$w < 1/2$ 时反向调整，$w = 1/2$ 时该项的当前梯度为零。每轮重新计算 stop-gradient 锚点；它与把负系数直接乘在一个无锚点 MSE 上具有不同的计算图。

完整目标还加入初始学生的速度先验约束：

$$L = L_{\mathrm{NFT}} + \lambda\mathbb{E}[w\|v_\theta - v_{\mathrm{ref}}\|^2].$$

该项限制能力遗忘；实际稳定性仍取决于权重归一化、β、先验强度和判别器更新。训练无需穿过黑盒教师，也无需保存整条反向采样链的梯度。

数据、比较与收益的范围

论文以 Seedance 2.0 成品为教师，对 Self-Forcing / Causal-Forcing 学生进行物理域适配，使用 200 条 VideoPhy-2 样本对应的提示。对照包含原学生、教师样本 SFT 和成品级对抗训练。该实验检验有限数据下的运动/物理适配，数据规模和目标域应连同通用质量保持一起阅读。[38]

8.8 蒸馏配方在实际视频系统中的组合

系统	公开的主要配方	应一起核对的接口
Vidu S1	因果教师、DMD + PCM 感知正则	TF/DF、音视频历史、TwinCache
Seedance 1.0	TSCD、RayFlow score distillation、偏好对抗蒸馏	运动、多镜头、CFG 与系统栈
MagiHuman	DMD2，8 步且无 CFG	低清 base、5 步超分、Turbo VAE
Vivix-A1	2 步、长时自分布、latent/原始空间联合优化	连续交互与动作多样性

这些行来自对应报告，详细公开程度不同。[52, 167, 161, 171]

Seedance 1.0 的报告将 TSCD 分段一致性与 RayFlow 的 score distillation 结合，再用偏好导向的对抗阶段改善少步伪影；它描述的是多阶段配方。MagiHuman 则公开使用 DMD2，低分辨率 base 与高分辨率超分分开计步。表中的“8 步”只指 base；1080p 输出还要计入超分与解码。

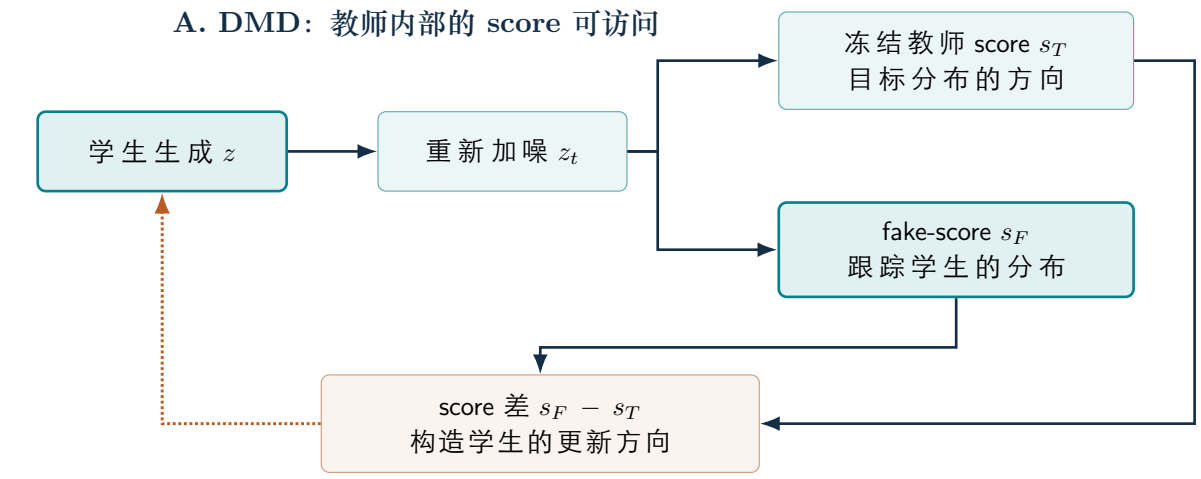


图 8.1　DMD 与 AFD：教师能提供什么?。白盒 score 接口与黑盒成品接口，决定了蒸馏监督怎样构造。重要：score 是连续 log-density 对状态的梯度。AFD 的黑盒教师无需提供内部 flow 目标；可观测成品比较与学生侧加噪构造共同提供学习信号。图中展示监督来源与更新链路。[22, 23, 38]

可用一个通用记账表达比较组合配方：

$$L = L_{\text{distribution}} + \lambda_C L_{\text{consistency}} + \lambda_R L_{\text{reward}} + \lambda_A L_{\text{adversarial}}.$$

这是损失类别的示意，各系统选择其中不同项、阶段和教师。已公开的名字并不足以确定完整训练：还要记录样本来自教师还是学生、监督作用于 latent 还是像素、奖励是否可微、哪些网络参与反传、初始化与预算。

本章思考与进阶练习

8.1 教师接口

题目 教师只返回 RGB 视频，学生使用不同 VAE。哪些蒸馏路线能够直接利用这个接口？
解题思路 教师样本 SFT、成品判别器与 AFD 等可用共同可观测样本。DMD 的局部教师 score、PCM 的教师轨迹对子需要额外接口或经过推导的变换。

8.2 代理梯度

题目 对 $L = \frac{1}{2}\|z - \mathrm{sg}(z - g)\|^2$ 求 z 的梯度，并解释去掉 stop-gradient 后会发生什么。
解题思路 保留 stop-gradient 时梯度为 g。若把 g 当常量且去掉 stop-gradient，损失化为常数 $\|g\|^2/2$，梯度为零；若 g 依赖 z，则得到与预定修正不同的 Jacobian 项。

8.3 进阶：自蒸馏的信息增量

题目 OPSD-V 的教师与学生共享底座，教学信号从何而来？设计一个消融来分离 EMA 与真实历史的贡献。
解题思路 教师获得更干净的特权历史，EMA 提供参数平滑。做相同/不同 cache 与启用/停用 EMA 的二乘二消融，并固定 rollout、步数和训练预算；分别测短视频质量、长时漂移和 prompt 服从。

第九章　后训练、奖励与强化学习

后训练把质量、偏好和任务反馈纳入生成模型。本章按监督数据、可微奖励、随机转移概率和前向过程回归展开，重点比较各路线的梯度路径、rollout 成本与稳定性。

9.1　SFT 与直接偏好优化

预训练学会生成，后训练改变质量、偏好、条件服从或部署行为；各项目标对应不同的数据与反馈接口。

SFT：用更好的配对数据继续训练

给定视频与条件 (X, c)，继续使用去噪/FM 目标：

$$L_{\mathrm{SFT}} = \mathbb{E}\,\|v_\theta(z_t, t, c) - u_t\|_2^2 .$$

SFT 可以沿用预训练损失，重点改变数据质量、任务条件与参数更新范围。连续视频生成器同样可以使用 SFT，选择 LoRA 或全量更新。

DPO 的成对偏好目标

数据为 (c, X^+, X^-)。离散语言里的原型是：

$$L_{\mathrm{DPO}} = -\log\sigma\left(\beta\left[\log\frac{p_\theta(X^+|c)}{p_{\mathrm{ref}}(X^+|c)} - \log\frac{p_\theta(X^-|c)}{p_{\mathrm{ref}}(X^-|c)}\right]\right).$$

视频 diffusion 的完整成品 likelihood 难直接求，Diffusion-DPO 用扩散模型的变分/去噪结构构造可优化目标。为了记忆，可写成简化误差差值形式：

$$L \sim -\log\sigma\{-\beta[(e_\theta^+ - e_{\mathrm{ref}}^+) - (e_\theta^- - e_{\mathrm{ref}}^-)]\}.$$

e 是相应加权去噪误差；实际目标进一步指定时间权重、共享随机量和参考模型处理。[39]

三种常用后训练接口

接口	监督内容	核心动作
高质量数据	这是要学习的样本	SFT / 加权回归
成对偏好	A 比 B 好	DPO / reward model 排序
在线打分	刚生成的结果有多好	可微 reward 或策略梯度

少步蒸馏也是后训练，但目的不同

质量对齐关心“生成什么更好”；少步蒸馏关心“怎样更快地产生近似同样的分布”。先 RL 后蒸馏、先蒸馏后 RL、联合优化都存在，训练收益与推理路径保持要分别评估。

LoRA 指定参数更新方式，可与 SFT、RL 或蒸馏搭配。用了 LoRA，仍要问训练数据、目标和 rollout 来自谁。

9.2 ReFL、DRaFT 与可微奖励

如果评分对画面的变化可求导，就可以沿“评分器 → 画面 → latent → 生成器”反传。

设生成视频 $\hat{X} = \mathrm{Dec}(G_\theta(\epsilon, c))$，冻结评分器 $R(\hat{X}, c)$：

$$L_R = -\mathbb{E}R(\hat{X}, c), \qquad \nabla_\theta L_R = -\mathbb{E}\left[\frac{\partial R}{\partial \hat{X}}\frac{\partial \hat{X}}{\partial G_\theta}\frac{\partial G_\theta}{\partial \theta}\right].$$

冻结 reward 参数，同时保留输入梯度

评分器的权重可以不更新，但需要保留它对输入画面的梯度。若先转 JPEG、调用只返回整数的远程 API，或做不可微 argmax，再把分数读回来，通常已经没有这条路径。

ReFL 的具体做法

ImageReward 是文本-图像偏好评分器；ReFL 在去噪过程较后阶段选取步骤，根据预测的干净图像得到 reward，并反传到生成模型，同时用原训练目标等约束维持生成能力。**ReFL 原作面向图像；视频适配还需时序评分与长链路显存设计。**[40] 相关 DRaFT 研究通过整个采样链反传，以及只反传最后 K 步的 DRaFT-K 等节省内存的变体。长视频中，VAE 解码、时序评分和多步计算图使成本更大。[41]

一个一维例子

假设生成值为 $x = \theta$，reward 为 $R(x) = -(x-3)^2$。直接优化 $-R$ 的梯度是 $2(\theta-3)$，会把输出推向 3。这是**路径导数**：知道怎样改动 x 会提高分数。

训练代价与诊断

- 奖励网络可能给出可利用但不符合人类判断的梯度，形成 reward hacking。
- 只优化美学可能牺牲动作、文本关系和多样性。
- 截断反传降低显存，把梯度限制在保留的采样区间。
- 可微训练需要保留评分器对输入的梯度接口。

可微 reward 问的是：**这张画面往哪改能涨分？**策略梯度问的是：**哪些采样行为导致高分，应该更常发生？**

9.3 VADER 与 VideoDPO

视频对齐既要处理逐帧画质，也要处理运动、时序和文本关系。

VADER：沿奖励梯度更新视频生成器

给提示 c 采样视频 $x_\theta = G_\theta(\epsilon, c)$，使用可微奖励 R：

$$\mathcal{L} = -\mathbb{E}R(x_\theta, c), \qquad \nabla_\theta \mathcal{L} = -\mathbb{E}\left[\left(\frac{\partial x_\theta}{\partial \theta}\right)^\top \nabla_x R(x_\theta, c)\right].$$

梯度经过奖励网络、视频解码器与生成步骤。VADER 研究视频上的奖励梯度对齐，以及截断反传等节省显存的方法，覆盖文本到视频和图像到视频。奖励可来自图像或视频模型；选择逐帧奖励时，还需配合时序指标观察运动。[119]

VideoDPO：把视频评价转成偏好对

对同一 prompt 生成多个视频，用 OmniScore 汇总帧内、帧间和文本符合度，选最高分为 x^+、最低分为 x^-，然后做偏好训练。论文使用 VidProm 的 1 万个提示，每个提示采样 4 个候选。[120] 其重加权还利用分数直方图频率 $p(s)$：

$$w_{\text{pair}} = \left(\frac{b}{\sqrt{p(s^+)p(s^-)}} \right)^a, \qquad \mathcal{L}_{\text{video}} = w_{\text{pair}} \mathcal{L}_{\text{DPO}}.$$

a 控制重加权强度，b 为比例常数。分数差决定 best / worst 配对，直方图频率进一步决定样本对权重；这里分开命名 a、b，以便与 DPO 的系数区分。

视频后训练的收益维度

VideoDPO 的 VideoCrafter2 实验中，VBench 总分由 80.44 到 81.93，语义分由 73.42 到 77.38。总分提升为 1.49 个百分点，语义分提升为 3.96 个百分点；其他基模的子指标变化各有差异。该结果衡量生成质量，采样时延另计。[120]

选路线的依据：有可微奖励且能承担反传显存，可研究 VADER；有可靠偏好对，可研究视频 DPO；只有标量在线反馈，可接前文 DDPO / DanceGRPO。训练奖励与独立测试指标分别记录。

9.4 VideoReward 与 Flow-DPO

通过相对去噪误差构造偏好目标，再把奖励分成画质、运动和文本维度。

先把评价数据与奖励模型做好

Improving Video Generation with Human Feedback 构建来自 12 个视频生成模型的多维偏好数据，包含约 18.2 万条 prompt-video-pair 标注；VideoReward 分别评估画质、运动质量和文本符合度。其修订版还研究平局标签与不同奖励 token 的信息访问范围。[121]

Flow-DPO：对比相对参考模型的误差变化

对优选 / 次选 latent z_0^+, z_0^-，分别加噪得到 $z_t^\pm$；$u^\pm = \epsilon^\pm - z_0^\pm$。定义

$$e_\theta^\pm = \left\| v_\theta(z_t^\pm, t, c) - u^\pm \right\|^2, \qquad \Delta^\pm = e_\theta^\pm - e_{\text{ref}}^\pm.$$

则可用紧凑形式表达论文采用的恒定系数目标：

$$\mathcal{L}_{\text{FlowDPO}} = -\mathbb{E} \log \text{sigmoid} \left[-\frac{\beta}{2} (\Delta^+ - \Delta^-) \right].$$

若优选误差相对参考下降更多，$\Delta^+ - \Delta^- < 0$，sigmoid 内的量增大，loss 下降。参考模型冻结；网络仍输出连续速度。由噪声误差直接换算会出现 $(1-t)^2$ 权重，论文比较后采用恒定 β。[121]

奖励模型的多种优化接口

Flow-RWR：给高奖励样本更高回归权重，可写为 $\mathbb{E}[e^{R(z_0,c)} \|v_\theta - u\|^2]$，奖励温度可并入 R。**Flow-NRG**：推理时使用适配带噪 latent 的奖励模型，通过对当前状态的梯度调整速度场；可在采样时调配各奖励维度权重。[121]

一个数值直觉与验证方式

假设优选误差从参考的 3 降到 2，次选误差保持 4，则 $\Delta^+ = -1, \Delta^- = 0$，偏好方向得到加强。若两者都同幅下降，偏好间隔保持原样。训练后除奖励胜率外，还需固定种子与 prompt，测独立 benchmark、人类盲评和多样性；画质、运动与文本指标分列，便于观察取舍。

公式沿用本文线性路径与时间方向，省略批次、位置平均等实现维度。训练期方法与推理期 guidance 分开统计额外成本。

9.5 DDPO 的连续动作策略梯度

奖励可以是黑盒；训练使用采样转移的概率密度。

DDPO = Denoising Diffusion Policy Optimization。把去噪看成 MDP：状态是 (z_t, t, c)，动作是下一状态 z_s，最终生成结果获得 reward。[42]

连续动作也有概率密度

若采样器的随机转移为：

$$p_\theta(z_s \mid z_t, c) = \mathcal{N}\big(\mu_\theta(z_t, t, c), \sigma_t^2 I\big),$$

则

$$\log p_\theta(z_s \mid z_t, c) = -\frac{\|z_s - \mu_\theta\|_2^2}{2\sigma_t^2} - d \log \sigma_t - \frac{d}{2}\log(2\pi).$$

整个动作向量可以是连续 latent。连续变量的单点概率为零；**概率密度描述单位体积中的概率质量**，其 log-density 可计算并用于学习。

策略梯度原理

设 $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(X_0, c)]$：

$$\nabla_\theta J = \mathbb{E}\left[(R - b)\sum_t \nabla_\theta \log p_\theta(z_s \mid z_t, c)\right].$$

b 是减少方差的 baseline。这个梯度不要求 $\partial R/\partial X$，所以可以用人类打分、OCR、规则检查或远程评审。它是从高/低分的采样结果反推哪些行为应更常出现。DDPO 的重要实现包括重要性采样和类似 PPO 的裁剪目标，限制一次更新对旧策略的偏离。[43]

确定性 ODE 的障碍

完全确定的转移对应 Dirac delta。使用上述高斯 logprob 时，需要显式随机转移；确定性生成也可选择路径导数等优化方法。初始噪声的随机性与逐步转移的随机性分别定义。

DDPO 用在线 rollout 和 reward 做策略优化；DPO 用偏好对及参考模型构造直接目标。两者分别依赖在线评分和成对偏好接口。

9.6 DanceGRPO 的组相对优化

用组相对优势优化图像与视频生成。

同一个 prompt 生成 G 个结果，奖励为 $R_1, \ldots, R_G$。组相对优势：

$$A_i = \frac{R_i - \bar{R}}{\mathrm{std}(R_1, \ldots, R_G) + \epsilon}.$$

直觉：同一道题里，比同组平均更好的样本应该更常出现；更差的应该更少出现。通常不需要单独训练一个 value 网络来预测 baseline。[44]

数字例子

奖励为 $(1,2,3)$，平均为 2，按总体标准差约为 0.816，则优势约为 $(-1.225, 0, 1.225)$。组内奖励完全一样时优势接近零，没有相对学习信号；稀疏二元 reward 也会遇到“全对/全错”的问题。

概率比与裁剪目标

令 $\rho_{i,t} = p_\theta(z_s^i \mid z_t^i, c)/p_{\text{old}}(z_s^i \mid z_t^i, c)$，简化的裁剪目标为：

$$J = \mathbb{E}_{i,t} \min\left(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1-\delta, 1+\delta) A_i\right).$$

实践通常最小化 $-J$，也可能带参考策略约束和其他正则。它优化的是转移概率；不需要奖励对画面的像素梯度。

对 Flow Matching 的重要适配

DanceGRPO 为 diffusion 和 rectified flow 构造随机 SDE 采样形式，使多条轨迹的探索和概率比可计算。随机转移提供概率比所需的密度。论文验证了 T2I、T2V、I2V 等设置。[45]

与 DDPO、ReFL 对照

方法	如何利用 reward	典型难点
ReFL / DRaFT	直接穿过评分与采样链求导	显存、链路可微、奖励梯度偏差
DDPO	reward 加权转移 logprob 梯度	rollout 成本、梯度方差
DanceGRPO	多样本组内优势 + 裁剪策略更新	每题多次生成、组内信号不足

9.7 Flow-GRPO：用随机转移建立连续策略密度

为明确采样方向，令 t 从 1 降到 0，$h > 0$ 表示向低噪端走的步长。记 $g(t)$ 为新增扩散系数，区别于前向路径的噪声尺度 σ_t。与理想 ODE 边缘密度相容的反向时间 SDE 为

$$dz_t = \left[v(z_t, t) - \tfrac{1}{2} g(t)^2 s(z_t, t)\right] dt + g(t) d\bar{W}_t.$$

在线性 RF 路径中，$\hat{\epsilon} = z_t + (1-t) v_\theta$，所以

$$s_\theta(z_t, t) = -\frac{z_t + (1-t) v_\theta(z_t, t)}{t}.$$

据此做 Euler–Maruyama 更新：

$$\mu_\theta = z_t - h\left[v_\theta + \frac{g(t)^2}{2t}(z_t + (1-t) v_\theta)\right],$$
$$z_{t-h} = \mu_\theta + g(t)\sqrt{h}\,\xi, \qquad \xi \sim \mathcal{N}(0, I).$$

噪声方差是 $g(t)^2 h$，因此转移 log-density 可直接计算，供 GRPO 概率比使用。h 始终为正，避免对负的 Δt 开平方。[130]

连续理论与有限步实现

边缘相容要求精确速度/score 与连续时间条件；有限步离散、CFG 和网络近似会改变实际分布。靠近端点的 $1/t$ 与噪声日程需要数值处理。训练时记录实际随机转移的 logprob，再用同一状态与动作重算新策略概率；仅比较最终视频的像素误差无法替代该概率比。

rollout 步数与部署步数

Flow-GRPO 的 denoising reduction 用较少采样步收集训练轨迹，部署仍可使用原有较多步数。其原作验证以图像生成为主；视频的长序列、解码和多样本组会显著增加 rollout 成本。选择组大小时应同步看组内奖励方差、每轮生成费用和独立测试集改善。

9.8 前向过程后训练：DiffusionNFT 与 DiffusionART

DiffusionNFT 的样本接口

DiffusionNFT 将完成的生成结果、标量奖励和已知前向加噪过程组合成正负 flow 回归。采样器可以使用既有 ODE 或少步方案，优化时无需计算逐步转移密度。AFD 将这一路径用于教师–学生成品比较；通用 NFT 则可以使用任务奖励。[134, 38]

以冻结 rollout 策略预测 v_{old} 为锚点，写成

$$v_\theta^{\pm} = v_{\text{old}} \pm \beta(v_\theta - v_{\text{old}}).$$

配合正负加权 MSE，等价的回归目标为

$$v_{\text{tgt}} = v_{\text{old}} + \frac{2w-1}{\beta}(u - v_{\text{old}}).$$

冻结 v_{old} 后对同一批数据做多次更新，会产生朝旧模型回拉的二次项；每次前向都把当前预测重新 detach 的写法，只在当前锚点处拥有对应的一阶方向。复现时应确认锚点更新时机与 rollout 策略滞后。

DiffusionART 的近期进展

2026 年 10 月 8 日的 DiffusionART 分析了前向再加噪诱导的理想场与实际反向采样场之间的差异。它先在同一 prompt 的 K 个结果上聚合奖励–成品协方差：[135]

$$C(c) = \frac{1}{K}\sum_i (R_i - \bar{R})(z_{0,i} - \bar{z}_0), \qquad G(c) = \frac{C(c)}{\mathrm{RMS}(C(c)) + \varepsilon}.$$

该方向来自指数奖励重加权的均值一阶变化。本书速度朝加噪方向定义，因此将论文朝去噪方向的速度换号后，目标写为

$$v_{\text{tgt}}(z_t, t, c) = \mathrm{sg}[v_{\text{old}}(z_t, t, c) - G(c)].$$

它在实际 rollout 状态上做回归，并结合第二次更新和 trust-region mask 控制漂移。核心变化是先聚合同 prompt 的奖励信息，再对各状态施加统一修正。论文在 HunyuanVideo-13B、等权画质/运动奖励设置中，报告达到 DiffusionNFT 后期平均训练奖励约需其 1/12.02 的 GPU-hours；独立测试和人评用于继续判断该奖励收益的泛化。

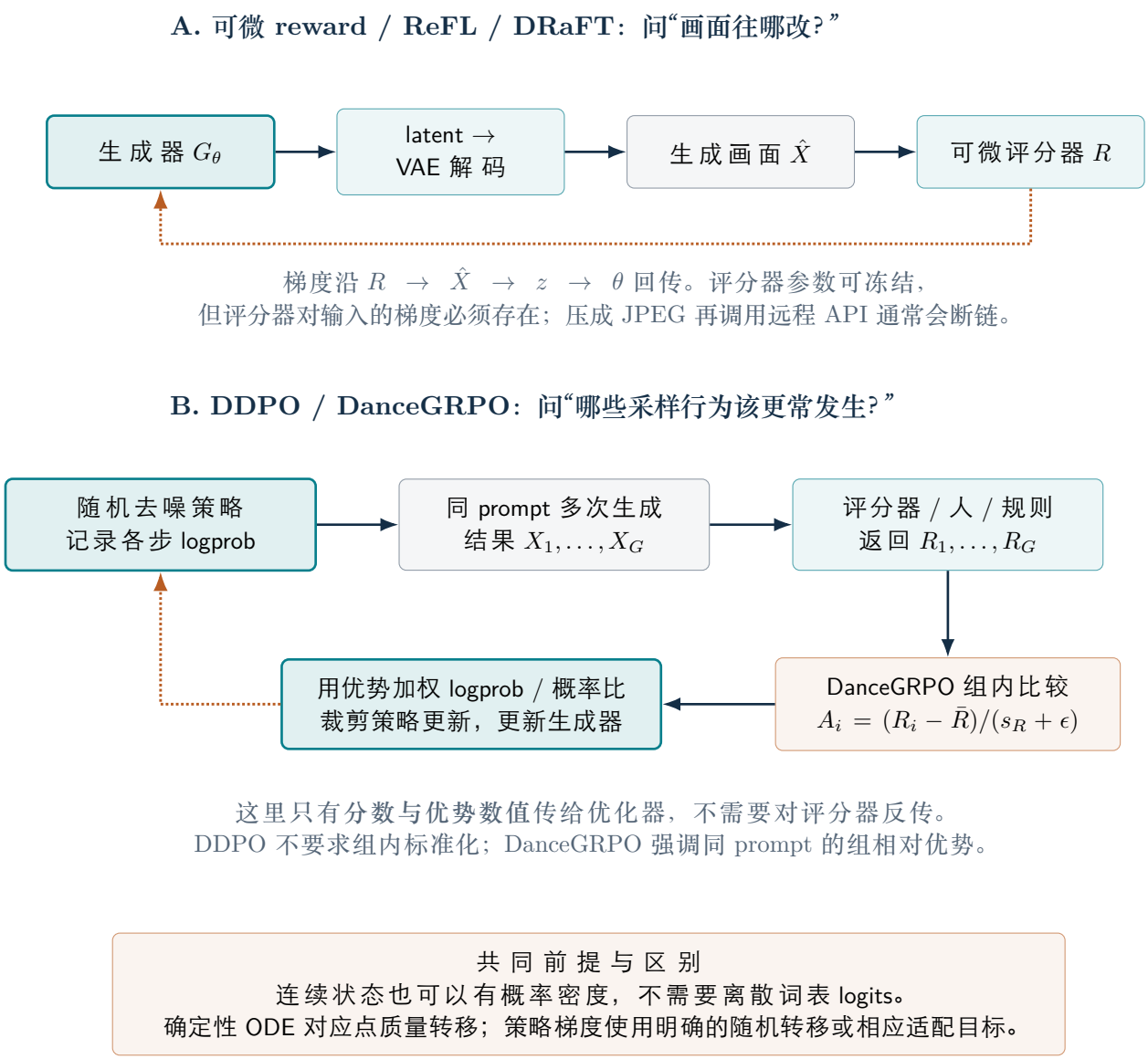


图 9.1 可微 reward 与策略梯度：两条不同的回路。一条沿画面梯度回传；另一条用结果分数改变采样行为的概率。**优化接口：**能否对 reward 输入求导？能否计算采样转移 logprob？再选直接梯度优化或 DDPO/GRPO。它们都可能更新同一个视频生成器，但使用的学习信号不同。[40, 41, 42, 44]

9.9 Seedance 的多维奖励与后训练案例

Seedance 1.0 把高质量 SFT 与视频定制的 RLHF 组合。报告区分基础能力奖励（文本对齐、结构稳定等）与运动奖励（伪影、幅度和生动性），采用成对的人类反馈训练奖励模型。1.5 pro 将声音保真、表达和音画同步纳入音视频后训练。[167, 168]

奖励向量到策略目标

为解释设计，可令奖励向量为 $\mathbf{r} = (r_{\text{align}}, r_{\text{structure}}, r_{\text{motion}}, r_{\text{audio}}, r_{\text{sync}})$。常见标量化为

$$R = \sum_j \lambda_j \frac{r_j - \mu_j}{\sigma_j + \varepsilon},$$

也可设置某些质量门槛，再优化其他维度。这是通用设计示例，报告未提供上述权重作为其正式配方。归一化避免量纲大的奖励垄断目标；约束则能避免“好看但不同步”依靠其他分数抵消。

高稳定性评分可能偏好静止或慢动作。评价中需要把运动幅度、动作完成度和伪影同时观察，用 prompt 指定动作的完成率作为独立项。多镜头还要检查主体、声线与叙事顺序；这些能力不能由单帧美学分数替代。1.5 pro 报告的近 3 倍 RLHF 训练加速针对其训练基础设施，与超过 10 倍生成加速属于不同口径。[168]

本章思考与进阶练习

9.1 三种反馈

题目 只有整数奖励且无法对画面求导，可选哪些更新？若有成对偏好但没有标量分数呢？
解题思路 前者可使用带随机转移密度的策略梯度、前向过程奖励回归或排序筛选；后者可构造 DPO、训练 BT reward，或使用支持 pairwise 的更新。选择取决于可用轨迹和计算预算。

9.2 转移密度

题目 在 Flow-GRPO 中让新增噪声 $g(t)$ 趋近零，同时继续使用同一个高斯 logprob 公式，会发生什么？
解题思路 方差趋零，分布退化为确定性转移，数值中的平方误差除以方差会失稳。应选择有明确正方差的随机训练转移，或改用适合确定性采样的优化接口。

9.3 进阶：信用分配

题目 一个 60 秒视频只因第 45 秒动作失败而得低分，整条轨迹共享同一 advantage 有什么问题？
解题思路 所有采样步骤都接收同一结果标签，局部正确动作也可能被共同压低，方差较大。可研究片段奖励、局部条件评分或状态级信用，但需防止裁剪后丢失长时因果信息，并保留最终任务成功率。

第十章　推理效率与系统部署

在采样预算之外，每次前向和整个服务系统都存在优化空间。本章从复用、稀疏和低精度进入 kernel、并行与调度，并将近期 attention 与容量路由工作放到同一成本模型中。

10.1　特征缓存与刷新策略

保持采样时间表，把部分完整网络计算换成便宜的特征复用。

设网络某块输出 $h_t = F_\theta(z_t, t, c)$。若相邻时间的特征变化较小，可近似

$$h_t \approx h_{t'} \quad \text{或} \quad h_t \approx h_{t'} + \widehat{\Delta h}(t, t').$$

缓存对象可以是层输出、残差、attention 或 MLP 特征。每隔若干步刷新，或者用误差估计动态决定刷新时机。

去噪轨迹中的局部冗余

去噪过程中，语义、构图等结构常在多个相邻步间缓慢变化；不同时间段和不同层的变化幅度也不同。固定间隔缓存简单，动态策略可以在变化快时多算、变化慢时多复用。

TeaCache 的思路

TeaCache 用经过时间 embedding 调制的输入差异，估计输出差异，再做缩放与阈值判断。这样可以在执行昂贵主干前，先用便宜指标决定是否复用缓存。原论文在 Open-Sora-Plan 上报告最高 4.41 倍加速，并同时报告对应质量变化。[82]

缓存命中与刷新成本

若总共 N 个采样时刻，其中 R 次执行完整块，其他时刻只做复用：

$$T \approx RC_{\text{full}} + (N - R)C_{\text{reuse}} + C_{\text{decide}}.$$

采样器仍更新 N 次状态，但完整网络计算次数和实际 FLOPs 已经下降。因此报告时同时给出 scheduler steps、full recomputations 和缓存开销。

三种 cache 的区别

缓存	复用依据	关键检查项
文本 cross-attention KV	同一层的文本输入保持固定	投影是否依赖 t / 其他模态
因果历史 KV	已完成历史位置保持固定	位置、条件、写入时机
去噪步间特征 cache	相邻步近似相似	刷新阈值、误差累积、质量变化

缓存策略应观察输出变化：运动大、镜头切换、prompt 更新或条件突变时，及时刷新通常更重要。

10.2 Token 裁剪、合并与稀疏注意力

分别减少要更新的位置、合并相似位置、限制位置间的连接。

Token Pruning：选一部分 token 继续计算

若当前有 N 个 token，挑选集合 S，$|S| = \rho N$。重要性可以来自变化幅度、attention、语义或可学习评分。选中的 token 执行昂贵块；其余位置可保留旧特征、从邻域恢复，或按设计跳过更新。

$$H'_S = F(H_S), \qquad H'_{\bar{S}} = \mathrm{Restore}(H, H'_S, \mathrm{cache}).$$

视频最终需要完整网格，所以要明确被跳过位置的恢复规则。缓存与 pruning 可以结合，例如优先更新变化较大的 token。[83]

Token Merging：把相似 token 暂时合并

把若干相似特征聚合为一个代表，完成计算后再展开回原位置。空间近邻或相似度匹配能减少重复处理。ToMe for Stable Diffusion 是这一路线的代表；它同时分析速度、显存与图像质量。[84]

Sparse Attention：保留 token，减少读取的邻居

$$o_i = \sum_{j \in \mathcal{N}(i)} \frac{\exp(q_i^\top k_j/\sqrt{d})}{\sum_{l \in \mathcal{N}(i)} \exp(q_i^\top k_l/\sqrt{d})} v_j.$$

$\mathcal{N}(i)$ 可包含局部窗口、同帧/邻帧、全局锚点或动态选中的块。全连接 attention 有约 N^2 对连接；若每个 query 只读 k 个位置，核心配对数量约为 Nk。

举个算量例子

把 token 数减为原来的 $\rho = 0.5$，dense attention 的配对数约变成 $0.25N^2$，MLP 约变成原来的 0.5。最终耗时还包括选择、聚合、恢复和小矩阵效率。

FlashAttention 与稀疏注意力的区别

标准 FlashAttention 通过分块与在线 softmax 减少 HBM 访存，精确计算原来的 dense attention；稀疏版本进一步改变连接集合。实现时分别标注**连接拓扑、计算精度和 kernel**，便于定位质量与速度变化。[85]

Pruning 看“哪些位置算”；sparse attention 看“每个位置读谁”；FlashAttention 看“这些读写怎样高效执行”。

在线 softmax：为何省显存而保持完整连接

对一个 query，完整概率矩阵的一行需要所有分数 $s_j = q^\top k_j/\sqrt{d}$。FlashAttention 分块读取 K/V，只维护最大值 m、指数和 l、加权分子 h。新块 B 到来时：

$$m' = \max\big(m, \max_{j \in B} s_j\big), \qquad a = e^{m-m'},$$

$$\widetilde{p}_j = e^{s_j - m'}, \qquad l' = al + \sum_{j \in B} \widetilde{p}_j,$$

$$h' = ah + \sum_{j \in B} \widetilde{p}_j v_j, \qquad O = h/l \ \text{（处理全部块后）}.$$

初值为 $m=-\infty, l=0, h=0$；全被 mask 的行需按实现单独处理。旧统计乘 a，是将过去的指数权重换到新的公共基准。在实数算术下，这与一次性 softmax 相同；浮点实现仍有舍入和归约顺序差异。低比特 attention 进一步近似其中的分数乘法与加权分子。[85, 137]

内存算例。当 $N=65536$，单个 head 的 N^2 矩阵以 16 位保存约需 8GiB；16 个 head 共约 128GiB。在线算法避免持久化这个矩阵，但 dense 的两次主要矩阵乘法仍约需 $4N^2d$ FLOPs/head。节省中间存储、减少连接、降低乘法精度分别改变不同成本项。

10.3 量化的对象、数制与误差预算

视频 DiT 的低精度优化有三个不同入口：线性层的权重与激活、attention 内部的两个矩阵乘法，以及长期保存的 KV。先写出计算，再决定压缩谁。

$$X \xrightarrow{W_Q,W_K,W_V} (Q,K,V), \quad S=QK^\top/\sqrt{d}, \quad P=\mathrm{softmax}(S), \quad O=PV.$$

$X\in\mathbb{R}^{N\times D}$ 是 token 特征；每个 attention head 的 $Q,K,V\in\mathbb{R}^{N\times d}$。投影矩阵 W_Q 是模型参数，Q 是本次前向产生的激活。量化 W_Q 与量化 $QK^\top$ 可以独立选择；W4A16 权重模型也可以搭配 INT8 attention。

位置	常见低精度对象	实际减少的成本
Linear / MLP	W 与输入激活 X	权重存储、GEMM 访存和乘法
Attention kernel	$Q,K,\widetilde{P},V$	$QK^\top$ 与 PV 的矩阵乘法
历史 KV	持久化 K,V 与 scale	显存、缓存读带宽和跨设备传输
通信	all-to-all 等传输的激活	网络字节数；另付编解码成本

$\widetilde{P}$ 表示在线 softmax 中尚未最终行归一化的分块指数权重。它与最终概率 P 的数值范围、累加方式需要分别处理。

INT8 / INT4：格点间距与截断

对有符号对称 b 位量化，令 $q_{\max}=2^{b-1}-1$，采用

$$s=\max|X|/q_{\max}, \qquad q=\mathrm{clip}(\mathrm{round}(X/s),-q_{\max},q_{\max}), \qquad \hat{X}=sq.$$

未截断时，每个数的绝对误差不超过 $s/2$；截断时还会增加超范围误差。INT4 采用 $[-7,7]$ 时，一个组里若出现 70，scale 为 10，数值 0.8 和 2.1 都可能变成 0。减小分组能隔离 outlier，但要存储更多 scale 并执行更多缩放。

以 $Y=XW$ 为例，若 X 按行、W 按输出列缩放，则

$$\hat{Y}_{ij}=s_i^X s_j^W \sum_k q_{ik}^X q_{kj}^W.$$

两个 scale 可以放在内积之外。若 scale 随归约维度 k 变化，则需要分组部分和或支持 block scaling 的硬件，无法用一个最终乘数完成还原。这解释了量化粒度为什么要和 Tensor Core 的 tile 布局一起设计。

FP8 / FP4：数值格式、缩放格式与累加器

浮点数用符号、指数和尾数分配动态范围。FP8 E4M3 与 E5M2 的范围和精度不同；还应写清 E4M3 的具体变体。FP4 E2M1 的格点非常稀疏，需要更细分组的缩放。

配置	数据与 scale	使用时的关键点
INT8	整数格点；浮点 scale	Tensor Core 指令、归约维缩放
FP8	E4M3 / E5M2；张量或块 scale	饱和、动态 amax、累加精度
NVFP4	E2M1，常见 16 元素块与 E4M3 scale	更细粒度；对应硬件支持
MXFP4	E2M1，32 元素块与 E8M0 scale	二次幂 scale；精度与吞吐权衡

两种 FP4 格式的描述对应 SageAttention3 所用的块缩放接口。实际库还可能叠加全局 scale。输入是 **4/8** 位、内部累加是 **16/32** 位、输出存储是 **BF16/FP16**，三者可以不同。SageAttention2++ 的重要改进正发生在累加器上。[86, 128, 138]

PTQ、QAT 与校准样本

PTQ 固定或校准已有模型，在代表性输入上选择 scale、裁剪阈值、混合精度和补偿项。QAT 在前向插入量化影响，再优化参数适应这一计算路径。常见 fake quantization 为

$$\hat{x} = s\,\mathrm{round}(x/s), \qquad \frac{\partial \hat{x}}{\partial x} \approx 1$$

（截断区间内的 straight-through 近似）。真实梯度不存在处用了代理梯度；训练后还要调用低比特 kernel 才能获得速度收益。在 BF16 中执行 round 后继续 BF16 GEMM，只验证量化误差。

视频校准集要覆盖：采样早/中/晚期、T2V/I2V/R2V、CFG 两分支、分辨率、运动幅度、不同 chunk 和 prompt 切换。每个时间点只有一张静态图的校准集容易漏掉时序误差。高敏感层可保留高精度；时间条件、归一化、输出头是否保留由测量决定。

误差怎样进入完整去噪轨迹

设真实速度为 v，量化速度为 $v + e_i$，反向 Euler 步长大小为 $h_i > 0$。令 $\delta_i = \hat{z}_i - z_i$，一阶展开得到

$$\delta_{i+1} \approx (I - h_i J_i)\delta_i - h_i e_i, \qquad J_i = \partial_z v(z_i, t_i).$$

早期误差会经后续 Jacobian 连乘传播。只报告单层 cosine similarity，无法推出最终视频质量。校准既可最小化层输出误差，也可比较相同 seed 的完整轨迹与最终视频。长视频还要检查错误通过历史 KV 继续传播的效应。

Roofline：字节少一半为什么未必快两倍

对一个算子，以 FLOPs 数 F、实际内存流量 B、硬件有效算力 C 与带宽 $\mathcal{B}$ 建立粗略下界：

$$T_{\mathrm{kernel}} \gtrsim \max(F/C,\ B/\mathcal{B}), \qquad I = F/B.$$

低精度可能同时降低 B 并提高 C，但完整耗时还包含量化、布局转换、scale reduction、路由和 launch。大 batch 的大 GEMM 可能算力受限；小矩阵可能受访存或启动开销限制。8 位权重的存储接近 16 位的一半，并不意味着整个模型显存减半：activation、KV、VAE、workspace、scale 和高精度副本都要计入。

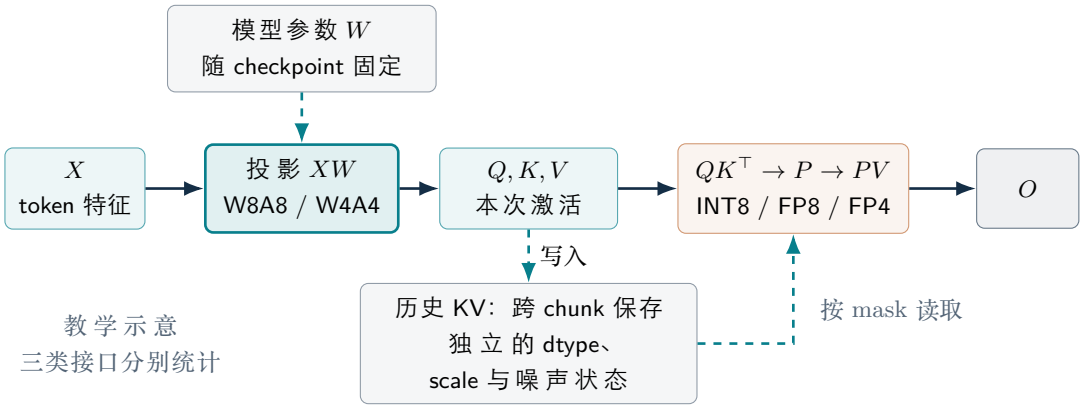


图 10.1　低精度的三个接口。权重、attention 激活和历史缓存有不同的生命周期与误差来源。**教学示意。**箭头是本次前向的数据流；持久缓存单列。测量时分别记录数据类型、累加器、scale 粒度与最终视频质量。[136, 144, 146]

10.4　线性层量化：ViDiT-Q、SVDQuant 与时间敏感配置

ViDiT-Q：激活分布随 token、条件和时间改变

ViDiT-Q 针对 DiT 的动态分布，结合细粒度激活量化、通道平衡和时间相关处理。混合精度扩展进一步区分敏感层与敏感时间。可把通道平衡理解为精确重参数化：对可逆对角矩阵 D，

$$XW = (XD^{-1})(DW).$$

将激活的大通道缩小，会把幅值转移到权重；分别量化重参数化后的两端，再寻找共同误差较小的配置。论文包含 W8A8 与更激进的 W4A8 实验；其 kernel 和端到端实现另外发布在官方仓库。[144, 145]

SVDQuant：高精度低秩支路吸收难量化部分

在平滑后的权重中取低秩分量 L_1L_2，其余部分 R 用 4 位表示：

$$W \approx L_1L_2 + \hat{R}, \qquad XW \approx (XL_1)L_2 + \mathrm{GEMM}_{4\mathrm{bit}}(\hat{X}, \hat{R}).$$

L_1, L_2 的秩远小于完整通道维度。SVD 选择的高精度支路承担部分 outlier，降低残差和激活的量化难度。Nunchaku 把低秩与低比特支路的读写融合；分开运行支路会增加 activation 访存，可能吃掉加速。论文的主要实验为 SDXL、PixArt-Σ 与 FLUX.1；将方法迁移到视频时，要重新验证时间条件、attention 占比和 VAE 成本。[146]

AdaTSQ：按去噪阶段分配位宽

AdaTSQ（2026）把时间步/层的混合精度选择组织成有预算的搜索，并利用时间敏感性安排校准样本。一个通用的表达是

$$\min_{b_{\ell,t}} E_{\mathrm{trajectory}}(\{b_{\ell,t}\}) \quad \text{s.t.} \quad \sum_t C_t(\{b_{\ell,t}\}) \le C_{\max}.$$

这是理解预算分配的示意目标。论文使用 Pareto 搜索和 Fisher 引导的校准。动态位宽在工程中还需付出权重格式切换、kernel 选择和多份权重驻留成本；最终策略应以真实 latency 为约束复核。[147]

10.5　SageAttention 系列：从 INT8 到 FP4

这一系列加速 attention 内部的矩阵乘法，保留 softmax 的核心结构。它通常接在 Q/K/V 投影、位置编码和 QK-Norm 之后。下面按论文算法区分版本，库的版本号与论文名字分别记录。

SageAttention 1：K-smoothing 与 INT8 $QK^\top$

先把 K 沿 token 轴求均值 $\bar{k} \in \mathbb{R}^d$。对于每个 query q_i，

$$q_i^\top (k_j - \bar{k}) = q_i^\top k_j - q_i^\top \bar{k}.$$

最后一项对这一行所有 j 相同。由于 $\mathrm{softmax}(s - a\mathbf{1}) = \mathrm{softmax}(s)$，减去全局 K 均值在精确运算下保持 attention 概率不变，同时压低部分公共 outlier。随后按块量化 Q/K，以 INT8 Tensor Core 计算分数；softmax 和缩放在较高精度执行，PV 采用 FP16 路径。[136]

举例：$k_1 = (100, 1), k_2 = (102, -1)$，减去均值后成为 $(-1, 1), (1, -1)$。共同的大偏置消失，INT8 的格点可更多用于表示两者差异。这种处理依赖 softmax 的行平移不变性；任意归一化轴的减均值没有同样结论。

SageAttention2：Q/K/V 平滑、INT4 与两级累加

将 query tile Q_i 减去块均值 $\bar{q}_i$，K 减去全局均值。记 $Q'_i = Q_i - \mathbf{1}\bar{q}_i$、$K' = K - \mathbf{1}\bar{k}$，则忽略 softmax 不敏感的行常量后，分数等价于

$$S_i \equiv \left[Q'_i K'^\top + \mathbf{1}(\bar{q}_i K'^\top)\right]/\sqrt{d}.$$

第二项随 key 变化，必须加回。单纯把 Q 减均值后直接 attention 会改变模型；补偿向量可用小 GEMV 求出。

论文用 INT4 计算 $QK^\top$，并将 $\widetilde{P}, V$ 量化为 FP8 计算 PV。V 的均值也可拆出：若 $V' = V - \mathbf{1}\bar{v}$，由 $P\mathbf{1} = \mathbf{1}$ 得 $PV = PV' + \mathbf{1}\bar{v}$。**这些等价式先保证预处理正确，再讨论低比特舍入误差。**[137]

Per-thread 的含义。GPU 的一个 warp 有 32 个线程，MMA 指令把矩阵元素按规定方式分配给线程。量化组与该布局对齐，每个线程的还原操作只处理很少的 scale；粒度比大块量化更细，同时控制 scale 读取与反量化成本。它描述硬件分组，不表示“每个模型 token 单独开一条 GPU 线程”。

两级累加。某些 FP8 指令的内部精度或累加范围不足以支撑很长的总和。先在快速累加器中完成短部分和，再定期转存高精度缓冲区，可限制误差随块数增长。分组越短，精度较好而转换更频繁。

SageAttention2++：更换 PV 指令与限制累加范围

2++ 保留 2 的平滑和量化思想，将 PV 换到 FP8 输入、FP16 累加的更快 MMA 路径。FP16 最大有限值为 65504；一次长度 32 的乘加要避免溢出，可施加保守条件

$$32 P_r V_r \le 65504.$$

若连续两次 MMA 才转入 FP32，则要求 $64 P_r V_r \le 65504$。论文示例 $P_r = 112, V_r = 4.5$，对应 $64 \times 112 \times 4.5 = 32256$。同时缩窄两端的量化范围并保留动态 scale，可以换取更快指令和更少 FP32 转换。[138]

这是对**量化后数值**的范围约束；scale 还原发生在配套位置。把网络张量简单 cast 为 FP8，再任意改累加 dtype，缺少这套约束与补偿。

SageAttention3：Blackwell 上的 microscaling FP4

SageAttention3 在两次矩阵乘法中使用块缩放 FP4。其 NVFP4 方案以 16 元素为组，数据为 E2M1，组 scale 采用 E4M3；论文同时比较了 32 元素组、E8M0 scale 的 MXFP4。[128]

$$s_g \approx \max_{j \in g} |x_j|/6, \qquad \hat{x}_j = \mathrm{round}_{\mathrm{E2M1}}(x_j / s_g), \quad x_j \approx s_g \hat{x}_j.$$

scale 自身也会舍入，所以真实误差包含格点误差和 scale 误差。特别是在线 softmax 中 $\widetilde{P}$ 经常很小；它的微缩 scale 可能落到 FP8 scale 格式难以精确表示的区域。方法引入二级缩放改善 scale 的有效范围，并继续采用 Q/K 平滑与硬件融合。softmax、统计量和关键累加仍需要适当的高精度。

FP4 提供硬件吞吐潜力，精度敏感任务仍需对照 2/2++。官方仓库也给出相应精度取舍提示。部署前核对 GPU 架构、CUDA 版本、head dimension、layout 与所调用的具体 kernel。[140]

系列对照与发布性能的读法

论文分支	主要变化	应记录的实验条件
SageAttention 1	INT8 QK、K 平滑、FP16 PV	3090/4090、head dim、预处理成本
SageAttention2	INT4 QK、FP8 PV、QKV 平滑	论文算法与库默认 INT8 路径分别记录
SageAttention2++	FP8 PV 的 FP16 累加与范围约束	具体 MMA、缓冲频率、GPU 架构
SageAttention3	microscaling FP4、scale 精度处理	Blackwell、NVFP4 路径、质量变化
SageBwd	低比特前向与反向	微调/预训练、tokens per step

截至 2026-10-18，本节覆盖官方公开的 1、2、2++、3 及 SageBwd 分支；例如软件包 2.2.0 包含 2++，版本号不能直接当作论文代数。[140]

原论文中的代表性 **kernel** 结果：1 在 RTX4090 上约为 FA2 的 2.1 倍；2 在其 RTX4090 设置中约 3 倍；2++ 的 INT4-QK/FP8-PV 版本在 RTX4090 上约为 FA2 的 3.9 倍；3 在 RTX5090 的指定设置中达到约 1038 TOPS。这些结果来自不同 shape、精度和实现，组成一张版本谱系，而非统一环境的排名。比较视频端到端收益时，必须另外计入投影、MLP、VAE 与量化预处理。[136, 137, 138, 128]

接口落地：选到哪个 kernel 必须可追踪

官方 `sageattention/core.py` 的 `sageattn` 根据 GPU 架构分派；可见分支包括 INT8 QK + FP16 PV，以及 INT8 QK + FP8 PV。因此调用函数名本身无法证明本次运行使用 INT4 或 FP4。[140]

```
from sageattention import sageattn
# q,k,v: [batch, heads, tokens, head_dim], CUDA FP16/BF16
out = sageattn(q, k, v, tensor_layout="HND", is_causal=False)
```

接入视频模型时先检查自定义 mask。`is_causal=True` 的普通三角 mask 与 chunk 内双向、chunk 间因果的 mask 不同；原模型还有参考帧、文本、音频等区域，不能直接互换。测试 GQA、变长输入、RoPE 和 QK-Norm 的位置，再进行短 clip 与长时 rollout 回归。官方 README 对 `torch.compile` 与 CUDA Graph 的支持分别描述；某个包可编译，不代表所有分派路径都能直接捕获。

10.6 低精度训练：SageBwd 与 Attn-QAT

反向还要计算哪些矩阵乘法

设上游梯度 $G = \partial L/\partial O$，对单 head：

$$\partial_V L = P^\top G, \qquad \partial_P L = G V^\top,$$
$$\partial_S L = P \odot \left[\partial_P L - \mathrm{rowsum}(P \odot \partial_P L)\right],$$
$$\partial_Q L = (\partial_S L) K/\sqrt{d}, \qquad \partial_K L = (\partial_S L)^\top Q/\sqrt{d}.$$

FlashAttention 风格反向会重算 $QK^\top$ 与 softmax，以节省保存 $N \times N$ 中间量的显存。因此前向两次、反向五次，共七次主要矩阵乘法。SageBwd 将其中六次量化，保留 $GV^\top$ 的较高精度，因为其误差直接进入小量相减的 score 梯度。[139, 128]

2026 年 SageBwd 后续研究把预训练偏差定位到 $\partial_S L$，分析 QK-Norm、K 平滑与 tokens per step 的作用。较大训练 batch 会改变真实梯度与量化噪声的相对尺度；减小 tokens per step 在其实验中帮助恢复全精度训练表现。推理 attention 指标稳定，只能支持前向近似的判断，训练还要观察梯度方向、loss 曲线和最终生成能力。

Attn-QAT：低比特前向与反向重算要一致

Attn-QAT（2026）分析“FP4 前向 + 高精度重算反向”产生的失配。若前向采用 $\hat{P}$，反向却用由原始 Q/K 重算的 P，反传对象发生了变化；FlashAttention 中利用 O 推导的梯度恒等式也有精度前提。它匹配低精度分数重算并修复梯度计算，使 QAT 学到实际部署路径。[148]

可采用两级验证：先对很小的 attention 检查高精度解析梯度与自动微分；再比较低精度 surrogate 梯度与同一低精度前向的参考实现。对 round 使用 STE 时，有限差分检验的是不同对象，不能把二者直接要求为相等。

10.7 Sparse-Linear Attention：稀疏与低秩的组合

先区分 sparse 与 linear

稀疏 attention 保留部分 query-key 连接；linear attention 则改写特征映射与乘法顺序。以非负特征映射 $\phi : \mathbb{R}^d \to \mathbb{R}^r$ 为例，

$$H = \phi(K)^\top V \in \mathbb{R}^{r\times d}, \quad u = \phi(K)^\top \mathbf{1} \in \mathbb{R}^r, \quad O_i = \frac{\phi(q_i)^\top H}{\phi(q_i)^\top u + \varepsilon}.$$

先计算小矩阵 H，无需构造完整 $N \times N$ 概率矩阵；当 r, d 固定时，主成本随 N 线性增长。代价是表达形式改变：$\phi(Q)\phi(K)^\top$ 的秩受 r 限制，无法任意表达高秩、尖锐的匹配关系。简单地将 softmax attention 换成这一公式，通常需要训练适配。[141]

SLA 的观察：尖峰少，但平缓部分的总质量不可忽略

对 full attention P 选一个 mask M，存在恒等分解

$$P = P \odot M + P \odot (1 - M).$$

SLA 观察到其 DiT 实验里，少量大权重部分仍具有较高秩；大量较小权重的剩余部分更接近低秩。小权重逐项虽小，累加后仍会影响输出。于是把块分成三类：重要块做稀疏 softmax，次重要块做线性补偿，极小块跳过。[141]

从 Q/K 到可执行的分块路由

将 token 按 b_q, b_k 个一组池化，得到低分辨率分数：

$$P_c = \mathrm{softmax}\Big(\mathrm{pool}(Q)\,\mathrm{pool}(K)^\top/\sqrt{d}\Big).$$

每个 query 块取高分 top 部分为重要集合 $\mathcal{S}_i$，中间部分为 $\mathcal{L}_i$。稀疏支路在 $\mathcal{S}_i$ 内进行在线 softmax；线性支路预计算每个 key 块的

$$H_j = \phi(K_j)^\top V_j, \quad u_j = \phi(K_j)^\top \mathbf{1}, \quad H_i^L = \sum_{j\in\mathcal{L}_i} H_j, \quad u_i^L = \sum_{j\in\mathcal{L}_i} u_j.$$

之后 $O_i^L = \phi(Q_i)H_i^L/(\phi(Q_i)u_i^L)$。原 SLA 将两路组合为

$$O = O^S + \mathrm{Proj}(O^L).$$

可训练的 projection 补偿两路的分布差异；替换 attention 后继续用生成模型目标微调。实现预计算块统计并融合相应 kernel，避免逐连接做标量稀疏运算。[142, 141]

复杂度口径。保留比例 ρ 的 softmax 部分仍约为 $O(\rho N^2 d)$；池化路由约为 $O(N^2 d/(b_q b_k))$；特征映射乘法约为 $O(Nrd)$，还包含按 query 块聚合统计的成本。固定 ρ 时整个算子仍含二次项，“linear”指补偿支路的结构。95% 稀疏只说明少算了多少 full-attention 连接，仍有路由、归约、投影和不规则访存。

SLA2：两路各自归一化后，为什么不能直接相加

这是一个容易漏掉的数学细节。令 $\alpha_i = \sum_{j:M_{ij}=1} P_{ij}$ 为 full attention 放在稀疏集合内的概率质量。稀疏 softmax 对选中位置重新归一化，得到 $P_{ij}^S = P_{ij}/\alpha_i$。精确输出可写为

$$O_i = \alpha_i O_i^S + (1-\alpha_i)O_i^R,$$

其中 O^R 是剩余位置**单独归一化**的输出。假如 $\alpha = 0.8$、$O^S = 10$、$O^R = 2$，原输出是 8.4；直接相加得到 12，放大了幅值。SLA 的 projection 需要同时补偿这个偏差与线性近似误差。

SLA2 使用可学习比例与路由：

$$O = \alpha \odot O^S + (1-\alpha) \odot O^L, \quad 0 \le \alpha \le 1,$$

$$R_c = \mathrm{Proj}_q(\mathrm{pool}\, Q)\,\mathrm{Proj}_k(\mathrm{pool}\, K)^\top, \qquad M_c = \mathrm{TopK}(R_c).$$

学习到的 α 估计所需混合比例，推理无需先算 full attention 的真实质量。构造稀疏 softmax 时，被排除的 logits 使用 $-\infty$ 或直接跳块；把它们乘成 0 后做普通 softmax 会给它们分配非零概率。[143]

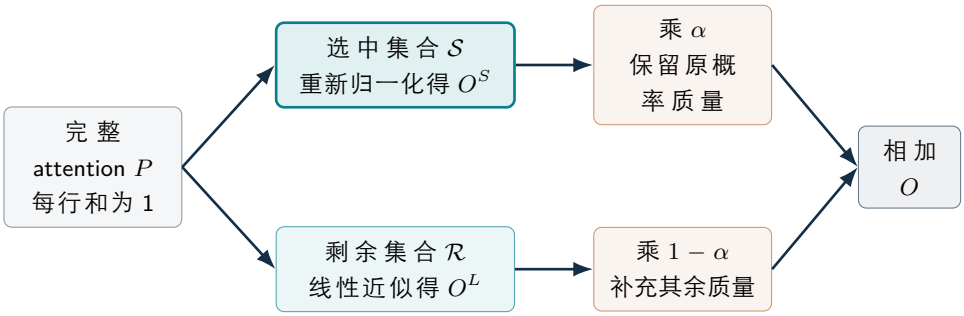
两阶段训练、QAT 与边界

第一阶段采集不同层、时间步的 Q/K/V，用 full-attention 输出 MSE 初始化 router 与混合比例；TopK 采用可导的 soft 近似，例如

$$\widetilde{M}_{ij} = \sigma(R_{ij}/\tau + \lambda_i), \qquad \sum_j \widetilde{M}_{ij} = k_i.$$

λ_i 控制该行预算。第二阶段冻结 router 参数，以端到端 diffusion loss 微调主干和比例，同时适配低比特前向。router 权重固定后，其输出仍随当前 Q/K 改变。[143]

SLA2 在 RTX5090 上报告约 18.6 倍 attention 加速；Wan-1.3B-480P 的 attention 时间从 97s 降到 7s，完整计算链约加速 2.30 倍。14B-720P 配置报告约 4.35 倍，**该表排除了 CPU offloading 耗时**，部署墙钟时间需把传输加回。由此可直接看到“稀疏比例、kernel 倍数、生成倍数”是三个不同量。



教学算例：$\alpha = 0.8$，$O^S = 10$，$O^L = 2$，加权输出为 8.4；直接相加为 12。
SLA 用投影补偿；SLA2 显式学习两路混合比例。

图 10.2 SLA 与 SLA2 的归一化。稀疏集合与剩余集合各自归一化，组合时还需要概率质量权重。教学示意。图中的 α 为解析说明；SLA2 在运行中学习其近似，并使用低成本路由。[141, 143]

10.8 相关稀疏路线与组合误差

路线	核心选择	训练与适用重点
STA	3D 局部窗口按硬件 tile 对齐	可免训练使用；微调支持更激进预算
SpargeAttn	在线压缩预测 + softmax 感知过滤	免训练稀疏，与 Sage 低比特结合
VSA / VSA2	粗到细、内容相关 router	可训练；全训练流程使用稀疏算子
SpargeAttention2	Top-k + Top-p，蒸馏微调	更高稀疏率下约束函数偏差
FPSAttention	FP8 与 3D tile 稀疏联合设计	训练适配两种近似的组合
SLA / SLA2	稀疏精算 + 线性补偿	为剩余连接保留低成本信息通道

STA：对齐 tile 比写一个局部 mask 更重要

传统逐 token 滑窗可能让一个 tile 中只有少量连接有效，硬件仍要加载和处理部分无效块。Sliding Tile Attention 将空间时间邻域与 block layout 对齐，尽量形成全算或全跳的块。窗口大小与 head/layer 的依赖范围共同影响运动与长程关系。原论文在 HunyuanVideo 中比较无需训练与微调后的速度/质量，二者代表不同工作预算。[149]

SpargeAttn 与 SpargeAttention2

SpargeAttn 先用压缩 Q/K 与相似性检查预测可跳的块，再在在线 softmax 阶段过滤贡献极小的乘法。若块内 token 差异很大，均值代表性会下降，需要保守处理。它把 sparse 与 quantized attention 放在同一实现中。[150]

SpargeAttention2（2026）增加训练。Top-k 固定连接数，可能漏掉分布很平坦的一行；Top-p 按累积质量保留，可能在尖锐行中给出过小预算。混合规则调节这两种风险，配合蒸馏式微调保持原模型函数行为。它报告 95% 稀疏与约 16.2 倍 attention 加速；实际收益还需观察不同时间步选中多少块、route overhead 与整体质量。[151]

VSA 到 VSA2：训练过程中的稀疏化

VSA 采用池化后的粗粒度重要性估计，再在选定 tile 内做细粒度 attention，提供可训练的硬件友好实现。VSA2（2026-09）进一步细化 router，允许不同 query 使用不同的连接预算，并研究从高稀疏训练到较低稀疏推理的课程，以及在渐进分辨率预训练中把 dense checkpoint 转为稀疏模型。它解决的是如何把稀疏计算纳入训练配方，而非只在最后替换一次推理算子。[152, 153]

FPSAttention：稀疏和量化一起训练

设 full 输出为 O，稀疏输出为 O_S，进一步量化后为 $\hat{O}_S$，则

$$\hat{O}_S - O = (O_S - O) + (\hat{O}_S - O_S).$$

两项通常相关，无法假设误差相互抵消。FPSAttention 围绕视频 3D 双向 attention 联合设计 FP8、稀疏 tile 与训练适配；SLA2 也通过 QAT 适配联合路径。评测应包含 dense-BF16、仅 sparse、仅 quant、sparse+quant 四组，固定 seed、scheduler 和比较数据。[154, 143]

稀疏率高但跑得慢的原因

路由需要先读 Q/K；TopK 与重排有额外 kernel；变长行会产生负载不均；太小的 tile 会降低 Tensor Core 利用率。令

$$T_{\text{sparse}} = T_{\text{route}} + T_{\text{pack}} + T_{\text{selected tiles}} + T_{\text{merge}},$$

只有右侧总和低于 dense 路径，才得到运行时加速。固定块数而改变索引，通常比每行动态分配内存更易结合 CUDA Graph；具体拓扑和缓冲区限制见下一节。

10.9 算子融合、编译与 CUDA Graph 的执行模型

GPU 运算的成本来自几层：Python 与框架调度、驱动提交、设备调度、实际 kernel、显存读写与同步。减少 FLOPs 只覆盖其中一部分。

Kernel fusion 与图回放优化不同层次

对 $y = \mathrm{SiLU}(x + b) \odot g$，分开执行会读写多个中间张量；融合可让中间结果留在寄存器，减少 HBM 流量和 kernel 次数。融合后寄存器用量可能上升，降低 occupancy；因此“少一个 kernel”仍需测实际时间。[87]

CUDA Graph 把一组 GPU 操作、内存地址和依赖构成有向无环图。capture 记录这次工作，instantiate 建立可执行图，replay 重复提交。图中仍有多个 kernel；模型参数和运算目标保持不变。Python 循环在 capture 时运行一次，replay 时执行的是记录好的 GPU 工作。

CPU 提交与 GPU 执行为什么不能简单相加

考虑单 stream 的 m 个相依 kernel。第 i 个 kernel 提交完成时刻为 a_i，GPU 执行耗时为 g_i。忽略其他干扰时，完成时刻满足

$$a_i = a_{i-1} + \ell_i, \qquad c_i = \max(a_i, c_{i-1}) + g_i.$$

ℓ_i 是 CPU 提交间隔。如果 ℓ_i 很小，CPU 可以提前排队，GPU 连续工作；如果 kernel 很短而 ℓ_i 很大，GPU 会等 CPU，时间线出现空洞。均匀情形近似为

$$T_{\text{eager}} \approx \ell + g + (m-1)\max(\ell, g).$$

图回放减少提交数量并压缩部分设备调度间隙：

$$T_{\text{graph}} \approx L_G + \sum_i g_i + \sum_i \delta_i.$$

L_G 是图提交开销，δ_i 为剩余调度间隙；这里按单链近似，多 stream 图还取决于关键路径和资源竞争。[155, 156]

教学算例。若 $m = 100, \ell = 5\,\mu s, g = 2\,\mu s$，上述 eager 约为 $502\,\mu s$；图若 $L_G = 3\,\mu s$ 且忽略间隙，约为 $203\,\mu s$。若 $g = 200\,\mu s$，则两者都接近 20ms，去掉几十/几百微秒提交开销的比例就小得多。算例由假设计算，实际测量见后表。

Capture 的核心约束：地址稳定，值可以变化

图回放通常复用 capture 时的 GPU 地址与 shape。固定地址的张量可以写入新值；Python 名称重新绑定到一个新张量，不会让旧图自动换地址。

```
model.eval().requires_grad_(False)  # inference-only example
static_x = example_x.detach().clone()
static_t = example_t.detach().clone()
# warm up model and allocations on a side stream first
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for _ in range(3):
        _ = model(static_x, static_t)
torch.cuda.current_stream().wait_stream(s)
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    static_y = model(static_x, static_t)
# keep static_x, static_t, static_y and g alive
static_x.copy_(new_x)
static_t.copy_(new_t)
g.replay()
# static_y will be overwritten on the next replay
result = static_y.clone()  # only when an independent copy is needed
```

上例用有效输入初始化固定缓冲区，并关闭参数梯度。完整附带脚本给出初始化、同步、错误检查和计时。输出 clone 也消耗时间与显存，计入返回独立输出的路径。[157]

容易捕获错误的状态

- **时间步**：捕获时读取 Python 标量 $t = 0.5$，replay 只会复用该值对应的工作。将 t 作为固定 GPU tensor 的内容更新，或为不同静态路径建图。
- **数据依赖控制流**：`tensor.item()`、CPU 同步和 Python 条件分支无法随 replay 自由重走。支持图的设备端控制能力需由特定 API 表达；普通 Python 分支不会自动转换。
- **随机数**：支持 capture 的 PyTorch CUDA RNG 可管理 replay 的状态推进。外部随机源或不兼容 generator 需单独检查；可将新噪声在图外产生并写入固定缓冲区。不要把 capture 的第一次噪声当成每次都应复用的值。
- **内存池**：图的中间张量有专用生命周期；缓存很多 shape 的图会保留大量内存。输入、输出与图对象必须保持存活。
- **多 stream 与 collective**：需要明确依赖和汇合；所有 rank 的通信次序必须一致，避免一部分走 graph、一部分走 eager 的不匹配。

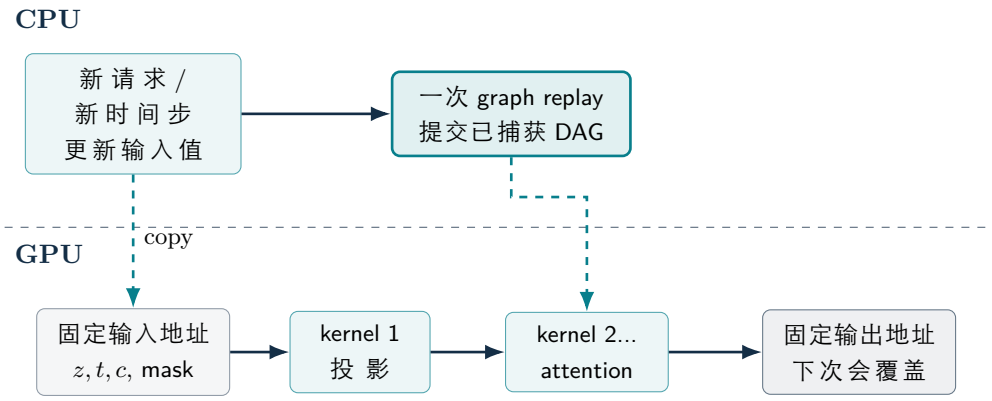
上述约束针对 PyTorch capture/replay 的常用接口；较底层 CUDA 支持部分图更新机制，仍需要满足 API 的更新限制。[157, 156]

视频 DiT 里捕获哪一层最合适

捕获单位	优点	需要固定或显式输入的状态
一个 Transformer block	易隔离、适配动态上层	shape、workspace、attention 接口
一次完整 denoiser	大幅减少重复 launch	z_t, t, c、CFG batch、mask、cache
整条少步采样链	进一步减少循环提交	步数、scheduler、噪声、输出存储
一个流式 chunk	可把去噪与 cache 更新联合捕获	cache 容量、写指针、位置、有效长度

分辨率、帧数、文本长度、dtype、CFG 分支组织和 cache 容量构成图缓存键。可按常见 shape 分桶并 padding；固定形状 TopK 索引张量的值可变，输出大小随 Top-p 改变则需要容量上界、有效长度或局部 eager 路径。动态图不一定完全放弃 Graph，关键是把变化压缩成固定缓冲区中的数据。

对流式 KV，使用固定容量 ring buffer，并把写位置、有效长度、RoPE 位置作为设备数据或受控图参数。每次 `torch.cat` 扩展 KV 会改变地址和长度，既不利于 capture，也使长期分配成本增长。切换 prompt 后，重新编码条件与重建 cache 的阶段可以留在图外，再回到稳定的去噪图。



机制示意：输入值可变，地址、shape 与依赖保持可复用。
图中仍有多个独立 kernel；CPU 用一次 replay 提交。

图 10.3　图回放复用地址与依赖。CPU 更新输入缓冲区，再提交固定的 GPU 运算图；值随请求变化。**执行机制示意，非 profiler 实测图。**箭头表示依赖，图内方框仍是独立 kernel。输出在下次 replay 时会被覆盖。[157, 88]

10.10 CUDA Graph：公开实机结果与可复现实验

官方数据分别证明什么

来源与机器	原始设置与数据	能读出的结论
NVIDIA 入门实验，V100	20 个短 kernel/步，1000 步；逐 kernel 同步约 9.6μs/个，移出同步约 3.8，Graph 约 3.4	先去掉无必要同步，再评估 Graph；9.6 到 3.4 包含同步改进
NVIDIA 2024 实验，RTX3060 + Xeon Silver 4208	CUDA 11.8 / 12.6；100 节点直链图，repeat 端到端 69 / 55μs	对比两个 CUDA 版本的图实现，非 eager 与 Graph 对比
同一 RTX3060 实验	100 节点实例化 168 / 127μs；repeat 设备执行 61 / 53μs	实例化成本、设备时间与墙钟需要分列

第一项来自 NVIDIA 的教学 microbenchmark；第二、三项来自 CUDA 12.6 性能博客。后者还报告 Ampere 上 10 节点以上直链图的重复 CPU launch 开销接近 $2.5\,\mu s + 1\,ns \times N$。这里的近常数项只描述该图拓扑和软件环境的 CPU 提交；GPU 仍要执行所有工作。[155, 156]

捕获成本何时摊平

设预热、捕获与实例化额外成本为 C，每次 eager 与 replay 的墙钟分别为 T_E, T_G。当 $T_E > T_G$ 时，至少需要约

$$R > C/(T_E - T_G)$$

次复用才能摊平。若建图用 120ms、每次省 0.3ms，约 400 次后回本。冷门分辨率只运行一次时，现建图可能更慢。在线服务要同时报告冷启动、热请求和图缓存命中率。

随书实验脚本：两种负载、两类时间

源目录 `experiments/bench_cuda_graph.py` 提供两种合成工作负载：

- `pointwise`：重复小逐元素 kernel，用来观察 CPU launch 和 kernel fusion 的影响。
- `attention`：以 $[B, H, N, d]$ Q/K/V 运行 SDPA 或显式 Sage 接口，比较输入 shape 对收益的影响。

它分别记录批量重复的同步墙钟、CUDA Events 流上经过时间、峰值显存、建图时间、输出误差和环境信息。Events 的起止间隔可包含 GPU 等待 CPU 提交的空洞；单个 kernel 的纯执行时长另从 profiler 读取。默认 SDPA 由 PyTorch 分派；要求明确 Flash backend 时使用 `--flash-only`，不支持时直接报错，避免把 math fallback 当成 FlashAttention。

```
python experiments/bench_cuda_graph.py --mode pointwise \
  --n 4096 --depth 20 --repeats 200 --out pointwise.json
python experiments/bench_cuda_graph.py --mode attention \
  --n 4096 --heads 16 --dim 128 --repeats 50 \
  --flash-only --out attn.json
python experiments/bench_cuda_graph.py --mode attention \
  --backend sage --n 4096 --out sage.json
nsys profile --trace=cuda,nvtx,osrt -o graph_trace \
  python experiments/bench_cuda_graph.py --mode pointwise
```

脚本会把输入 copy 计入两条路径，保持比较对象一致。没有 CUDA 时输出 `status: skipped`；某个 Sage 路径 capture 失败时保留报错，不把它伪装成 successful replay。默认只测 microbenchmark，生成质量要回到真实模型评估。

实机记录表与 profiler 判读

至少记录 GPU/驱动、PyTorch/CUDA/Sage 版本、dtype、形状、mask、warm-up、迭代数、计时边界。再按顺序看：

1. **Nsight Systems**：CPU 提交与 GPU kernel 之间是否有空洞？graph replay 是否减少 CPU API 数和空洞？是否被 copy、同步或网络等待取代？
2. **Nsight Compute**：选代表性 GEMM/attention 查看 Tensor Core 利用率、DRAM 吞吐、寄存器与 occupancy。Graph 很少改变同一个 kernel 的数学计算量。
3. **真实模型四组**：eager、仅 compile/fusion、仅 Graph、compile+Graph；确认后者没有被自动编译模式重复启用或隐式禁用。
4. **流式回归**：测首 chunk、稳态每 chunk、prompt 切换、不同 cache 长度、并发尾延迟，检查静态缓存是否读到了旧数据。

当前编写环境为 Apple Silicon，未运行 NVIDIA GPU 性能测试。上表是标明机器与版本的公开实测；附带程序已做语法和无 CUDA 分支校验。本节算例为解析估算，GPU 实测结果由运行脚本后写入 JSON。

10.11 Context Parallel 与 Sequence Parallel

把长视频 token 分到多张卡，再通过通信完成跨分片 attention。

视频 token 数 N 很大。若按序列拆成 P 片，每张卡先持有约 N/P 个位置，局部 MLP / Norm 激活可以随之缩小。全局 attention 仍要让每个 query 访问所需的 K/V。

Ulysses：在序列切分与 head 切分之间转换

开始时每卡持有部分序列的所有 heads；all-to-all 后，每卡得到完整序列的一部分 heads。完成 attention，再用 all-to-all 把布局变回序列分片。[89]

$$[N/P, H, d_h] \quad \longleftrightarrow \quad [N, H/P, d_h].$$

这是整除情形的示意。head 数与并行度、网络互联带宽以及重排开销共同影响扩展效率。

Ring Attention：Q 留在本地，K/V 分块流过

各卡保留本地 Q，K/V 块沿环交换；收到一块就计算局部 attention 贡献，用在线 softmax 累积正确归一化。通信可与块计算重叠，最后得到完整 attention 输出。[90]

并行术语与实际张量布局

不同框架的术语粒度不同。Context Parallel 通常指上下文维度分片并跨卡完成 attention；Sequence Parallel 在一些实现中指相似长序列并行，在 Megatron 语境中也常指与 tensor parallel 配合，对 Norm / Dropout 等激活沿序列分片。实际阅读时直接画出**张量 shape 和 collective 通信**。[91]

加速比、并行效率与总费用

$$T_P \approx T_{\text{compute}}/P + T_{\text{comm}} + T_{\text{sync}} + T_{\text{imbalance}}, \quad S_P = T_1/T_P, \quad \eta_P = S_P/P.$$

例如 4 卡取得 3 倍加速，扩展效率为 75%。训练中可换取更长序列或更大 batch；推理中可降低单样本延迟，但 GPU-seconds 成本也要单独记录。

并行的第一层收益是“装得下”；第二层是“算得快”；经济性还需看每个视频消耗多少 GPU 时间。

10.12 新一代 attention kernel 与容量路由

FlashAttention-4：重排精确计算的硬件执行

FlashAttention 系列通过分块和在线 softmax 避免将完整注意力矩阵写入显存。FA4 针对 Blackwell 上矩阵计算、共享内存与非线性运算扩张不均的情况设计流水线，使用更大的 tile、异步执行和新的内存组织。论文的 BF16 B200 测试给出相对 cuDNN 9.13 最高约 1.3×、相对 Triton 约 2.7× 的 attention 加速；这是指定 kernel 的比较。[127]

TRACK：把大模型预算分配给敏感步骤

TRACK（2026 年 9 月）使用 latent 空间与 sampler 相容的大小 checkpoint，先在离线校准集沿全大模型轨迹计算两者预测差。用简化记号表示每步分歧：

$$d_i = \mathbb{E}_{\text{calib}} \frac{\|v_L(z_{t_i}) - v_S(z_{t_i})\|}{\|v_L(z_{t_i})\| + \varepsilon}.$$

按照分歧排名把固定的大模型调用预算分给敏感步骤，其余步调用小模型；推理每步只执行一个模型。该式概括相对分歧的校准思想，具体归约和归一化按原实现。[129]

这一方案保持步数，通过减少平均每步模型成本加速。它允许非连续切换，与单次永久交接不同。校准集之外的 prompt、CFG 和分辨率可能改变敏感步骤，因而要验证策略迁移；同时计入双模型常驻显存或权重交换开销。

10.13 TurboDiffusion、TurboServe 与服务调度

少做几步、每步更便宜、多人服务更高效，分别是三个层次的优化。

TurboDiffusion：模型内部的组合加速

其主要组件包括低比特 SageAttention、可训练 Sparse-Linear Attention、rCM 少步蒸馏和 W8A8 线性层量化。论文标题中的 100–200 倍来自整套组合相对给定基线的实验，具体配置包含 GPU、分辨率、步数和各加速组件。[50]

TurboServe：会话与集群调度

流式视频会话长期存活，用户有活跃/空闲状态，且每个 chunk 有时延要求。TurboServe 联合考虑会话放置和 GPU 数量，使用迁移感知调度、动态扩缩容、同 GPU 活跃会话的 chunk 合并处理，以及挂起/恢复的状态卸载。[51] 它报告在相应生产 trace 评测下，最坏 chunk 时延降低 37.5%，GPU 运行成本平均降低 37.2%。这两项分别衡量服务时延和集群成本，NFE 与单卡 kernel 性能属于另外的测量层次。

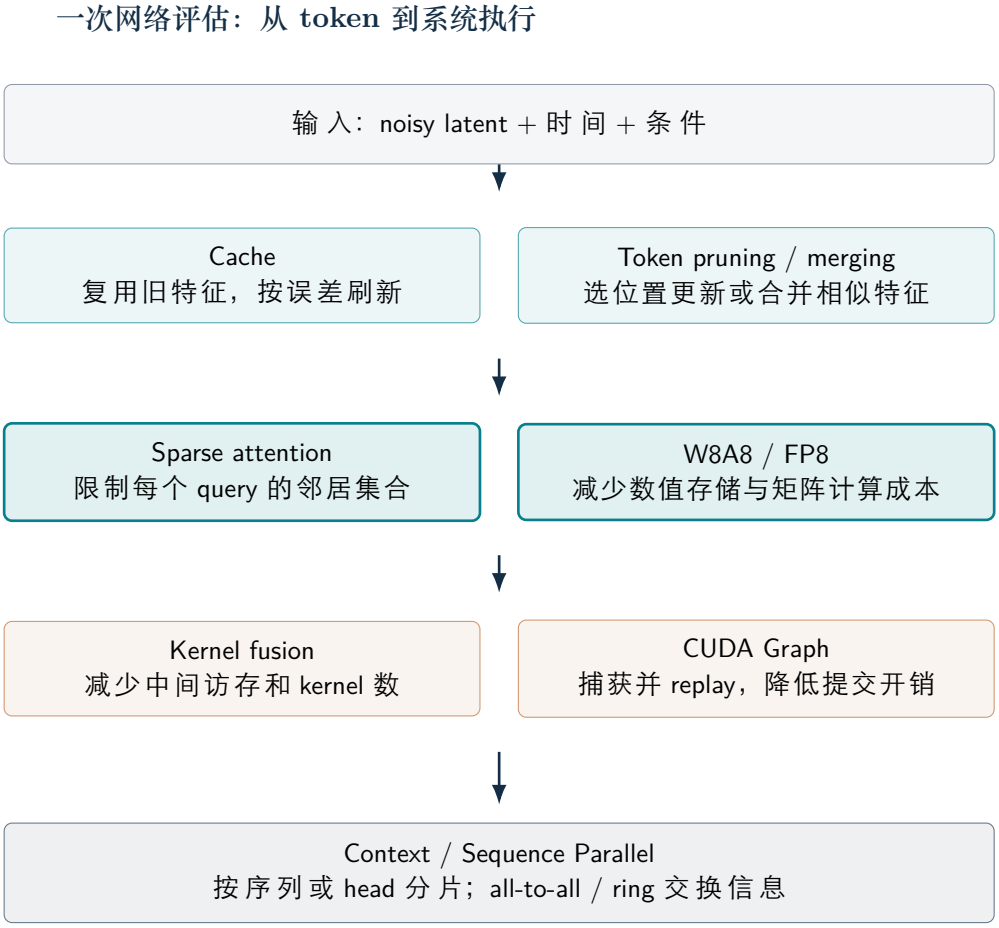


图 10.4　每一步的成本花在哪里?。缓存、token、连接、精度、kernel 与通信分别提供优化切入点。使用方法：先用 profiler 确定瓶颈，再选对应组件，最后复测整条链路的时间与质量。[82, 85, 86, 88, 89]

收益表格至少要有这些列

硬件型号与数量；分辨率与帧数；NFE 与 CFG；precision；batch/并发；是否含 VAE；首帧延迟；每 chunk 延迟及尾延迟；稳态 FPS；显存；质量与失败率。

TurboDiffusion 主要让一次生成更快；TurboServe 主要让很多长期会话更经济地同时运行。

10.14 分布式执行与服务案例：MagiAttention、MagiCompiler、Vivix

MagiAttention：让异构 mask 的算量更均衡

因果 mask 的早期 query 读得少，后期 query 读得多。简单等分 token 未必等分 FLOPs。MagiAttention 以可表达 Q/K 范围的切片描述、细粒度分片和 dispatch solver 组织工作；GroupCast 与 GroupReduce 按实际依赖传递 K/V 与梯度，配合多阶段计算通信重叠。它是 context parallel 执行路线；attention 的数学 mask 由上层任务给出。[160]

以 rank r 的工作时间 T_r 计，step 时间受 $\max_r T_r$ 主导；负载均衡因子可写为 $\max_r T_r / \operatorname{mean}_r T_r$。拓扑、通信带宽与分块粒度决定是否能隐藏通信，不能把原始“理论通信字节降低”直接换成相同倍数的加速。

MagiCompiler：跨层融合与通信合并

MagiHuman 报告使用全图 PyTorch 编译器 MagiCompiler，跨 Transformer 层融合操作并减少 collective 调用次数，在其 H100 测试中约有 1.2 倍收益。它与 CUDA Graph 可以在不同层面配合：一个改变 kernel/通信组织，另一个减少重复提交开销。需通过 profiler 确认两者叠加后仍有独立收益。[161]

Vivix Model Infra：首反应的关键路径

Vivix 的官方系统报告组织了感知/推理、DiT、VAE 与流传输，采用分离阶段与 KV 状态传输，围绕持续会话和中断调度优化。理解这类 prefill/decode 分离时，应画出长上下文条件准备与后续增量生成的任务边界，不能直接套用 LLM 的逐 token 计算量。[174]

$$T_{\mathrm{TTFR}} \approx T_{\mathrm{input}} + T_{\mathrm{reasoning}} + T_{\mathrm{first\ chunk}} + T_{\mathrm{decode}} + T_{\mathrm{network}} + T_{\mathrm{queue}}.$$

上式为无重叠路径的记账示例，有流水线时取关键路径。少一步去噪省掉的时间可能被网络跨服务、排队或大 chunk 等待掩盖。优先级抢占还需要处理已经编码、正在生成和已经发送的三类状态，避免重发、音画错位或使用失效 cache。

把模型优化与系统优化拆成增量实验

先固定模型、精度和 scheduler 测服务基线；然后分别加入少步、量化、稀疏、Graph、多卡和调度。每一步重测质量、首响应、p50/p95 chunk 延迟、GPU-seconds 与显存。多个优化都减少同一瓶颈时，最终加速通常不能简单连乘。

本章思考与进阶练习

10.1 Amdahl

题目 去噪占端到端时间的 80%，将去噪提速 5 倍，整个系统提速多少?
解题思路 归一化时间为 $0.2 + 0.8/5 = 0.36$，加速约 2.78 倍。剩余编码、解码、传输和调度开始占主导。

10.2 模型切换

题目 TRACK 使用更小模型执行部分步骤，但在显存不足时必须频繁调入权重。什么情况下会更慢?
解题思路 当交换和同步开销大于减少的前向时间时，总延迟上升。应以 $\sum_i T_{m_i} + \sum_i T_{\text{switch}}$ 估计，优先测实际 PCIe/NVLink 路径、驻留策略和切换频率。

10.3 进阶：多卡效率

题目 单卡 10 秒，8 卡 2 秒。speedup、并行效率和 GPU-seconds 分别是多少?
解题思路 加速 5 倍，并行效率 $5/8 = 62.5\%$；总 GPU-seconds 从 10 变成 16。它改善单请求延迟，同时增加该口径下的算力费用，吞吐还需测多请求并发。

量化与执行进阶练习

10.4 Smoothing 的不变量

题目 为什么减去 K 的公共均值通常无需补偿，而减去 Q 的公共均值需要补偿?
解题思路 前者给每个 query 的整行分数减去同一个常数，softmax 保持不变；后者产生随 key 变化的项 $\bar{q}K'^{\top}$，需加回。检查均值轴是 token 轴，并把 $1/\sqrt{d}$ 同时应用到补偿项。

10.5 SLA 的混合权重

题目 一行选中集合在 full attention 中占 0.7 的质量，两个单独归一化的输出为 4 和 -1，正确结果与直接相加分别为多少?
解题思路 加权结果为 $0.7 \times 4 + 0.3 \times (-1) = 2.5$；直接相加为 3。选择块后再归一化会改变每路的质量，学习比例解决这一尺度问题。

10.6 Graph 的动态状态

题目 一个流式模型把旧 KV 用 torch.cat 延长，并用 t.item() 选择去噪分支。怎样改造为可复用的图?
解题思路 使用固定容量缓存与有效长度/写指针，预设 shape bucket；时间步保留为 GPU tensor；把必须在 CPU 决定的分支移到图外，或为有限静态分支分别捕获。先验证新输入后输出确实变化，再测延迟和显存。

第十一章　评价协议与研究方法

质量和速度需要共同定义。本章把参考感知、无参考质量、偏好、动作与长时能力拆开，统一 NFE 和费用口径，并给出配对统计、消融和质量–延迟曲线的组织方法。

11.1　感知质量、偏好与视频评分

参考相似度、无参考质量、条件偏好和时序能力，是不同问题。

工具/维度	核心含义	配合观察什么
LPIPS	和参考图在感知特征上像不像?	需要对齐参考；另测任务成功率
DeQA-Score	不看参考图，自身画质怎么样?	文本符合与时序稳定
HPSv3++	结合文本，人更可能偏好哪个?	时序指标、独立人评
CLIP 类对齐	图文语义是否相关?	细粒度数量、空间关系未必可靠
VBench	视频在多个维度表现如何?	各子维度与失败样本
FVD/FID	两批样本特征分布差多少?	单样本质量与人工评价

DeQA-Score

DeQA-Score 使用多模态模型与分数分布监督做无参考图像质量回归。与只让模型输出一个自由文本分数相比，其设计强调对质量等级/分数分布的建模。它可以逐帧用于视频评价，但**逐帧平均本身看不到完整运动连续性**。[46]

HPSv3 与 HPSv3++ 的版本信息

HPS 系列用于人类偏好预测。HPSv3++ 进一步研究能力和 RL 迭代条件相关的 reward 建模，其官方资源包含美学和文本服从两类偏好数据/评测入口。使用时要核对具体 checkpoint、条件参数、输出尺度及训练域。跨版本比较时，先统一评分量表与评估协议。[47]

视频 reward 应是多维的

可构造示意组合：

$$R = \lambda_a R_{\text{aesthetic}} + \lambda_c R_{\text{alignment}} + \lambda_m R_{\text{motion}} + \lambda_i R_{\text{identity}} - \lambda_f R_{\text{flicker}}.$$

先统一尺度和方向，再设权重。若只奖励“相邻帧相似”，静止视频可能钻空子；若只奖励运动幅度，抖动可能得高分。要区分**动态程度**和**动作质量/物理合理性**。

训练评分器和用评分器训练生成器是两件事

RM 可用人类成对偏好训练；生成器随后用 RM 输出做排序、过滤、DPO 数据构造、可微反传或 RL。在线 reward 的提升必须用**独立指标、人工盲评、留出 prompt 与多样性**交叉检验。[40, 48]

11.2 视频能力评价与实验面板

质量、可控性、时序、物理和效率共同构成视频生成的评价面板。

三组关键评价入口

VBench 将视频质量和语义拆成细项；**VBench-2.0** 进一步围绕人类保真、可控性、创造性、物理与常识评估内在忠实度。**T2V-CompBench** 用 1400 条提示覆盖属性、空间、动作、运动、交互和计数等七类组合能力，结合多模态模型、检测与跟踪度量。[48, 122, 123]

建议保留的实验面板

维度	典型观察	控制项
画质与时序	清晰度、闪烁、运动自然度	分辨率、帧率、视频时长
条件符合度	主体属性、动作、数量、运镜	提示集合、参考图与随机种子
长时能力	身份、背景、事件顺序、漂移	历史长度、分块大小、提示切换
多样性	多种构图、动作与合理结果	每条提示的采样数量
人类偏好	盲评胜 / 平 / 负与分歧	评分维度、评审人数与置信区间
系统效率	首段延迟、稳态 FPS、显存	硬件、NFE、精度、计时边界

11.3 NFE、延迟、显存与收益量纲

先找比较对象和计时范围，再计算改善幅度。

单次生成的成本分解

$$T_{\text{total}} = T_{\text{cond}} + T_{\text{denoise}} + T_{\text{codec}} + T_{\text{IO}},$$
$$T_{\text{denoise}} \approx \sum_{j=1}^{N_{\text{eval}}} C_j + T_{\text{launch}} + T_{\text{comm}}.$$

A 改变达到目标质量所需的评估次数；B/C 让模型适应更大的跨度；D 减少 C_j、重复计算或系统开销；E 改变生成任务的分解与关键路径。

Amdahl 定律的端到端示例

若原总时间 100 秒，其中 90 秒去噪、10 秒其他；去噪加速 4 倍后：

$$T' = 90/4 + 10 = 32.5 \text{ 秒}, \qquad S = 100/32.5 \approx 3.08.$$

一般式为 $S = 1/[(1-p) + p/a]$，p 是被加速部分的原占比，a 是局部倍数。进一步优化时，占比会变化，因此组合结果最好重新测量。

分数差、相对比例和倍数

- VBench 80.91 → 81.65：提高 0.74 分；相对原分数约提高 0.91%。
- 延迟 10 秒 → 2 秒：加速 5 倍，同时延迟下降 80%。

- FID 10 → 8：FID 降低 20%；画质含义需结合样本量、分布和人评。

一张可复用的实验记录卡

类别	必填内容
模型与条件	checkpoint、分辨率、帧数、prompt 集、seed、CFG
计算预算	steps、NFE、完整重算次数、precision、batch、GPU 数
时间	去噪 / VAE / 端到端；首包、均值、P95 / P99、稳态 FPS
资源	峰值显存、GPU-seconds、通信与缓存开销
质量	画质、文本对齐、运动、多样性、长期漂移、人评与失败率
训练代价	数据访问、教师接口、GPU-hours、额外训练分支

最有说服力的结果是同一协议下的质量–成本曲线：给定预算谁更好，给定质量谁更省。

11.4 实验设计、置信区间与总预算

质量匹配的加速曲线

固定 prompt、seed、分辨率、时长和基础 checkpoint，分别扫描 NFE、缓存比例、量化方案与并行卡数，画出质量–延迟 Pareto 曲线。报告两个方向：固定质量阈值下最快延迟，以及固定预算下最高质量。若以 checkpoint 的默认设置互相比，会把采样预算和算法贡献混在一起。

配对人评与统计单位

设每条 prompt 上模型 A 相对 B 的胜、平、负记为 $1, 1/2, 0$，平均得到胜率分数。简单独立二项近似的标准误为 $\sqrt{p(1-p)/n}$；视频评审中多种子、同 prompt 和同评审员会相关，更适合按 prompt 聚类 bootstrap，再计算区间。重复相邻帧的评分并不会创造同等数量的独立样本。

评价集、奖励与数据泄漏

训练 reward、离线选 checkpoint 的验证集、最终测试集分开。若论文用某个质量模型驱动训练，另选独立评分和盲评核对。同步报告多样性、motion degree 与 motion quality，防止静态输出靠“稳定”指标得高分。长视频至少在多个时长上测漂移，并包括新提示、镜头变化、遮挡和主体重新出现。

组合消融与交互项

设优化 A、B 都能提速，至少保留 baseline、A、B、A+B 四组。在同一预算下比较质量变化 ΔQ，交互项

$$I_{AB} = \Delta Q_{A+B} - \Delta Q_A - \Delta Q_B$$

可提示两者是否共享误差或互补。时间也按同一分解记录：初始化、编码、去噪、解码、传输；蒸馏额外训练费用可用节省的单次推理费用估算回本次数。

$$N_{\text{break-even}} = \frac{C_{\text{extra train}}}{C_{\text{base sample}} - C_{\text{fast sample}}}.$$

分子分母采用同一费用单位，且质量满足预先设定的要求；这是部署决策的预算模型。

本章思考与进阶练习

11.1 收益量纲

题目 分数从 80.91 到 81.65、耗时从 20 秒到 5 秒，分别如何表述？
解题思路 分数提高 0.74 分，相对约 0.91%；速度为原来的 4 倍，耗时减少 75%。质量分数是否为百分制、是否具有比例意义要另查指标定义。

11.2 统计独立性

题目 100 条 prompt，每条 4 个 seed、每个视频 100 帧，能否把 40000 帧视为独立人评样本？
解题思路 帧在同视频内高度相关，prompt 和 seed 也构成层级结构。按 prompt 组织配对评价或聚类 bootstrap，报告视频数量与独立 prompt 数，避免虚假缩小置信区间。

11.3 进阶：奖励漏洞

题目 训练 reward 与 VBench 都改善，但用户更不喜欢视频。如何定位？
解题思路 核对 reward 与评价是否共享模型或数据、动作幅度是否被压低、是否损失多样性、是否在少数题型退化。使用独立盲评、分维度分析和真实任务成功率定位分歧，并回放全部生成与筛选过程。

第十二章　生成系统与跨模态前沿

前面的生成器可以成为 Agent 工具，也可以与理解、语音、三维和世界状态建模结合。本章按输入输出和反馈接口比较这些扩展，保留它们与视频生成主线的连接。

12.1　生成 Agent 的规划、工具与反馈

Agent 描述有状态的决策与反馈闭环，既可以围绕外部生成器，也可以位于统一模型内部。

一个具体的生成任务

用户说："做一个 20 秒产品广告，保持角色一致，结尾出现产品名。"系统可能执行：

1. Planner 拆成镜头/动作/时长，确定角色参考和约束；
2. 工具检索参考，生成角色图与关键帧；
3. 视频生成器生成各片段；
4. Critic 检查身份、文字、动作和镜头衔接；
5. 只重试失败片段，再拼接配音、字幕或转场。

剪辑是可选工具之一。单张海报生成也可以用 agent；单段视频也可以反复观察与修改。

三个不同层次

层次	做法	是否必须修改视频模型
外部编排	LLM 调用生成、编辑、评价工具	不需要
先推理后生成	先输出计划/布局，再条件生成	可能需要模型或接口支持
交错推理与生成	观察中间视觉结果，再规划下一步	需要维护多模态状态，训练可不同

"CoT → 视频"可以提供计划；进一步的 Agent 闭环包括**观察、决策、动作、反馈、停止条件**。

GenEvolve：蒸馏的是生成 Agent 的行为经验

GenEvolve 面向图像生成 agent：比较同一请求下多条工具轨迹，将好/坏差异提炼成结构化视觉经验，只给特权教师分支；再在学生轨迹上提供 token-level 蒸馏信号。它改善检索、参考选择、工具与 prompt 组织，**监督作用在 Agent 的文本与工具行为上**。[53]

SEED：事后经验变成训练时技能

SEED 从已完成 on-policy 轨迹提取自然语言 hindsight skills，比较普通上下文与技能增强上下文下已采样动作的概率，形成密集蒸馏信号，并与结果奖励 RL 联合。它研究通用 Agent 学习；用于视频系统时，先指定它优化的 planner 或工具决策层。[54]

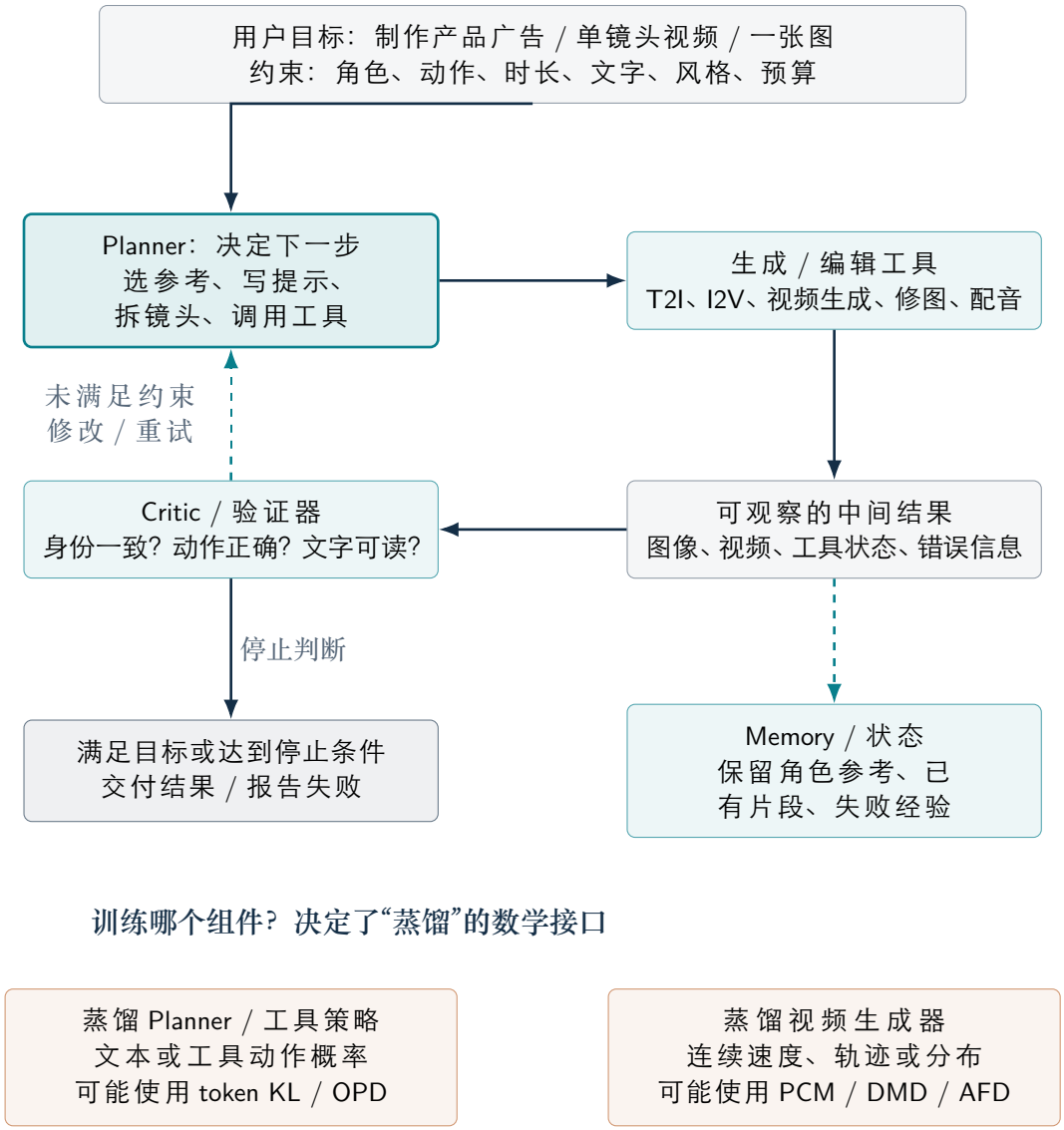


图 12.1 Agentic generation：生成只是一个工具动作。Agent 的关键是规划、调用、观察、评价、再决策的闭环。**OPD 蒸馏哪一层?** GenEvolve 的核心是生成 Agent 的行为经验；SEED 研究技能增强上下文下的自蒸馏。视频工具可以保持固定，由 Agent 学习更有效的调用方式。[53, 54]

12.2 理解生成统一与共享主干

可以共享主干，同时保留不同视觉编码器、输出头与训练目标。

理解与生成的表示需求

理解任务希望保留语义、关系和可回答的问题，允许丢掉大量像素细节；生成任务需要保留足以重建纹理、颜色和几何的表示。可以根据任务选择共享表示、专属编码器或混合设计。

三个公开方向实例

- **BAGEL**：公开的统一多模态理解/生成模型，结合语义视觉表示与生成用 VAE 表示，在共享模型中支持不同任务。统一模型可以同时拥有文字 CE 与图像 flow 等目标。[55]
- **Janus 系列**：强调解耦理解和生成的视觉编码路径，再在统一框架中处理。Janus-Pro 与 JanusFlow 的分别采用各自的生成机制。[56]
- **Show-o 系列**：研究在单 Transformer 下统一多模态理解与生成；不同版本的离散/连续生成设定需要分别阅读。[57]

这些例子展示多种统一架构。**能力清单可分别记录图像生成、视频理解、未来帧预测与实时长视频生成。**

推理条件与视觉生成

概念上可以先采样文本计划 a，再生成视觉结果：

$$p(X, a \mid c) = p(a \mid c)\, p(X \mid a, c).$$

也可以交错输出文字和图像，再根据视觉结果继续推理。这个分解描述计划与生成的条件关系；消融实验可比较无计划、错误计划、长计划和额外推理预算的作用。

视频 Agent 的 OPD 应先确定蒸馏哪层

- 蒸馏 planner：对齐文本/工具动作分布，可能是 token KL。
- 蒸馏视频生成器：对齐速度、score、轨迹或视频分布。
- 蒸馏整体工具轨迹：学习何时搜索、生成、评估、重试和停止。

“理解生成统一”讨论模型能力与表示；“agentic”讨论决策闭环；“OPD”讨论训练状态来源。三者从能力、决策流程与训练状态三个角度描述系统，可以组合。

12.3 音视频联合生成与推理驱动的生成接口

LTX-2：两种 latent，共同去噪

LTX-2 使用各自的音频与视频 VAE，并在不对称双流 DiT 中联合建模：视频流约 14B 参数，音频流约 5B。每个流处理自身 attention、文本条件和跨模态 attention；视频内部使用三维位置，音视频交换信息时使用共同的时间位置。[125]

用一般联合 FM 记号表达这一训练接口：

$$(\hat{v}_V, \hat{v}_A) = F_\theta(z_t^V, z_t^A, t, c),$$
$$L = \lambda_V \|\hat{v}_V - u_V\|^2 + \lambda_A \|\hat{v}_A - u_A\|^2.$$

两种表示的维数、时间采样率和解码器各自定义；λ 与归一化共同决定两项梯度尺度。跨模态交互使声音事件与画面变化相互条件化。该式是联合回归接口示意，实际日程、条件丢弃和双模态 CFG 按报告配置。

生成前的推理与工具式 Agent

给定任务 c，可以先生成文字计划 r，再生成视觉结果 X：

$$p(r, X \mid c) = p_\theta(r \mid c)p_\theta(X \mid r, c).$$

这可由共享模型完成，也可由 planner 调用独立生成器完成。若进一步让评审决定重新生成、局部编辑和镜头拼接，动作空间扩展为工具调用轨迹。推理文本质量、最终画面质量与整项任务成功率构成三个评价层。

UniGRPO：把两种策略接口接到同一奖励

UniGRPO（2026 年 3 月）在推理驱动图像生成中，对离散文字使用 GRPO，对连续图像采样使用 Flow-GRPO，并研究 CFG 与参考约束的配套处理。公开实验以单轮推理到图像为主。[131]

视频 Agent 的扩展需要定义额外动作：镜头分解、参考选择、运动控制、编辑与接受/重试。对 planner 做 OPD，可让学生产生计划并由教师反馈；对视频工具做蒸馏，则使用本书的连续轨迹、score 或成品反馈。统一任务奖励还需在多次调用间分配信用，统计所有被放弃样本的时间与费用。

12.4 三维生成与动作条件世界模型

视频预测、可编辑三维资产与环境动力学，分别对应不同输出和验证标准。

3D 生成的表示选择

输出可以是 mesh、点云、隐式场、体素、3D Gaussian 或这些表示的 latent。训练仍可以使用 VAE 与 diffusion/FM，decoder 将 latent 还原为几何或场。

Hunyuan3D 2.0 的代表性拆分

先用 Hunyuan3D-DiT 生成几何，再用 Hunyuan3D-Paint 给 mesh 合成纹理。把形状和纹理解耦，有利于分别控制几何质量和外观，也可以给手工 mesh 上纹理。输出包含可供渲染与编辑的几何和纹理。[58]

HunyuanWorld 1.0 的代表性路径

利用全景图作为 360 度世界代理，进行语义分层、分解与重建，得到可导出的分层 mesh 世界。它重点是可探索、可交互的三维场景构建，其输出是三维场景表示，视频预测型世界模型则输出未来观察。[59]

动作条件视频世界模型

另一类模型直接建模：

$$p(o_{t+1} \mid o_{\le t}, a_{\le t}),$$

其中 o 是观察，a 是动作或控制。可以通过 AR diffusion 逐步预测未来视觉状态。要评估控制响应、跨视角一致、长时记忆和物理合理性，而非只看单帧美观。

目标	输出	典型验证
视频生成	像素帧序列	画质、动作、文本符合
3D 资产	几何 + 纹理	多视角一致、mesh 可用性
场景生成	可探索的场景表示	遮挡、几何、导航、交互
动态世界模型	动作条件未来状态	因果控制、长期状态、预测正确性

术语“world model”使用很宽泛。读论文时明确它学习的是可渲染场景、视频预测器，还是能支持决策规划的环境动力学；再按动作响应、物理规律和长时一致性验证其能力。

12.5 语音生成架构的对照

语音也同时存在离散 AR 和连续 flow；这能帮助理解视频里的“AR + diffusion”。

代表模型	核心生成路径	与视频概念的对应
Spark-TTS	LLM 预测解耦语音 codes，再解码波形	离散 token AR + codec decoder
F5-TTS	文本/参考条件下用 Flow Matching 生成声学表示	连续去噪/速度场 + 声码器
Qwen3-TTS	离散多码本 LM，配套语音 tokenizer/decoder	码本序列由专用解码器还原波形
CosyVoice	以语音 token LM 与条件 flow 等组成系统	AR 决定结构，连续模型补声学细节

以上按所引官方版本概括，CosyVoice 与 Qwen3-TTS 都是系列，具体 tokenizer、流式实现和模块结构随版本变化。[60, 61, 62, 63]

同一问题，换一种模态再看

语音 codec token 可以是码本索引，具有离散 logits；mel 或声学 latent 是连续张量，可以做 L1/L2/FM。两者都可用 Transformer，因而都可能有 Q/K/V。**attention 处理特征，输出头决定索引或连续数值。**

流式也分多个环节

文本是否流式到达？声学 token 是否流式生成？声码器需要多少 lookahead？首包延迟与稳态实时率怎样计？视频里的基模/Refiner/VAE 分层问题，在 TTS 里同样存在。

语音结构、流式延迟与质量评价

AR 与少步 flow 的速度都受具体实现影响。模型大小、码率、并发、缓存、声码器和硬件都会影响结果。语音质量还需看可懂度、发音错误、说话人相似与自然度，与实时率一起构成质量–成本评价。

一个模型里可以同时有**离散结构生成**与**连续细节生成**。这正是理解许多多模态系统的通用视角。

12.6 产业模型索引与公开资源

各系列按本书模块展开，下表把版本与章节对应起来。资料核对截至 2026-10-18；架构细节以列出的论文、官方报告与代码为依据。

系列	本次覆盖的版本/工作	主要阅读位置
生数 / Vidu	U-ViT 研究背景；S1、TurboDiffusion、TurboServe	骨干基础；第五章 TF/DF、TwinCache；第十章加速与服务
Vivix	A1、W1、Vivix-Turbo、IVI、Model Infra	第五章交互与状态；第八章蒸馏；第十章关键路径
MiniMax	Hailuo 02 NCR、H3 Base/Ref2VA/FL2VA/Regenerate	第一章表示；第二章主干；第五章高清再生成；第六章数据
SandAI	MAGI-1、MagiHuman、MAGI-2 Preview、MagiAttention/Compiler	第二章单流与 MoE；第五章 AR；第十章分布式执行
ByteDance Seed	Seedance 1.0、1.5 pro、2.0、2.5	第二章音视频结构；第六章数据；第八章蒸馏；第九章奖励

论文、代码、权重和在线入口

MAGI-1、MagiHuman 与 H3 提供公开实现/模型入口。Vidu S1、Vivix 与 Seedance 的技术报告各自公开了不同粒度的方法；在线访问与开放权重分别核对。U-ViT 的公开论文用于理解跳连 Transformer 主干，当前商业 Vidu 各版本的精确配置应以各自报告为准。[6, 52, 159, 161, 165, 169, 172]

体验入口：Vidu（S1 报告另提供 流式入口）、Vivix、Hailuo、SandAI、Seedance 官方模型页。在线功能与开放范围随版本更新。

公开模型与实现入口

系列/项目	可从官方入口核对什么	读法
Wan2.1	代码、权重、VAE 与生成管线	适合对照 latent 视频实际实现
HunyuanVideo	原版架构与视频生成实现	对照双流/单流、文本条件
LongLive	流式训练/推理与后续分支	严格区分 1.0 / 2.0 / Plug
PCM	方法、少步权重与训练示例	匹配 phase 和 sampler
BAGEL / Janus / Show-o	理解与生成统一的实现	分别记录理解与生成能力

模型发布资产的检查表

1. 许可证是否允许所用的使用方式？代码与权重许可可能不同。
2. 发布的是 base、distilled 还是 LoRA？是否还依赖未提供的底座？
3. 推理默认步数是否只是示例？哪个 scheduler 与参数化匹配？
4. 论文模型与公开 checkpoint 是否完全一致？训练脚本是否包含完整数据流程？

公开入口见文末一手资料。可据此定位论文、实现、权重和体验渠道。

本章思考与进阶练习

12.1 Agent 与模型

题目 规划器写出分镜后调用现有视频工具，与单一模型生成 CoT 再输出画面，训练接口有何差异？
解题思路 前者的策略动作主要是文本计划和工具参数，视频工具可冻结；后者可联合优化离散推理与连续生成。两者都能在最终任务上评估，但梯度可达性、logprob 与预算分配各不相同。

12.2 音视频时间网格

题目 视频 latent 每秒 6 个时间位置，音频 latent 每秒 25 个，跨模态 attention 怎样对齐？
解题思路 把位置映射到共同的物理时间坐标后编码，再允许一对多交互。直接按索引对齐会混淆不同时间；同步指标还应覆盖音素、嘴形和物体碰撞等不同事件尺度。

12.3 进阶：世界模型

题目 一个视频模型视觉上很逼真，为什么仍需另外评估它作为规划环境的适用性？
解题思路 控制要求动作对状态的可预测影响、反事实一致性与长时闭环成功。可用相同起点施加不同动作，评估状态转移、可重复性和规划成功率；视觉评分只覆盖其中一部分。

附录 A　术语与接口速查

本附录用于查找术语；完整推导与训练流程见各主题章节。

概念	最短但不失真的理解
$q_\phi(Z \mid X)$	编码器看到 X 后，为 Z 预测的概率分布
KL / VAE	平衡重建保真与 latent 分布规则化
L1 / L2	数值误差；若作用在参数上，则是另一类正则
LPIPS	输出和参考画面在感知特征上像不像?
Hinge / logistic	分类间隔 / 平滑分类与偏好概率
R1 / R2	在真/假样本处约束判别器的输入梯度
VAE / patch	先压缩，再把 latent 切块送进网络
KV / logits	历史特征缓存 / 词表分布的参数
Diffusion / FM	学习去噪或连续速度，再由 solver 积分生成
AR / causal	先后生成的概率分解 / 不看未来的信息约束
CM / PCM / rCM	全程终点一致 / 分段一致 / 连续一致性加分布匹配
DMD	用教师与学生分布 score 的差来纠正生成器
Self Forcing	在自己生成的历史上练习
LongLive	长历史 rollout，短片段监督与缓存管理
OPSD-V	同一学生状态，教师用更干净的真实长视频上下文
AFD	黑盒成品比较，转为学生侧 flow 更新
ReFL / DRaFT	可微 reward 直接传回生成器
DDPO	用最终奖励优化连续去噪转移概率
DanceGRPO	同 prompt 多次生成，用组相对优势学习
DeQA / HPS	无参考画质 / 条件下的人类偏好
Refiner	把已有低清视频变成高质量高分辨率视频
Agentic generation	规划、生成、观察、评估、重试的决策闭环

读公式的五个检查点：

随机变量是什么? 期望对谁取? 哪些参数更新? 哪里停止梯度? 训练和推理的条件是否一样?

最核心的统一视角：

模型处理的是数字，但训练目标决定这些数字代表什么；架构决定信息怎样流动，采样器决定怎样执行，评价协议决定“收益”究竟意味着什么。

一手资料与延伸阅读

[1] Auto-Encoding Variational Bayes.
https://arxiv.org/abs/1312.6114

[2] High-Resolution Image Synthesis with Latent Diffusion Models.
https://arxiv.org/abs/2112.10752

[3] Denoising Diffusion Probabilistic Models.
https://arxiv.org/abs/2006.11239

[4] Flow Matching for Generative Modeling.
https://arxiv.org/abs/2210.02747

[5] Scalable Diffusion Models with Transformers (DiT).
https://arxiv.org/abs/2212.09748

[6] All are Worth Words: A ViT Backbone for Diffusion Models (U-ViT).
https://arxiv.org/abs/2209.12152

[7] Scaling Rectified Flow Transformers for High-Resolution Image Synthesis (SD3).
https://arxiv.org/abs/2403.03206

[8] HunyuanVideo：官方代码与架构说明.
https://github.com/Tencent-Hunyuan/HunyuanVideo

[9] Wan2.1：官方代码、VAE 与模型资源.
https://github.com/Wan-Video/Wan2.1

[10] Classifier-Free Diffusion Guidance.
https://arxiv.org/abs/2207.12598

[11] The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS).
https://arxiv.org/abs/1801.03924

[12] Generative Adversarial Networks.
https://arxiv.org/abs/1406.2661

[13] Spectral Normalization for Generative Adversarial Networks.
https://arxiv.org/abs/1802.05957

[14] Which Training Methods for GANs do actually Converge?.
https://arxiv.org/abs/1801.04406

[15] Consistency Models.
https://arxiv.org/abs/2303.01469

[16] Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference.
https://arxiv.org/abs/2310.04378

[17] Progressive Distillation for Fast Sampling of Diffusion Models.
https://arxiv.org/abs/2202.00512

[18] Phased Consistency Models.
https://arxiv.org/abs/2405.18407

[19] PCM：官方实现与配置说明.
https://github.com/G-U-N/Phased-Consistency-Model

[20] Large Scale Diffusion Distillation via Score-Regularized Continuous-Time Consistency.
https://arxiv.org/abs/2510.08431

[21] rCM：NVIDIA 官方研究项目页.
https://research.nvidia.com/labs/dir/rcm/

[22] One-step Diffusion with Distribution Matching Distillation.
https://arxiv.org/abs/2311.18828

[23] Improved Distribution Matching Distillation for Fast Image Synthesis (DMD2).
https://arxiv.org/abs/2405.14867

[24] Learning Few-Step Diffusion Models by Trajectory Distribution Matching.
https://arxiv.org/abs/2503.06674

[25] Transition Matching Distillation for Fast Video Generation.
https://arxiv.org/abs/2601.09881

[26] Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion.
https://arxiv.org/abs/2407.01392

[27] From Slow Bidirectional to Fast Autoregressive Video Diffusion Models (CausVid).
https://arxiv.org/abs/2412.07772

[28] Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion.
https://arxiv.org/abs/2506.08009

[29] Causal Forcing: Autoregressive Diffusion Distillation Done Right for High-Quality Real-Time Interactive Video Generation.
https://arxiv.org/abs/2602.02214

[30] LongLive: Real-time Interactive Long Video Generation.
https://arxiv.org/abs/2509.22622

[31] LongLive：官方方法与训练文档.
https://nvlabs.github.io/LongLive/docs/

[32] LongLive 1.0：DMD 官方实现.
https://github.com/NVlabs/LongLive/blob/main/LongLive1.0/model/dmd.py

[33] LongLive 系列：官方性能统计与 FPS 口径.
https://github.com/NVlabs/LongLive/blob/main/LongLive2.0/README.md

[34] LongLive-Plug: Once-for-All Distillation for Video Generation.
https://arxiv.org/abs/2609.38154

[35] MiniLLM: Knowledge Distillation of Large Language Models.
https://arxiv.org/abs/2306.08543

[36] On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes.
https://arxiv.org/abs/2306.13649

[37] OPSD-V: On-Policy Self-Distillation for Post-Training Few-Step Autoregressive Video Generators.
https://arxiv.org/abs/2607.08766

[38] On-Policy Adversarial Flow Distillation for Autoregressive Video Generation.
https://arxiv.org/abs/2605.26105

[39] Diffusion Model Alignment Using Direct Preference Optimization.
https://arxiv.org/abs/2311.12908

[40] ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation.
https://arxiv.org/abs/2304.05977

[41] Directly Fine-Tuning Diffusion Models on Differentiable Rewards (DRaFT).
https://arxiv.org/abs/2309.17400

[42] Training Diffusion Models with Reinforcement Learning (DDPO).
https://arxiv.org/abs/2305.13301

[43] DDPO：作者官方实现.
https://github.com/jannerm/ddpo

[44] DanceGRPO: Unleashing GRPO on Visual Generation.
https://arxiv.org/abs/2505.07818

[45] DanceGRPO：作者官方实现.
https://github.com/XueZeyue/DanceGRPO

[46] DeQA-Score：Teaching Large Language Models to Regress Accurate Image Quality Scores using Score Distribution.
https://github.com/zhiyuanyou/DeQA-Score

[47] HPSv3++: Scaling Reward Models Across the Full Spectrum of Diffusion Model Capabilities.
https://github.com/PlantPotatoOnMoon/HPSv3-PlusPlus

[48] VBench: Comprehensive Benchmark Suite for Video Generative Models.
https://arxiv.org/abs/2311.17982

[49] SoL-Refiner: Speed-of-Light One-Step Refinement for High-Resolution Video.
https://arxiv.org/abs/2609.37969

[50] TurboDiffusion: Accelerating Video Diffusion Models by 100-200 Times.
https://arxiv.org/abs/2512.16093

[51] TurboServe: Serving Streaming Video Generation Efficiently and Economically.
https://arxiv.org/abs/2606.19271

[52] Vidu S1: A Real-Time Interactive Video Generation Model.
https://arxiv.org/abs/2607.03118

[53] GenEvolve: Self-Evolving Image Generation Agents via Tool-Orchestrated Visual Experience Distillation.
https://arxiv.org/abs/2605.21605

[54] SEED: Self-Evolving On-Policy Distillation for Agentic Reinforcement Learning.
https://arxiv.org/abs/2607.14777

[55] BAGEL：统一多模态理解与生成官方实现.
https://github.com/ByteDance-Seed/Bagel

[56] Janus 系列：统一多模态理解与生成.
https://github.com/deepseek-ai/Janus

[57] Show-o 系列：单 Transformer 统一理解与生成.
https://github.com/showlab/Show-o

[58] Hunyuan3D 2.0：几何与纹理生成官方实现.
https://github.com/Tencent-Hunyuan/Hunyuan3D-2

[59] HunyuanWorld 1.0: Generating Immersive, Explorable, and Interactive 3D Worlds from Words or Pixels.
https://arxiv.org/abs/2507.21809

[60] Spark-TTS：官方实现.
https://github.com/SparkAudio/Spark-TTS

[61] F5-TTS：官方实现.
https://github.com/SWivid/F5-TTS

[62] Qwen3-TTS：官方架构与发布资源.
https://github.com/QwenLM/Qwen3-TTS

[63] CosyVoice：官方实现与版本说明.
https://github.com/QwenAudio/CosyVoice

[64] High Fidelity Neural Audio Compression (EnCodec).
https://arxiv.org/abs/2210.13438

[65] Scaling Diffusion Transformers to 16 Billion Parameters (DiT-MoE).
https://arxiv.org/abs/2407.11633

[66] Score-Based Generative Modeling through Stochastic Differential Equations.
https://arxiv.org/abs/2011.13456

[67] Implicit Generation and Generalization in Energy-Based Models.
https://arxiv.org/abs/1903.08689

[68] Denoising Diffusion Implicit Models.
https://arxiv.org/abs/2010.02502

[69] DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps.
https://arxiv.org/abs/2206.00927

[70] DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models.
https://arxiv.org/abs/2211.01095

[71] UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models.
https://arxiv.org/abs/2302.04867

[72] Fast Sampling of Diffusion Models with Exponential Integrator (DEIS).
https://arxiv.org/abs/2204.13902

[73] Consistency Trajectory Models: Learning Probability Flow ODE Trajectory of Diffusion.
https://arxiv.org/abs/2310.02279

[74] Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models.
https://arxiv.org/abs/2410.11081

[75] Score identity Distillation: Exponentially Fast Distillation of Pretrained Diffusion Models for One-Step Generation.
https://arxiv.org/abs/2404.04057

[76] Adversarial Diffusion Distillation.
https://arxiv.org/abs/2311.17042

[77] SDXL-Lightning: Progressive Adversarial Diffusion Distillation.
https://arxiv.org/abs/2402.13929

[78] Hyper-SD: Trajectory Segmented Consistency Model for Efficient Image Synthesis.
https://arxiv.org/abs/2404.13686

[79] One Step Diffusion via Shortcut Models.
https://arxiv.org/abs/2410.12557

[80] Mean Flows for One-step Generative Modeling.
https://arxiv.org/abs/2505.13447

[81] Flow map matching with stochastic interpolants: A mathematical framework for consistency models.
https://arxiv.org/abs/2406.07507

[82] Timestep Embedding Tells: It's Time to Cache for Video Diffusion Model.
https://arxiv.org/abs/2411.19108

[83] Token Pruning for Caching Better: 9 Times Acceleration on Stable Diffusion for Free.
https://arxiv.org/abs/2501.00375

[84] Token Merging for Fast Stable Diffusion.
https://arxiv.org/abs/2303.17604

[85] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.
https://arxiv.org/abs/2205.14135

[86] FP8 Formats for Deep Learning.
https://arxiv.org/abs/2209.05433

[87] PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation.
https://pytorch.org/assets/pytorch2-2.pdf

[88] PyTorch CUDA semantics: CUDA Graphs.
https://docs.pytorch.org/docs/stable/notes/cuda.html

[89] DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models.
https://arxiv.org/abs/2309.14509

[90] Ring Attention with Blockwise Transformers for Near-Infinite Context.
https://arxiv.org/abs/2310.01889

[91] Reducing Activation Recomputation in Large Transformer Models.
https://arxiv.org/abs/2205.05198

[92] Imagen Video: High Definition Video Generation with Diffusion Models.
https://arxiv.org/abs/2210.02303

[93] Elucidating the Design Space of Diffusion-Based Generative Models.
https://arxiv.org/abs/2206.00364

[94] Efficient Diffusion Training via Min-SNR Weighting Strategy.
https://arxiv.org/abs/2303.09556

[95] Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow.
https://arxiv.org/abs/2209.03003

[96] InstaFlow: One Step is Enough for High-Quality Diffusion-Based Text-to-Image Generation.
https://arxiv.org/abs/2309.06380

[97] Neural Discrete Representation Learning (VQ-VAE).
https://arxiv.org/abs/1711.00937

[98] MaskGIT: Masked Generative Image Transformer.
https://arxiv.org/abs/2202.04200

[99] Language Model Beats Diffusion: Tokenizer is Key to Visual Generation (MAGVIT-v2).
https://arxiv.org/abs/2310.05737

[100] Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction.
https://arxiv.org/abs/2404.02905

[101] Autoregressive Image Generation without Vector Quantization (MAR).
https://arxiv.org/abs/2406.11838

[102] Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think.
https://arxiv.org/abs/2410.06940

[103] Diffusion Transformers with Representation Autoencoders.
https://arxiv.org/abs/2510.11690

[104] Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders.
https://arxiv.org/abs/2601.16208

[105] Back to Basics: Let Denoising Generative Models Denoise (JiT).
https://arxiv.org/abs/2511.13720

[106] Video Diffusion Models.
https://arxiv.org/abs/2204.03458

[107] Align your Latents: High-Resolution Video Synthesis with Latent Diffusion Models.
https://arxiv.org/abs/2304.08818

[108] Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets.
https://arxiv.org/abs/2311.15127

[109] Latte: Latent Diffusion Transformer for Video Generation.
https://arxiv.org/abs/2401.03048

[110] CogVideoX: Text-to-Video Diffusion Models with an Expert Transformer.
https://arxiv.org/abs/2408.06072

[111] HunyuanVideo: A Systematic Framework For Large Video Generative Models.
https://arxiv.org/abs/2412.03603

[112] Wan: Open and Advanced Large-Scale Video Generative Models.
https://arxiv.org/abs/2503.20314

[113] RoFormer: Enhanced Transformer with Rotary Position Embedding.
https://arxiv.org/abs/2104.09864

[114] Adding Conditional Control to Text-to-Image Diffusion Models.
https://arxiv.org/abs/2302.05543

[115] IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models.
https://arxiv.org/abs/2308.06721

[116] AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models without Specific Tuning.
https://arxiv.org/abs/2307.04725

[117] CameraCtrl: Enabling Camera Control for Video Diffusion Models.
https://arxiv.org/abs/2404.02101

[118] Packing Input Frame Context in Next-Frame Prediction Models for Video Generation.
https://arxiv.org/abs/2504.12626

[119] Video Diffusion Alignment via Reward Gradients (VADER).
https://arxiv.org/abs/2407.08737

[120] VideoDPO: Omni-Preference Alignment for Video Diffusion Generation.
https://arxiv.org/abs/2412.14167

[121] Improving Video Generation with Human Feedback (VideoReward / Flow-DPO).
https://arxiv.org/abs/2501.13918

[122] VBench-2.0: Advancing Video Generation Benchmark Suite for Intrinsic Faithfulness.
https://arxiv.org/abs/2503.21755

[123] T2V-CompBench: A Comprehensive Benchmark for Compositional Text-to-video Generation.
https://arxiv.org/abs/2407.14505

[124] HunyuanVideo 1.5 Technical Report.
https://arxiv.org/abs/2511.18870

[125] LTX-2: Efficient Joint Audio-Visual Foundation Model.
https://arxiv.org/abs/2601.03233

[126] Helios: Real Real-Time Long Video Generation Model.
https://arxiv.org/abs/2603.04379

[127] FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling.
https://arxiv.org/abs/2603.05451

[128] SageAttention3: Microscaling FP4 Attention for Inference and An Exploration of 8-Bit Training.
https://arxiv.org/abs/2505.11594

[129] Accelerating Video Diffusion via Training-Free Trajectory Routing (TRACK).
https://arxiv.org/abs/2609.30096

[130] Flow-GRPO: Training Flow Matching Models via Online RL.
https://arxiv.org/abs/2505.05470

[131] UniGRPO: Unified Policy Optimization for Reasoning-Driven Visual Generation.
https://arxiv.org/abs/2603.23500

[132] ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.
https://arxiv.org/abs/1910.02054

[133] Wan2.2: Official Models, Architecture and Inference Code.
https://github.com/Wan-Video/Wan2.2

[134] DiffusionNFT: Online Diffusion Reinforcement with Forward Process.
https://arxiv.org/abs/2509.16117

[135] DiffusionART: Efficient, Stable and Scalable Reinforcement of Diffusion Models.
https://arxiv.org/abs/2610.12572

[136] SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration.
https://arxiv.org/abs/2410.02367

[137] SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization.
https://arxiv.org/abs/2411.10958

[138] SageAttention2++: A More Efficient Implementation of SageAttention2.
https://arxiv.org/abs/2505.21136

[139] SageBwd: A Trainable Low-bit Attention.
https://arxiv.org/abs/2603.02170

[140] SageAttention official implementation and GPU dispatch.
https://github.com/thu-ml/SageAttention

[141] SLA: Beyond Sparsity in Diffusion Transformers via Fine-Tunable Sparse-Linear Attention.
https://arxiv.org/abs/2509.24006

[142] SLA official implementation.
https://github.com/thu-ml/SLA

[143] SLA2: Sparse-Linear Attention with Learnable Routing and QAT.
https://arxiv.org/abs/2602.12675

[144] ViDiT-Q: Efficient and Accurate Quantization of Diffusion Transformers for Image and Video Generation.
https://arxiv.org/abs/2406.02540

[145] ViDiT-Q official kernels and quantization code.
https://github.com/thu-nics/ViDiT-Q

[146] SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models.
https://arxiv.org/abs/2411.05007

[147] AdaTSQ: Pushing the Pareto Frontier of Diffusion Transformers via Temporal-Sensitivity Quantization.
https://arxiv.org/abs/2602.09883

[148] Attn-QAT: 4-Bit Attention With Quantization-Aware Training.
https://arxiv.org/abs/2603.00040

[149] Fast Video Generation with Sliding Tile Attention.
https://arxiv.org/abs/2502.04507

[150] SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference.
https://arxiv.org/abs/2502.18137

[151] SpargeAttention2: Trainable Sparse Attention via Hybrid Top-k+Top-p Masking and Distillation Fine-Tuning.
https://arxiv.org/abs/2602.13515

[152] VSA: Faster Video Diffusion with Trainable Sparse Attention.
https://arxiv.org/abs/2505.13389

[153] Improving Video Sparse Attention with Fine-grained Router and Sparse Rebasing (VSA2).
https://arxiv.org/abs/2609.32882

[154] FPSAttention: Training-Aware FP8 and Sparsity Co-Design for Fast Video Diffusion.
https://arxiv.org/abs/2506.04648

[155] NVIDIA: Getting Started with CUDA Graphs (V100 microbenchmark).
https://developer.nvidia.com/blog/cuda-graphs/

[156] NVIDIA: Constant Time Launch for Straight-Line CUDA Graphs (CUDA 12.6).
https://developer.nvidia.com/blog/constant-time-launch-for-straight-line-cuda-graphs-and-other-performance-enhancements/

[157] PyTorch CUDA semantics and graph memory management.
https://docs.pytorch.org/docs/2.14/notes/cuda.html

[158] MAGI-1: Autoregressive Video Generation at Scale.
https://arxiv.org/abs/2505.13211

[159] MAGI-1 official models and implementation.
https://github.com/SandAI-org/MAGI-1

[160] MagiAttention: distributed attention for heterogeneous-mask training.
https://github.com/SandAI-org/MagiAttention

[161] Speed by Simplicity: A Single-Stream Architecture for Fast Audio-Video Generative Foundation Model.
https://arxiv.org/abs/2603.21986

[162] MAGI-2 Preview: Scaling Video Generation Models Efficiently.
https://sand.ai/blog/magi-2-preview

[163] MiniMax Hailuo 02: Noise-aware Compute Redistribution.
https://www.minimax.io/news/minimax-hailuo-02

[164] MiniMax H3: An Open Model Breaking the Boundaries Between Tasks and Modalities.
https://www.minimax.io/blog/minimax-h3

[165] MiniMax H3 official model code and system overview.
https://github.com/MiniMax-AI/MiniMax-H3

[166] MiniMax H3 Transformer configuration.
https://github.com/MiniMax-AI/MiniMax-H3/blob/main/transformer/config.json

[167] Seedance 1.0: Exploring the Boundaries of Video Generation Models.
https://arxiv.org/abs/2506.09113

[168] Seedance 1.5 pro: A Native Audio-Visual Joint Generation Foundation Model.
https://arxiv.org/abs/2512.13507

[169] Seedance 2.0: Advancing Video Generation for World Complexity.
https://arxiv.org/abs/2604.14148

[170] Seedance 2.5 official release and architecture lineage.
https://seed.bytedance.com/en/blog/one-take-creation-flexible-referencing-introducing-seedance-2-5

[171] Vivix-A1 Technical Report.
https://www.vivix.ai/research/tech-report-vivix-a1

[172] Vivix-W1 Technical Report.
https://www.vivix.ai/research/tech-report-vivix-w1

[173] Vivix IVI Technical Report.
https://www.vivix.ai/research/tech-report-vivix-ivi

[174] Vivix Model Infra Technical Report.
https://www.vivix.ai/tech-report-vivix-model-infra